

Executive strategy for unifying your data



Summarize this article for me

Data goal: Every organization wants its data to drive confident business decisions. Data must be trusted, easy to reuse for analytics and AI, and secure by default. **Data challenge:** For most organizations, this goal is hard to achieve. Data is spread across systems and teams. Standards vary. Governance is inconsistent. These issues make analytics and AI difficult to use with confidence.

Data solution: Many organizations are addressing this challenge by unifying their data platform with Microsoft Fabric. Fabric enables teams to create trusted data products that can be governed and securely used for analytics and AI across the organization (see Figure 1). This guidance provides decision makers with the framework they need to make that shift and establish a unified data foundation.

Microsoft's Cloud Adoption Framework
Unified data platform for AI and analytics (conceptual architecture)

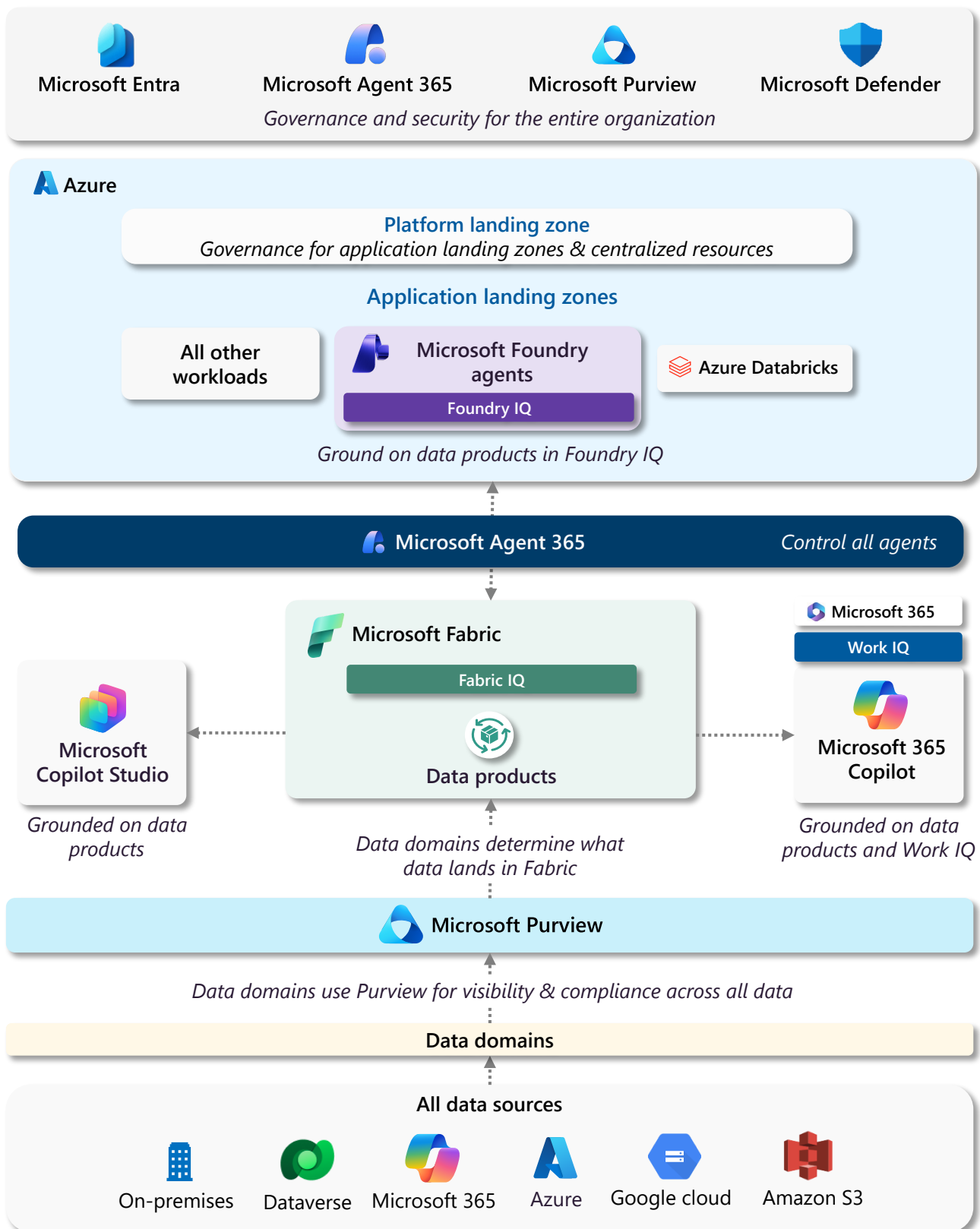


Figure 1. Unified data platform for AI and analytics.

Why a unified data platform with Fabric?

Most business and technology leaders understand the cost of fragmented data. What often holds them back is the belief that fixing it requires large, risky migrations. Microsoft Fabric takes a different approach and delivers value without disruption. Key benefits include:

- **Minimal business disruption:** Fabric connects to existing systems by using virtualization (shortcuts) and selective replication (mirroring). Teams can unify access to data without interrupting current operations.
- **Built-in governance:** Fabric brings data engineering, analytics, and BI into a single platform. Security and governance policies are defined once and applied consistently, instead of being recreated and enforced differently across multiple tools.
- **Foundation for AI and analytics:** Fabric enables organizations to produce reusable, high-quality data products. These trusted products speed up analytics and AI initiatives. [Fabric IQ](#) helps unify and contextualize data. [Foundry IQ](#) allows Microsoft Foundry agents to reason over governed, trusted data.

What level of investment is required?

Unifying the data platform is an investment in capability, not a wholesale replacement of every system. The goal is to keep using your existing data systems and build a shared foundation that can grow over time. Key cost factors include:

- **Microsoft Fabric cost factors:** Primary [cost factors](#)  include (see Figure 2):
 - **Compute:** The compute capacity you create (Fabric capacities).
 - **Storage:** The storage you use in OneLake.
 - **Replication:** The data replication you perform (Mirroring).
 - **Power BI:** Ensure that users have either sufficient Microsoft Fabric capacity that includes Power BI access or separate Power BI [licenses](#), as summarized in the licensing guidance.

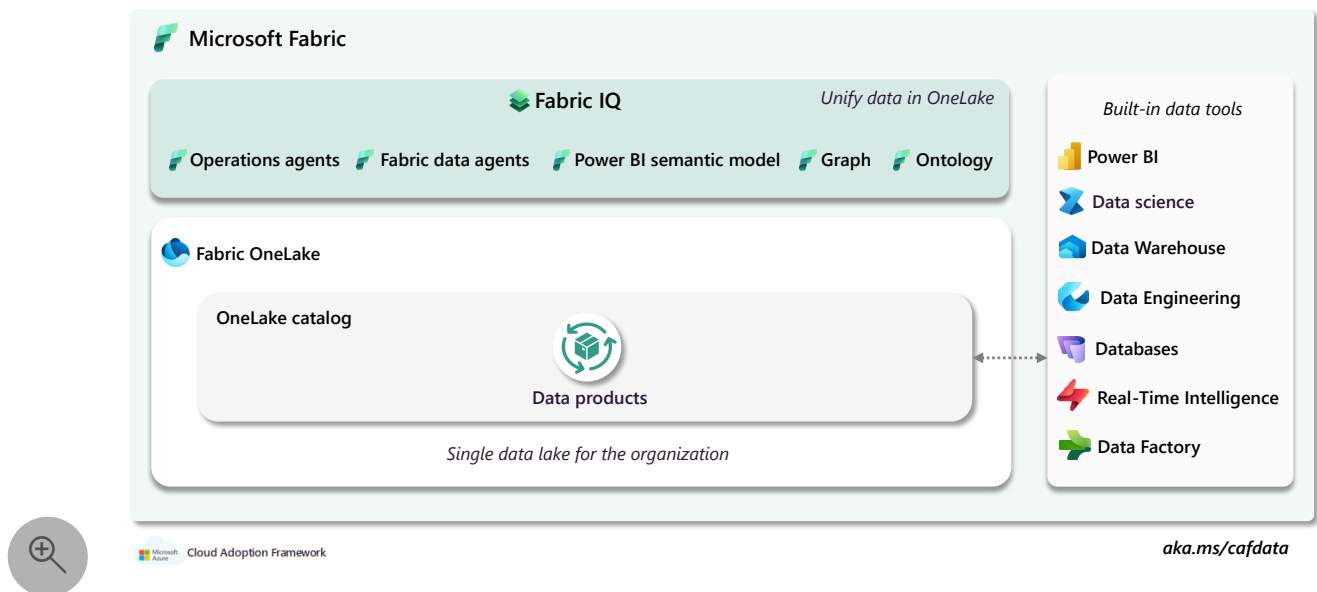


Figure 2. Microsoft Fabric capabilities to create business value from data.

- **Microsoft Purview cost factors:** Use Microsoft Purview for unified data governance and compliance. Purview provides a centralized data catalog, data classification, and policy enforcement across your entire data estate. The data can be in OneLake, Azure, on-premises, third-party SaaS, or other cloud platforms. Key [Purview cost factors](#) include [subscription-based licensing](#) and [consumption-based capabilities](#). Budget for both ongoing licensing and the volume of data and services you govern with Purview.
- **Azure cost factors:** You use Azure subscriptions to host Fabric compute (capacities) and your Microsoft Purview account. There's no extra cost for Azure subscriptions. If you integrate other Azure services, such as Azure Databricks or Azure Machine Learning, into your unified platform, remember these services have their own pricing models. Plan for those costs. See cost factors for [Azure Databricks](#) and [Azure Machine Learning](#).

How long until you see value?

Microsoft Fabric is designed to deliver value quickly. Time to value is short because unification doesn't depend on full migration. Teams can start with a small set of high-value data products. Each step adds value while limiting risk. In practice, many organizations see value within weeks for initial analytics or AI scenarios. As Fabric becomes the standard foundation for data products, analytics, and AI, value grows through reuse and consistent standards across the organization.

How do you unify your data platform?

Microsoft's Cloud Adoption Framework outlines a four-step framework for unifying your data platform. The process spans planning and organizing your data strategy. It covers architecture

decisions. It also helps you set governance and security baselines and define operational standards.

1. **Organizational readiness.** Define your data strategy and establish data ownership and domains. Clarify how data creates business value and who is accountable for which data. See [Organizational readiness](#).
2. **Architecture:** Provide the technology needed to unify your data platform. Set up Microsoft Fabric and required environments in Azure. See [Architecture](#).
3. **Governance and security baselines:** Use Microsoft Purview to gain central visibility and governance across your data estate. Build security and compliance baselines into your Fabric architecture from the start. See [Governance and security baselines](#).
4. **Operational standards.** Define consistent processes for ingesting raw data, creating data products, and managing their lifecycle. Establish how data products are published, secured, and consumed across the organization. See [Operational standards](#).

By following these steps, you can unify your data platform in a structured way. If you don't know where to start, use the following decision tree for guidance.

Decision tree for unifying your data platform

Microsoft's Cloud Adoption Framework
Decision tree: Unify your data platform for AI and analytics

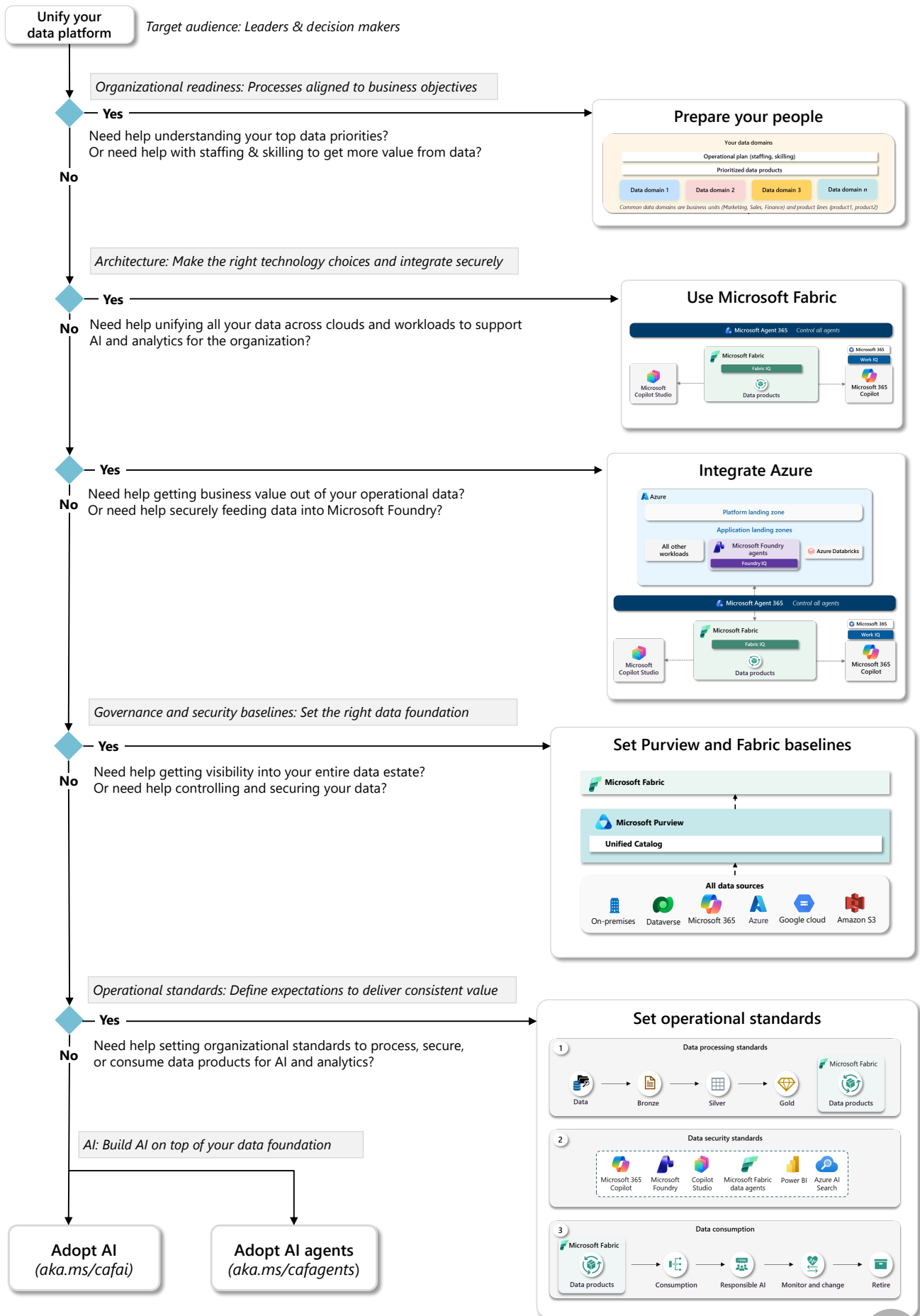


Figure 3. Microsoft's decision tree for unifying your data platform.

Next step

In the following sections, you'll find guidance, checklists, best practices, decision guidance, and trade-offs at each step. The guidance is for leaders and decision makers who oversee organizational strategy and governance.

Organizational readiness

Key terms

 Expand table

Key term	Definition
Analytics	The practice of generating insights from data to support decision-making. It includes dashboards, reports, and visualizations, for example, in Power BI .
AI	Systems that use data as input into models that automate business functionality. This category includes traditional machine learning models (predictive) and generative AI models.
Data product	Data that's in a form that's valuable for your business, such as datasets, tables, feature sets, or AI training data.
Data domain	A boundary of responsibility and ownership for data products, such as business units (HR, Marketing, Finance, Sales, Operations) and product lines (Product 1, Product 2).
Data management landing zone	An environment (consisting of one or more Azure subscriptions) for data management resources, such as Microsoft Purview accounts and Fabric capacities.
Data landing zone	An environment (consisting of one or more Azure subscriptions) for data and AI/ML resources, such as Azure Databricks, Azure Data Lake Storage, and Azure Machine Learning.

Last updated on 03/11/2026

Executive strategy for unifying your data



Summarize this article for me

Data goal: Every organization wants its data to drive confident business decisions. Data must be trusted, easy to reuse for analytics and AI, and secure by default. **Data challenge:** For most organizations, this goal is hard to achieve. Data is spread across systems and teams. Standards vary. Governance is inconsistent. These issues make analytics and AI difficult to use with confidence.

Data solution: Many organizations are addressing this challenge by unifying their data platform with Microsoft Fabric. Fabric enables teams to create trusted data products that can be governed and securely used for analytics and AI across the organization (see Figure 1). This guidance provides decision makers with the framework they need to make that shift and establish a unified data foundation.

Microsoft's Cloud Adoption Framework
Unified data platform for AI and analytics (conceptual architecture)

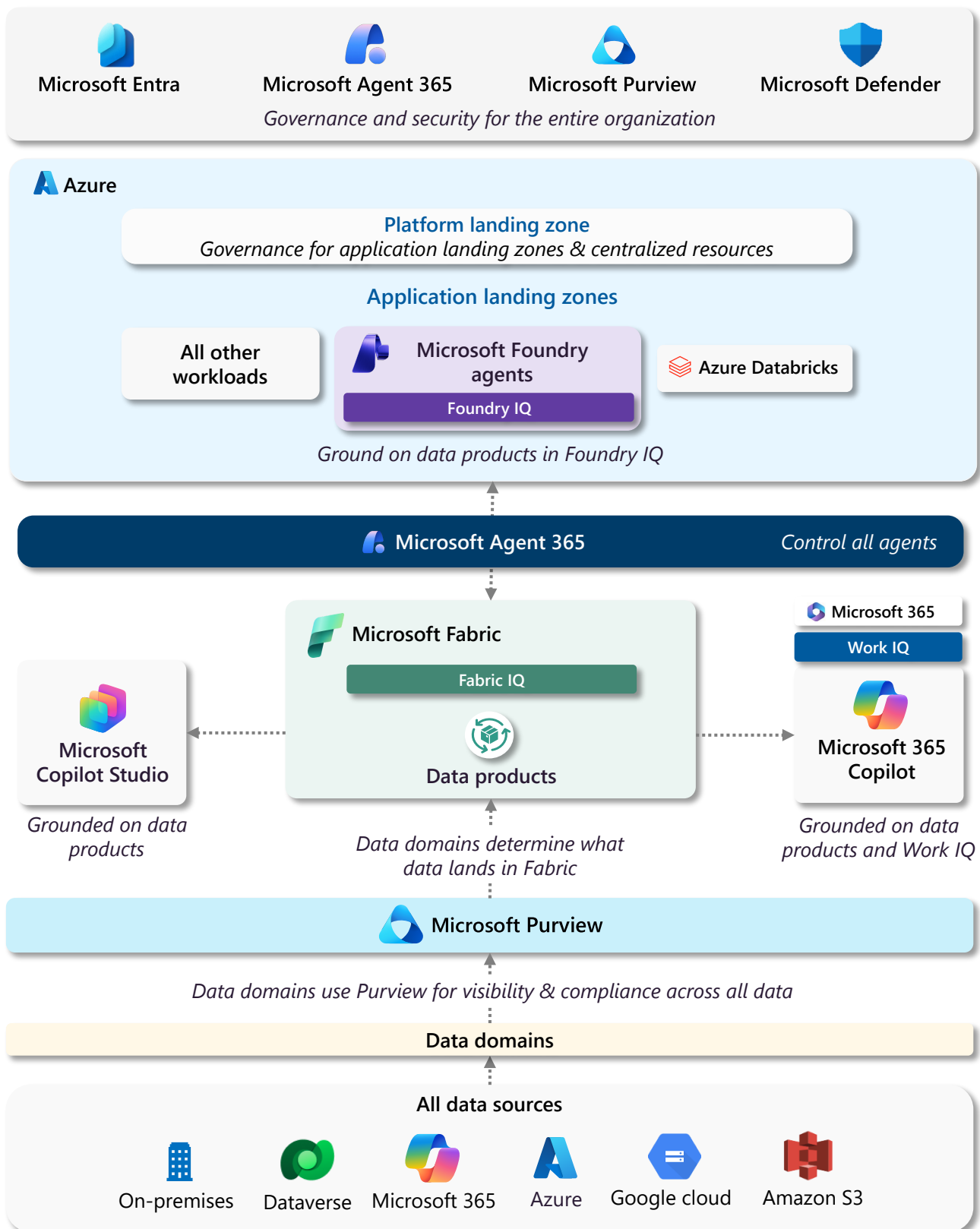


Figure 1. Unified data platform for AI and analytics.

Why a unified data platform with Fabric?

Most business and technology leaders understand the cost of fragmented data. What often holds them back is the belief that fixing it requires large, risky migrations. Microsoft Fabric takes a different approach and delivers value without disruption. Key benefits include:

- **Minimal business disruption:** Fabric connects to existing systems by using virtualization (shortcuts) and selective replication (mirroring). Teams can unify access to data without interrupting current operations.
- **Built-in governance:** Fabric brings data engineering, analytics, and BI into a single platform. Security and governance policies are defined once and applied consistently, instead of being recreated and enforced differently across multiple tools.
- **Foundation for AI and analytics:** Fabric enables organizations to produce reusable, high-quality data products. These trusted products speed up analytics and AI initiatives. [Fabric IQ](#) helps unify and contextualize data. [Foundry IQ](#) allows Microsoft Foundry agents to reason over governed, trusted data.

What level of investment is required?

Unifying the data platform is an investment in capability, not a wholesale replacement of every system. The goal is to keep using your existing data systems and build a shared foundation that can grow over time. Key cost factors include:

- **Microsoft Fabric cost factors:** Primary [cost factors](#)  include (see Figure 2):
 - **Compute:** The compute capacity you create (Fabric capacities).
 - **Storage:** The storage you use in OneLake.
 - **Replication:** The data replication you perform (Mirroring).
 - **Power BI:** Ensure that users have either sufficient Microsoft Fabric capacity that includes Power BI access or separate Power BI [licenses](#), as summarized in the licensing guidance.

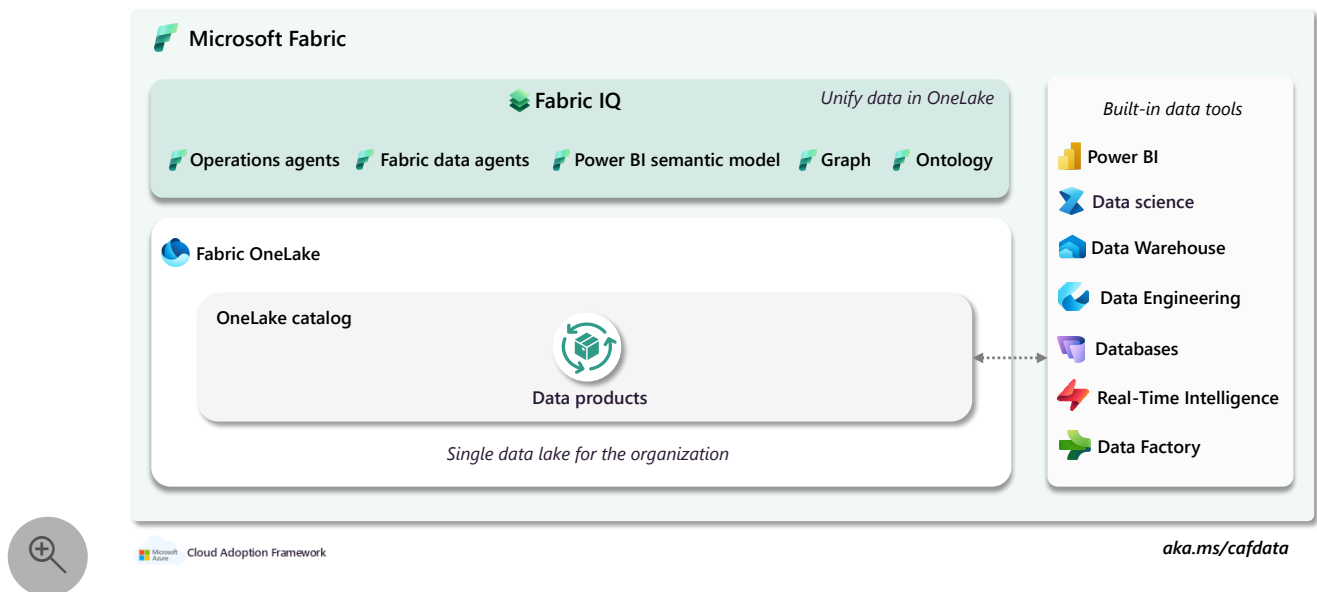


Figure 2. Microsoft Fabric capabilities to create business value from data.

- **Microsoft Purview cost factors:** Use Microsoft Purview for unified data governance and compliance. Purview provides a centralized data catalog, data classification, and policy enforcement across your entire data estate. The data can be in OneLake, Azure, on-premises, third-party SaaS, or other cloud platforms. Key [Purview cost factors](#) include [subscription-based licensing](#) and [consumption-based capabilities](#). Budget for both ongoing licensing and the volume of data and services you govern with Purview.
- **Azure cost factors:** You use Azure subscriptions to host Fabric compute (capacities) and your Microsoft Purview account. There's no extra cost for Azure subscriptions. If you integrate other Azure services, such as Azure Databricks or Azure Machine Learning, into your unified platform, remember these services have their own pricing models. Plan for those costs. See cost factors for [Azure Databricks](#) and [Azure Machine Learning](#).

How long until you see value?

Microsoft Fabric is designed to deliver value quickly. Time to value is short because unification doesn't depend on full migration. Teams can start with a small set of high-value data products. Each step adds value while limiting risk. In practice, many organizations see value within weeks for initial analytics or AI scenarios. As Fabric becomes the standard foundation for data products, analytics, and AI, value grows through reuse and consistent standards across the organization.

How do you unify your data platform?

Microsoft's Cloud Adoption Framework outlines a four-step framework for unifying your data platform. The process spans planning and organizing your data strategy. It covers architecture

decisions. It also helps you set governance and security baselines and define operational standards.

1. **Organizational readiness.** Define your data strategy and establish data ownership and domains. Clarify how data creates business value and who is accountable for which data. See [Organizational readiness](#).
2. **Architecture:** Provide the technology needed to unify your data platform. Set up Microsoft Fabric and required environments in Azure. See [Architecture](#).
3. **Governance and security baselines:** Use Microsoft Purview to gain central visibility and governance across your data estate. Build security and compliance baselines into your Fabric architecture from the start. See [Governance and security baselines](#).
4. **Operational standards.** Define consistent processes for ingesting raw data, creating data products, and managing their lifecycle. Establish how data products are published, secured, and consumed across the organization. See [Operational standards](#).

By following these steps, you can unify your data platform in a structured way. If you don't know where to start, use the following decision tree for guidance.

Decision tree for unifying your data platform

Microsoft's Cloud Adoption Framework
Decision tree: Unify your data platform for AI and analytics

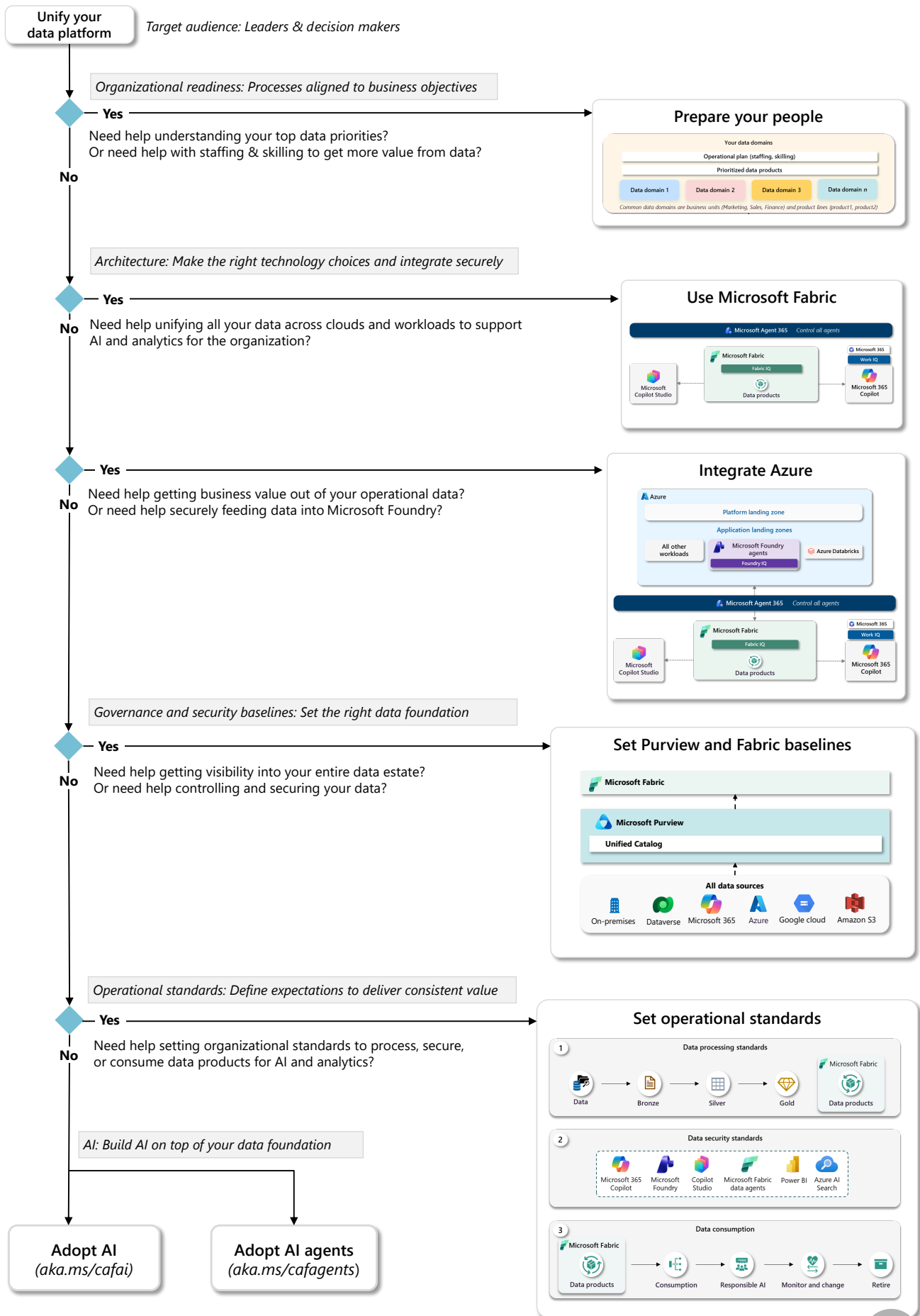


Figure 3. Microsoft's decision tree for unifying your data platform.

Next step

In the following sections, you'll find guidance, checklists, best practices, decision guidance, and trade-offs at each step. The guidance is for leaders and decision makers who oversee organizational strategy and governance.

Organizational readiness

Key terms

 Expand table

Key term	Definition
Analytics	The practice of generating insights from data to support decision-making. It includes dashboards, reports, and visualizations, for example, in Power BI .
AI	Systems that use data as input into models that automate business functionality. This category includes traditional machine learning models (predictive) and generative AI models.
Data product	Data that's in a form that's valuable for your business, such as datasets, tables, feature sets, or AI training data.
Data domain	A boundary of responsibility and ownership for data products, such as business units (HR, Marketing, Finance, Sales, Operations) and product lines (Product 1, Product 2).
Data management landing zone	An environment (consisting of one or more Azure subscriptions) for data management resources, such as Microsoft Purview accounts and Fabric capacities.
Data landing zone	An environment (consisting of one or more Azure subscriptions) for data and AI/ML resources, such as Azure Databricks, Azure Data Lake Storage, and Azure Machine Learning.

Last updated on 03/11/2026

Organizational readiness for unifying your data platform

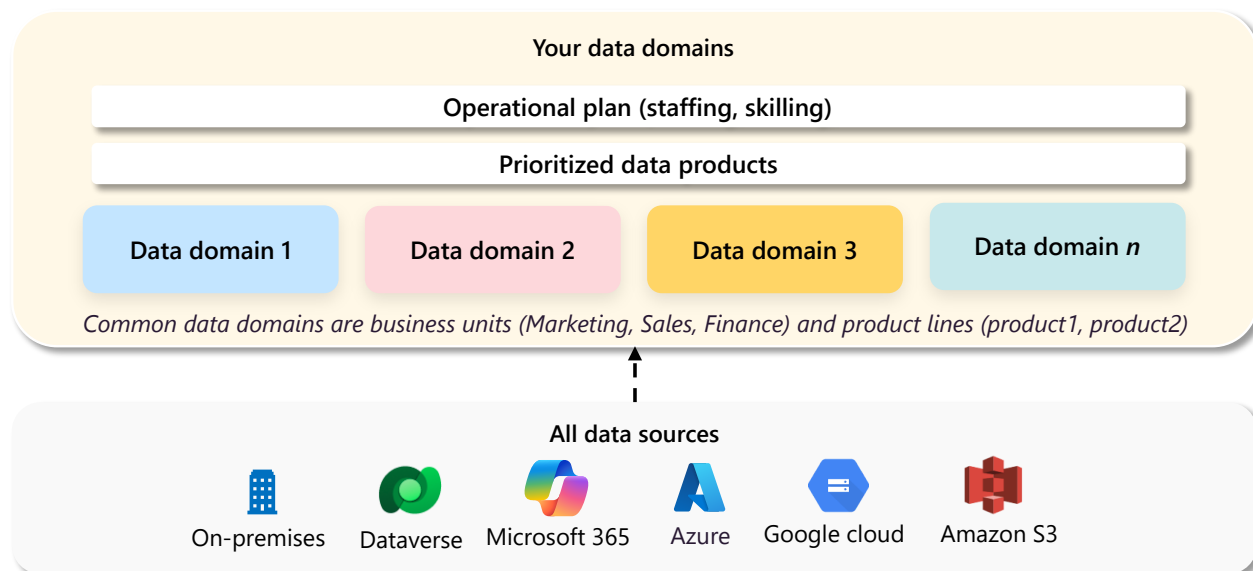


Summarize this article for me

Organizational readiness focuses on aligning data initiatives with business priorities. Rather than starting with technology, start by defining *who* owns the data and *why* it matters to the business. **Recommendation:** Make sure business value drives data product choices instead of unplanned technical decisions. To apply this recommendation, use this article as a checklist:

1. Identify data domains

The first step is to define how your organization assigns responsibility for data. Many successful organizations use data domains. Data domains are boundaries of data responsibility. They determine who owns data and why that data exists from a business perspective. Common data domains are business units (HR, Sales, Marketing, Finance, Operations) or product lines (Product 1, Product 2). **Recommendation:** Define data domains that reflect how the organization makes decisions so accountable leaders own data products end to end (see Figure 1). To apply this recommendation, use the following checklist:



Microsoft Cloud Adoption Framework

aka.ms/cafddata

Figure 1. Overview of data domains.

1. **Align data domains to the organizational structure.** Data domains represent boundaries where business decisions occur and where authority already exists. A well-defined domain matches how leaders allocate budget, set priorities, and accept risk. **Best practices:** Map domains to business units or product lines that already own outcomes. Make sure each domain can define data meaning and quality without external approval. Publish domain-

owned data products through the OneLake catalog for enterprise reuse. **Decision guidance:** Align domains to existing business structures when authority already exists. Redefine domains only when no leader can make end-to-end decisions for the data.

- 2. **Appoint data domain leadership.** Each data domain requires a senior leader who represents business intent for that data. This leader owns definitions, quality expectations, and sharing decisions. **Best practices:** Assign a leader who understands both business outcomes and data impact. Grant authority to approve what the domain shares through the OneLake catalog. Make this accountability visible in governance forums. **Decision guidance:** Choose a leader with decision authority over outcomes, not a technical owner. Revisit domain boundaries if no leader can accept accountability.
- 3. **Define data domain responsibilities.** After establishing who owns each data domain, document what each domain is responsible for. Data domain ownership covers planning, delivery, operations, cost control, and policy compliance for data products. Clear responsibility prevents escalation and delays. **Best practices:** Enable domains to manage their own resources within enterprise guardrails. Allow domains to control Fabric capacity usage and prioritization within approved limits. **Decision guidance:** Grant autonomy when domains demonstrate ownership maturity. Choose tighter guardrails when regulatory or cost risk requires centralized oversight.
- 4. **Adjust data domains over time.** Stable domains reduce friction, but misaligned domains create constant negotiation. Domain changes restore decision clarity when ownership no longer reflects reality. **Best practices:** Monitor where data decisions require repeated cross-domain arbitration. Treat that signal as a need to revisit boundaries. Adjust early to avoid governance bottlenecks. **Decision guidance:** Keep domains stable when decisions stay local. Realign domains when authority fragments or accountability becomes unclear.

2. Define data products

After you establish domains and their leadership, shift your focus from organizational structure to business outcomes. Now, determine *what data products each domain should create*. Data products translate business priorities into durable data assets that domains own and publish for reuse. A clear definition ensures investment decisions focus on outcomes that support analytics and AI across Microsoft data platforms. **Recommendation:** Define and prioritize data products only when leaders can trace each product to a business goal and an accountable data domain. To apply this recommendation, use the following checklist:

Table 1: Example template for defining a data product.

Business goal	Data domain: data domain goals	Data domain goal priority	Data products required (consumer)	Required data assets for data product
Increase Sales	Sales: Close deals faster and improve win rates.	High	Current sales pipeline (analytics) Customer purchase behavior (ML) Sales playbooks (AI agents)	CRM opportunity records from Azure SQL or Microsoft Dataverse
Improve customer satisfaction	Product 1: Improve product performance	High	Historical product metrics (analytics) Product demand (ML) Product documentation (AI agent)	Application telemetry from Azure Monitor, Application Insights
Reduce Costs	Finance: Improve budget accuracy and reduce reporting time	Medium	Financial actuals (analytics) Cost drivers (ML) Finance policies (AI agents)	General ledger transactions from Azure SQL or Fabric Warehouse

1. **Anchor data products to business goals.** Business goals define what success means for the organization over the near and medium term. Clear goals prevent investment in data assets that lack material impact. **Best practices:** Ask senior leadership to state current priorities in plain business terms. Use these priorities as the filter for all data product proposals. Reject proposals that don't affect goal achievement. **Decision guidance:** Decide to approve data work only when leaders can explain how the data product changes business outcomes. Choose to defer data products that lack a measurable link to stated goals.
2. **Identify data products per domain.** Data domains translate business goals into domain-specific data products. The focus shifts from existing data to required information. **Best practices:** Require each domain leader to answer one question: Which data products must this domain deliver to support the stated business goals? Allow proposals only when the absence of the product would slow progress toward the goal. **Decision guidance:** Decide to fund data products that remove decision friction or unlock automation. Choose to deprioritize products that provide convenience but no outcome impact.
3. **Assess requirements for each data product.** For every proposed data product, determine what it takes to make it real and trustworthy. This assessment includes identifying:

- **Data sources:** What source systems and datasets are needed? Where do these data currently reside, and how are they generated?
- **Ownership:** Who owns this data product? Identify data owners, and clarify who is accountable for the quality and maintenance of the data product over time.
- **Capabilities needed:** Determine what capabilities are required to produce and maintain the data product, such as data engineering, integration, data modeling, analytics, and machine learning.
- **Tools and platforms:** Identify the technologies and services required. For example, do you only need Fabric? Do you need Azure Databricks for certain transformations or machine learning? What about reporting tools like Power BI or other analytics platforms?

This assessment might reveal gaps in your current state, like missing data sources, data quality problems, or lacking tooling and skills. It's okay and expected. The point is to map out what needs to be done to create each high-value data product so you can plan effectively.

AI tip: Use Copilot or Researcher to draft an initial outline of data sources and ownership for a proposed data product. Treat the output as a starting point and require domain review before approval.

4. **Prioritize data products.** Now that you have a clear list of candidate data products and their requirements, leadership decides which ones to do first. You can't do everything at once. **Best practices:** Rank data products by business urgency first. Identify products that create shared foundations. Balance fast delivery with long-term value. Respect explicit dependencies between products. **Decision guidance:**

- **Business urgency:** Is the data product needed for an imminent initiative or deadline?
- **Foundational value:** Does this data product provide a core data asset that supports many future analytics and AI use cases? These foundational data products, while possibly more time-consuming to build, unlock value across multiple teams once they exist.
- **Quick wins vs. strategic builds:** Some data products might be delivered quickly and show immediate value (quick wins). Others are strategic and provide long-term benefits but might require more time. Balance your portfolio to include both types, aligned with your current priorities.
- **Dependencies:** Are certain data products prerequisites for others? Prioritize data products that enable subsequent work.

5. **Review the data product portfolio regularly.** Business priorities evolve and data portfolios must adapt without uncontrolled growth. **Best practices:** Establish a fixed review cadence with domain leaders and stakeholders. Retire products that no longer serve a goal. Add new proposals only when strategy changes. **Decision guidance:** Keep products active when they support current goals. Retire or pause products when value declines or ownership weakens.

Outcome: As a result of this step, you should have a roadmap of data products. Each data product is tied to business goals with clear requirements and a sense of priority. Each data product should either be approved to move forward now or marked as deferred. Document reasons, such as waiting on data quality improvements or dependent systems.

3. Organize data domains

Clear domain boundaries and a prioritized set of data products help leaders resource domains with intent and accountability. This guidance helps decision makers align team structure and skills with domain goals so that Microsoft and Azure data investments deliver sustained value. **Recommendation:** Resource each data domain based on its mission and maturity, and embed skill development into domain delivery plans. To apply this recommendation, use the following checklist:

1. Staff the data domain

Data domains differ in size, risk, and business impact. Leaders must set staffing models that fit each domain's mission while staying within budget and governance constraints.

Recommendation: Assign accountability for domain staffing to the data domain lead and require an explicit staffing decision for every domain. To apply this recommendation, use the following checklist:

1. **Use a dedicated team when speed and context matter.** A dedicated team serves one domain and stays fully aligned to its data products and consumers. **Best practices:** Assign dedicated teams to domains with high strategic value, strong regulatory exposure, or rapid iteration needs. This model supports faster decisions and deeper domain expertise. **Decision guidance:** Choose a dedicated team when the domain drives core business outcomes and requires continuous delivery. Accept higher resource cost in exchange for responsiveness and focus.
2. **Use shared teams when scale and consistency matter.** A shared team supports multiple domains through a central or pooled model. **Best practices:** Use shared teams to concentrate scarce skills and enforce standards across data domains. This model improves efficiency and reuse. **Decision guidance:** Choose a shared team when skills are limited or

when standardization outweighs delivery speed. Expect queue pressure and reduced domain agility.

3. **Apply a hybrid model across the portfolio.** Different data domains require different staffing models at the same time. **Best practices:** Assign dedicated teams to large or high-priority data domains and shared services to smaller or emerging data domains. Governance bodies review these assignments on a regular cadence. **Decision guidance:** Decide which domains qualify for dedicated capacity and define clear entry and exit criteria. Revisit decisions as domain maturity and demand change.
4. **Define a clear request path for dedicated resources.** Domains need a transparent way to request staffing changes. **Best practices:** Require a lightweight business case that explains value, urgency, and risk. Central governance reviews requests against budget and strategic alignment. **Decision guidance:** Choose strict approval when resources are constrained. Choose a lighter review when speed and innovation take priority.

2. Build data skills

Data capability doesn't grow without deliberate investment, and leaders set priorities that either enable or block skill development. **Recommendation:** Make data skill development part of normal delivery expectations and hold data domains accountable for outcomes. To apply this recommendation, use the following checklist:

1. **Adopt planned, domain-led learning as the default.** Data domains own the skills needed to build and operate their data products. **Best practices:** High-performing organizations tie learning goals to active domain work. Central teams provide standards, learning paths, and communities without taking ownership away from domains. **Decision guidance:** Choose this option when adoption, quality, and decision confidence matter. Accept slower initial delivery in exchange for better outcomes over time.
2. **Limit ad hoc learning to exceptional cases.** Unplanned learning depends on individual effort and available time. **Best practices:** Avoid this model except during short-term delivery spikes. Leaders recognize that skills stagnate under constant pressure. **Decision guidance:** Choose this option only when immediate delivery outweighs long-term value. Expect fragile solutions and limited reuse.
3. **Centralized training.** Use centralized training to set a baseline. Central programs establish a shared understanding across domains. **Best practices:** Use centralized training to support consistency and onboarding. Leaders don't rely on courses alone to change behavior. **Decision guidance:** Choose this option when standard knowledge matters more than applied change. Pair it with domain-led learning to affect outcomes.

Use training resources

Support your teams with relevant training materials and certifications. Microsoft offers a range of learning resources tailored to different roles and technologies. Some examples include:

 Expand table

Category	Focus area	Intended audience	When to use
Training paths	Microsoft Fabric.	Data engineers, analysts, and platform teams	When teams are adopting or expanding Fabric workloads
	Azure Databricks 	Data engineers, data scientists	When building or scaling advanced analytics and engineering pipelines
	Power BI	Analysts, business users	When improving reporting, self-service analytics, and adoption
	Training for Data Analysts	Data analysts	When strengthening analytical skills and business insight
	Data Scientists	Data scientists	When developing advanced modeling and AI capabilities
Role-based certifications	Fabric Analytics Engineer Associate and study guide	Analytics engineers	When validating analytics engineering skills and platform usage
	Fabric Data Engineer Associate and study guide	Data engineers	When standardizing data engineering skills on Fabric
	Azure AI Engineer and study guide	AI and ML engineers	When building or operationalizing AI solutions
	Power Platform Solution Architect and study guide	Solution architects	When designing end-to-end data and Microsoft Power Platform solutions

Next step

By planning your organizational readiness in these ways, you create a solid foundation. This planning makes sure that your technology investments in a unified data platform have a clear purpose.

Last updated on 03/10/2026

Fabric architecture for a unified data platform



Summarize this article for me

After aligning on why and what data products you need (organizational readiness), the next step is to decide how the technology fits together. This step involves creating an architecture that connects and integrates Microsoft Fabric with Azure.

This article helps you make the right architecture decisions for designing **Microsoft Fabric** as the foundation of your unified data platform. Microsoft Fabric provides a data platform with a built-in SaaS data lake called [OneLake](#) that centralizes data for analytics and AI across the organization.

Recommendation: Adopt Microsoft Fabric with OneLake as the default analytics data platform and define clear leadership decisions for compute, regions, recovery, and workspace ownership. This setup enables all workload teams to operate within consistent standards (see Figure 1). To apply this recommendation, use this article as a checklist:

Unified data platform architecture

Figure 1 shows an integrated Fabric-in-Azure architecture aligned to data domains, with enough operational structure to make the design concrete. It serves as a reference model to help organizations design and standardize their architecture.

Microsoft's Cloud Adoption Framework
Unified data platform for AI and analytics (architecture)

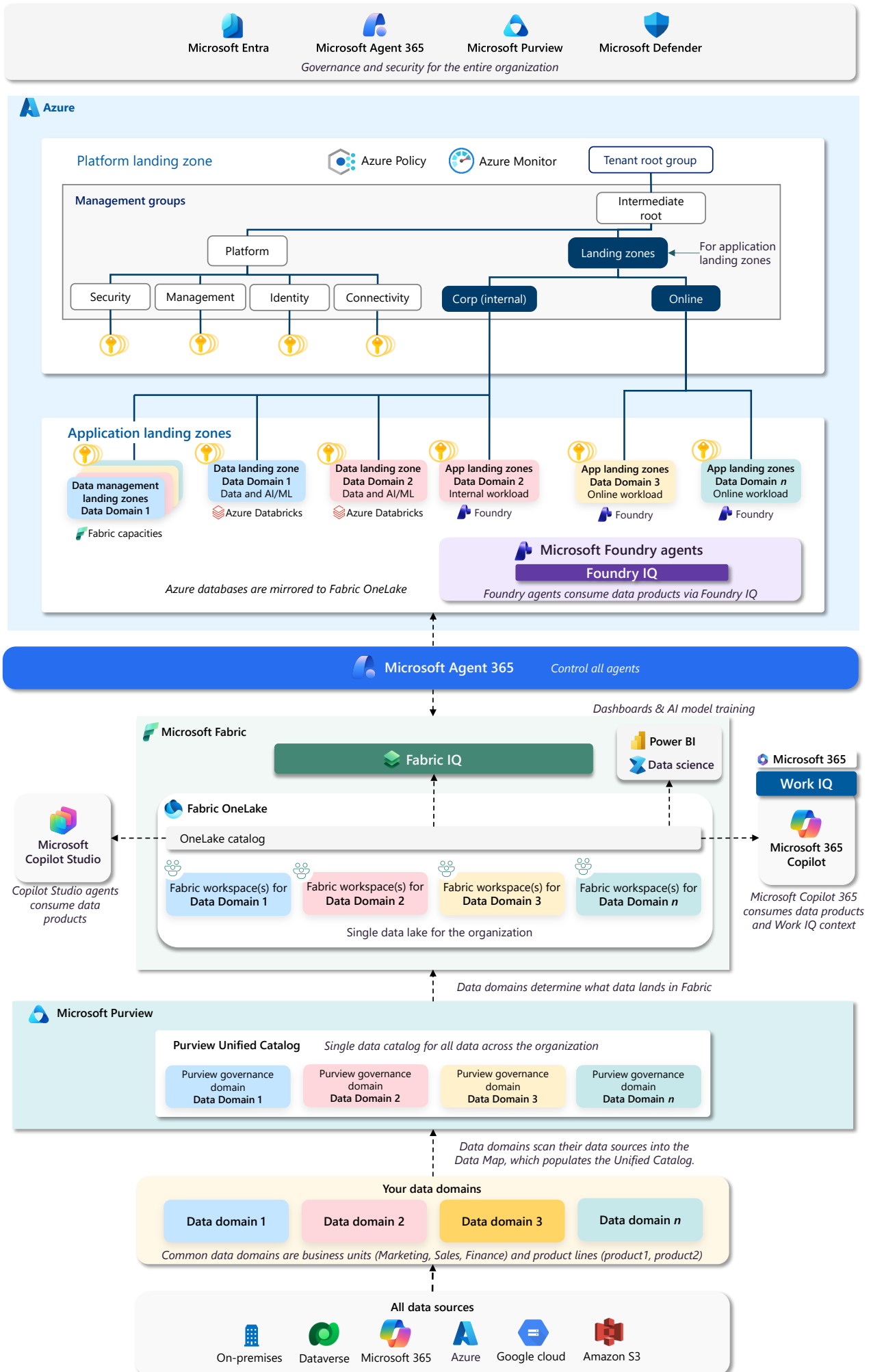


Figure 1. Architecture: Unified data platform for AI and analytics.

1. Plan for Fabric compute

Fabric compute runs on capacity, which defines available resources, performance, and concurrency for all workloads that share that capacity. Capacity choices directly affect cost control and service reliability. **Best practices:** Allocate budgets to data domain teams and allow those teams to own Fabric capacities that match their business criticality. Create each Fabric capacity in a data management landing zone. To learn more, see [Data management landing zones](#).

2. Plan your Fabric regions

Each Fabric capacity runs in a single Azure region. That region determines where compute and OneLake data for workspaces on that capacity reside. Region decisions affect data residency, latency, and service availability. **Best practices:** treat region selection as a governance decision. Publish a short list of supported regions and enforce that list through policy. **Decision guidance:** Decide whether to use [Multi-Geo](#). Choose Multi-Geo when the organization needs formal governance across regions and clear rules for why regions exist. Multi-Geo is unnecessary when operations remain confined to a single geography.

3. Plan your Fabric workspaces

A [Fabric workspace](#) groups data assets and acts as the primary boundary for security, administration, and cost visibility. Workspace design shapes how domains operate inside OneLake. **Best practices:** Give each data domain one or more dedicated workspaces to manage its data products. Apply access control, cost tracking, and operational policies at the workspace level. Use [Fabric domains](#) to apply shared governance across related workspaces (see Figure 2).

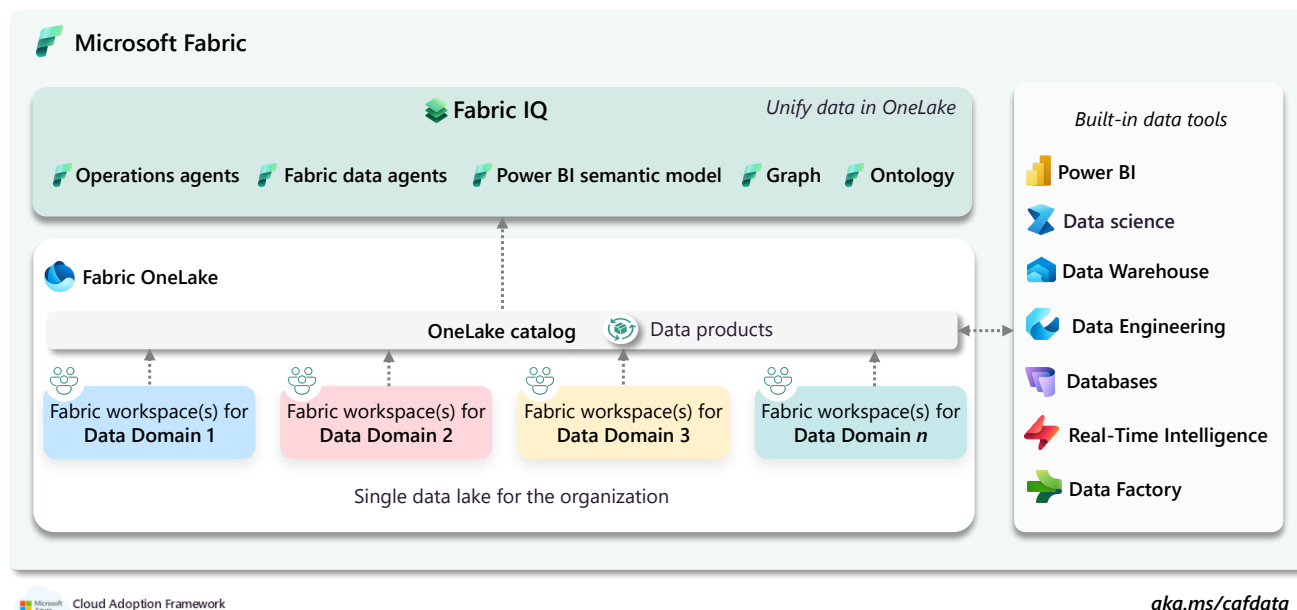


Figure 2. Microsoft Fabric architecture.

4. Plan for Fabric IQ

Fabric IQ provides a semantic intelligence layer over OneLake. This layer defines shared business concepts that analytics tools and AI agents can interpret consistently. **Best practices:** Use Fabric IQ to provide shared meaning across data domains. Focus on scenarios where shared definitions reduce reporting risk or improve AI outcomes. **Decision guidance:** See the following guidance to [choose the right Fabric IQ capability](#).

5. Plan for Fabric disaster recovery

Fabric disaster recovery copies OneLake data to a paired Azure region. This capability supports workload recovery after a regional outage. **Best practices:** Treat Fabric disaster recovery as a business continuity decision. Enable disaster recovery where business impact justifies. **Decision guidance:** Decide to enable disaster recovery when downtime creates material risk. Choose not to enable it for exploratory or noncritical workloads. To learn more, see [OneLake Disaster Recovery and Data Protection](#).

Next step

Azure architecture for a unified data platform

Azure architecture for a unified data platform



Summarize this article for me

After you align on why and what data products you need (organizational readiness), the next step is to decide how the technology fits together. Create an architecture that connects and integrates Microsoft Fabric with Azure and your existing systems. As a decision maker, you must design Azure environments that support a unified data platform so that your organization can standardize how data flows from operational systems into analytics and AI consumption.

Recommendation: Establish architecture patterns for your Azure environments. Include data management landing zones, application landing zones, and data landing zones. To apply this recommendation, use this article as a checklist:

Unified data platform architecture

Figure 1 shows an integrated Fabric-in-Azure architecture aligned to data domains, with enough operational structure to make the design concrete. It serves as a reference model to help organizations design and standardize their architecture.

Microsoft's Cloud Adoption Framework
Unified data platform for AI and analytics (architecture)

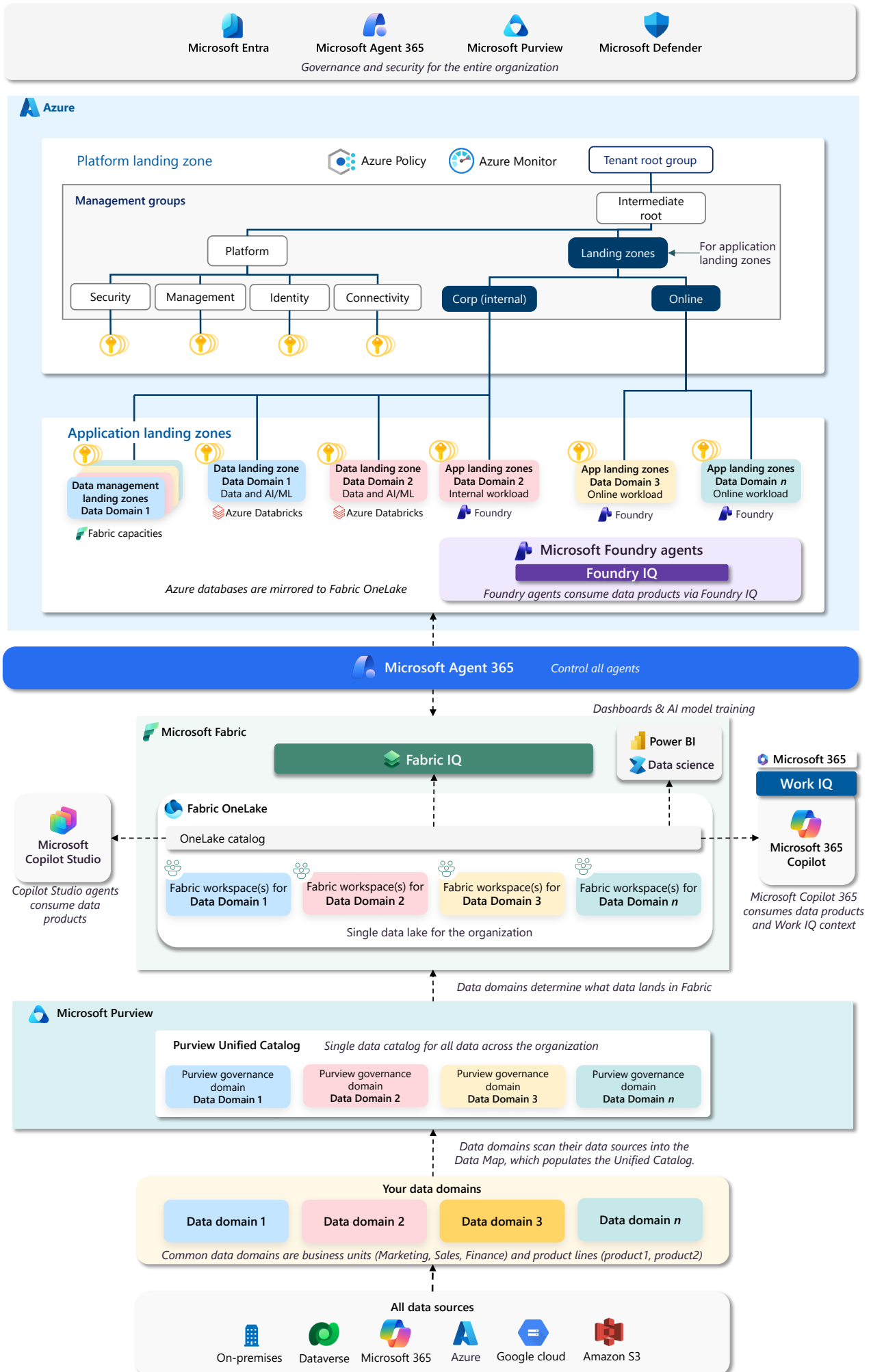
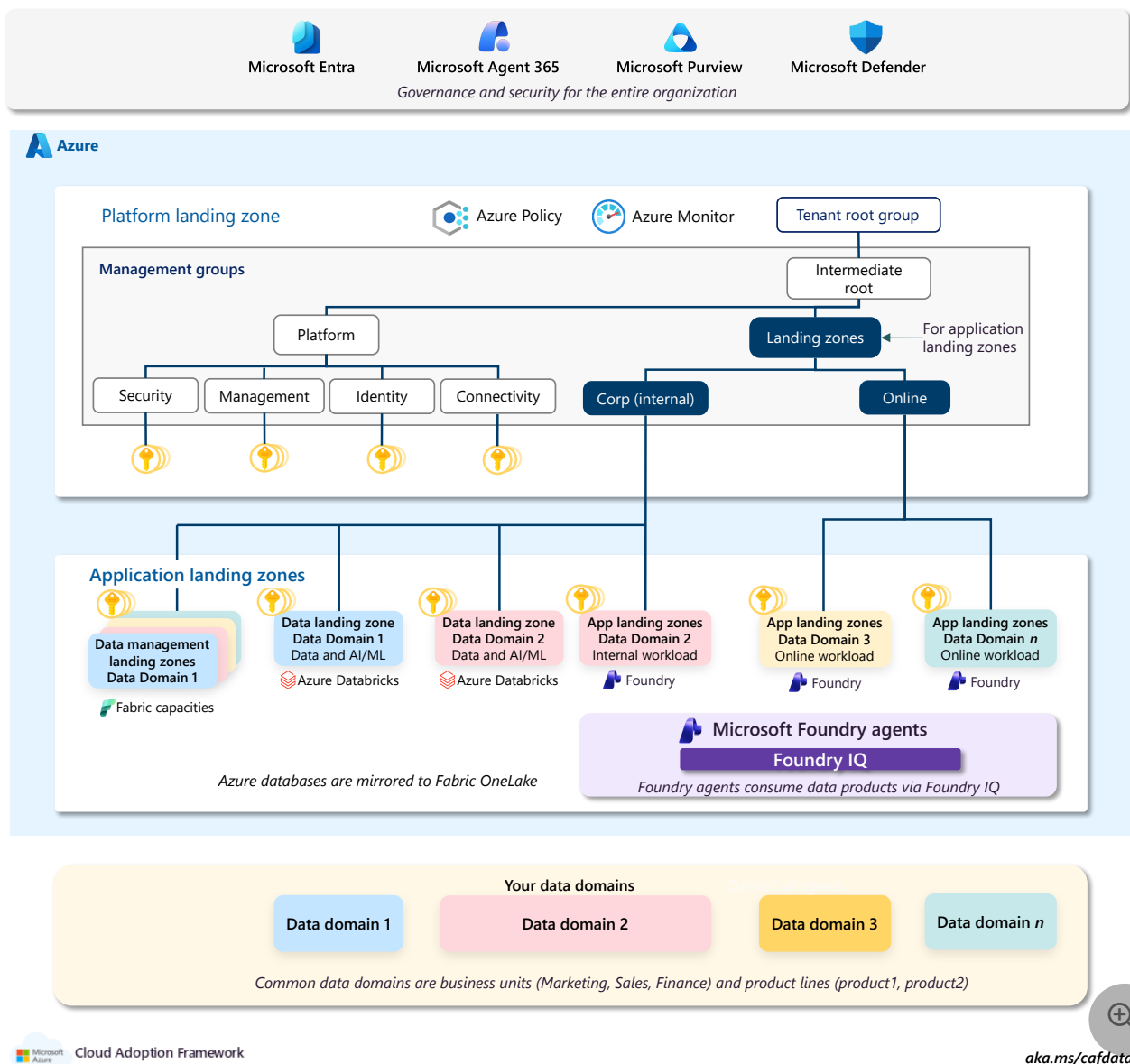


Figure 1. Architecture: Unified data platform for AI and analytics.

1. Plan your data management landing zones

A data management landing zone (DMLZ) is one or more Azure subscriptions for data management resources. **Recommendation:** Use DMLZs to host your Microsoft Purview account and Microsoft Fabric capacity. To apply this recommendation, use the following checklist:

*Microsoft's Cloud Adoption Framework
Azure architecture for a unified data platform*



1.1 Place Microsoft Purview account in a data management landing zone

A Microsoft Purview account provides organization-wide data governance and catalog capabilities. Its placement defines ownership boundaries and governance scope across the

tenant. **Best practices:** If you use Microsoft Purview for governance, deploy your Purview account into its own data management landing zone. Use one data management landing zone per Microsoft Entra ID tenant for Purview. This model matches Purview's tenant-wide governance role and avoids fragmented control.

1.2 Host Microsoft Fabric capacity in data management landing zones

Compute in Microsoft Fabric is called [Fabric capacity](#). Microsoft Fabric capacity provides pooled compute for Fabric workloads, similar to Power BI capacity. **Recommendation:** Decide whether to assign Fabric capacity per data domain, share capacity across domains, or combine both approaches based on data domain maturity and usage predictability. To apply this recommendation, review the following options:

Option 1: Choose decentralized Fabric capacities for mature data domains. In this model, each data domain has its own DMLZs. They create and manage their own Microsoft Fabric capacity in their DMLZs. **Best practices:** Use this model when a data domain demonstrates operational maturity and stable demand. Allow the domain to control scaling decisions and budget accountability. **Decision guidance:** Decide on decentralized capacity when workloads require predictable performance and clear cost ownership. Accept higher operational overhead and possible idle capacity as tradeoffs. Smaller capacities might also limit access to some Power BI features.

Option 2: Choose centralized Fabric capacity for early or variable usage. In this model, you manage capacity through a central team. A centralized model places multiple data domains on one or a few shared Fabric capacities. **Best practices:** Use this model to simplify operations and improve overall capacity utilization. Central teams handle monitoring, scaling, and governance standards. **Decision guidance:** Choose centralized capacity when data domains show low maturity or fluctuating demand. Accept the risk of resource contention and reduced cost transparency without strong governance controls.

Option 3: Use a hybrid capacity model to support growth. A hybrid approach keeps smaller or new data domains on shared capacity while assigning dedicated capacity to domains with sustained demand or higher criticality. You combine shared and dedicated capacity models. **Best practices:** Define clear thresholds for capacity graduation. Base thresholds on sustained usage, uptime requirements, or isolation needs. Apply consistent governance across both models. **Decision guidance:** Choose a hybrid model when the organization expects uneven data domain growth. Accept added governance complexity as the tradeoff for flexibility and long-term scalability.

For more information, see [Deployment Patterns for Microsoft Fabric](#).

2. Integrate your application landing zones

Application landing zones host operational workloads, such as business applications, services, and AI solutions that produce or consume enterprise data. These landing zones sit outside data management landing zones and usually have different owners. As a decision maker, you define how these environments interact with the unified data platform to protect governance, reduce duplication, and maintain operational independence.

Recommendation: Set clear integration standards that require application landing zones to access enterprise data through governed Fabric and OneLake mechanisms rather than direct or unplanned data paths. To apply this recommendation, use the following checklist:

1. **Standardize Microsoft Foundry data access.** Data access defines how applications and AI solutions retrieve governed data from OneLake. Inconsistent access patterns create shadow data copies and weaken governance controls. **Best practices:** Require application teams to use approved Fabric and Foundry integration mechanisms to access data stored in OneLake. These mechanisms include the Fabric Data Agent and Azure AI Search with Foundry IQ indexes. This approach enforces consistent governance, security, and audit controls across all consuming applications. **Decision guidance:** Choose mandatory standardized access when data sensitivity or regulatory requirements are high. This choice limits flexibility but preserves governance integrity. Avoid allowing direct database access when long-term data consistency and trust matter more than short-term convenience.
2. **Integrate Azure operational databases through Fabric mirroring.** Many application landing zones host online transaction processing (OLTP) databases that support core business operations, such as sales, finance, and customer records. These systems generate critical data but can't absorb analytical workloads without risk. **Best practices:** Use Fabric [mirroring](#) to replicate selected operational data into OneLake in near real time. This approach keeps operational systems independent while providing current data for analytics and AI across the organization.
3. **Integrate major external systems (SAP, Oracle) in OneLake.** Large enterprise systems, such as SAP and Oracle, often operate outside Azure or have strict uptime and change-control requirements. These systems still represent authoritative sources of business data. **Best practices:** Integrate SAP and Oracle data into Fabric by using mirroring for [SAP](#) and [Oracle](#). This model creates a consistent and governed ingestion path into OneLake while preserving the operational independence of these platforms. **Decision guidance:** Choose Fabric mirroring when the organization needs a single convergence layer for enterprise data without disrupting core systems. This choice simplifies governance and analytics but requires clear ownership for data freshness and availability expectations.

3. Plan your data landing zones (as needed)

A data landing zone is an environment (consisting of one or more Azure subscriptions) for data and AI/ML resources that operate alongside Microsoft Fabric. These platforms include Azure Databricks, Azure Data Lake Storage (ADLS), and Azure Machine Learning. These platforms often serve specialized use cases that Fabric doesn't replace. As a decision maker, you define when to introduce these environments and how they integrate with the unified data platform.

Recommendation: Use data landing zones when the organization requires data or AI platforms beyond Fabric. Integrate those environments with OneLake through governed patterns. To apply this recommendation, use the following checklist:

3.1 Choose how data products map to data landing zones

A data domain can group multiple data products in one data landing zone or assign each data product its own landing zone. This choice affects isolation, governance scope, and operational effort. **Recommendation:** Align landing zone structure to the independence and sensitivity of data products within the domain. Keep the structure simple unless regulatory or operational requirements require stronger separation. To apply this recommendation, review the following options:

Option 1: Multiple data products in one DLZ. Choose one data landing zone for multiple data products when the products share security requirements and infrastructure standards. This option simplifies governance and reduces management overhead. Accept reduced cost separation and shared risk if one product encounters problems.

Option 2: One DLZ per data product. Choose one data landing zone per data product when the product is critical or requires distinct security or compliance controls. This option provides strong isolation and clear cost attribution. Accept higher operational overhead and some duplication of platform services.

Option 3: Apply hybrid models intentionally. Some data domains require both shared and dedicated data landing zones based on product maturity or risk. Choose a hybrid approach when some data products require isolation and others don't. Avoid inconsistent or unplanned decisions, because inconsistent structure increases governance complexity and operational risk.

3.2 Use Fabric and Databricks integration patterns

If your data platform includes Azure Databricks, you need to integrate it with your Microsoft Fabric OneLake environment. Two supported integration patterns exist between Azure Databricks and Microsoft Fabric. **Recommendation:** Establish decision criteria to help data

domains determine where the system of record should be. To apply this recommendation, review the following options:

Option 1: OneLake as the system of record. In this pattern, you configure Databricks workspaces to read from and write to OneLake, not Azure Data Lake Storage Gen2. **Best practices:** Use this pattern when the organization prioritizes a single authoritative lake under Fabric governance. Centralize security and policy controls in Fabric. Reduce infrastructure management by avoiding separate Azure Data Lake Storage accounts. **Decision guidance:** Choose this option for new data platforms or strategic rebuilds. Accept initial integration setup as a tradeoff for long-term simplicity and centralized ownership. Recognize that teams accustomed to Databricks-managed storage must align to Fabric governance and operating standards. See [Azure Databricks integration with OneLake](#).

Option 2: ADLS as the system of record. In this pattern, Azure Databricks continues to write to Azure Data Lake Storage Gen2 while Microsoft Fabric accesses the same data through OneLake shortcuts that reference ADLS locations. **Best practices:** Apply this pattern to existing Databricks estates with mature pipelines. Avoid data movement or pipeline refactoring. Expose curated datasets to Fabric through managed shortcuts. Register ADLS sources in Microsoft Purview to maintain catalog visibility and governance consistency. **Decision guidance:** Choose this option when speed and continuity matter more than consolidation. Accept split ownership between ADLS operations and Fabric governance. Plan for added coordination across teams that manage storage, security, and metadata. See [Azure Data Lake Storage \(ADLS\) Gen2 shortcut](#).

Next step

Data governance and security baselines with Purview

Last updated on 03/10/2026

Fabric architecture for a unified data platform



Summarize this article for me

After aligning on why and what data products you need (organizational readiness), the next step is to decide how the technology fits together. This step involves creating an architecture that connects and integrates Microsoft Fabric with Azure.

This article helps you make the right architecture decisions for designing **Microsoft Fabric** as the foundation of your unified data platform. Microsoft Fabric provides a data platform with a built-in SaaS data lake called [OneLake](#) that centralizes data for analytics and AI across the organization.

Recommendation: Adopt Microsoft Fabric with OneLake as the default analytics data platform and define clear leadership decisions for compute, regions, recovery, and workspace ownership. This setup enables all workload teams to operate within consistent standards (see Figure 1). To apply this recommendation, use this article as a checklist:

Unified data platform architecture

Figure 1 shows an integrated Fabric-in-Azure architecture aligned to data domains, with enough operational structure to make the design concrete. It serves as a reference model to help organizations design and standardize their architecture.

Microsoft's Cloud Adoption Framework
Unified data platform for AI and analytics (architecture)

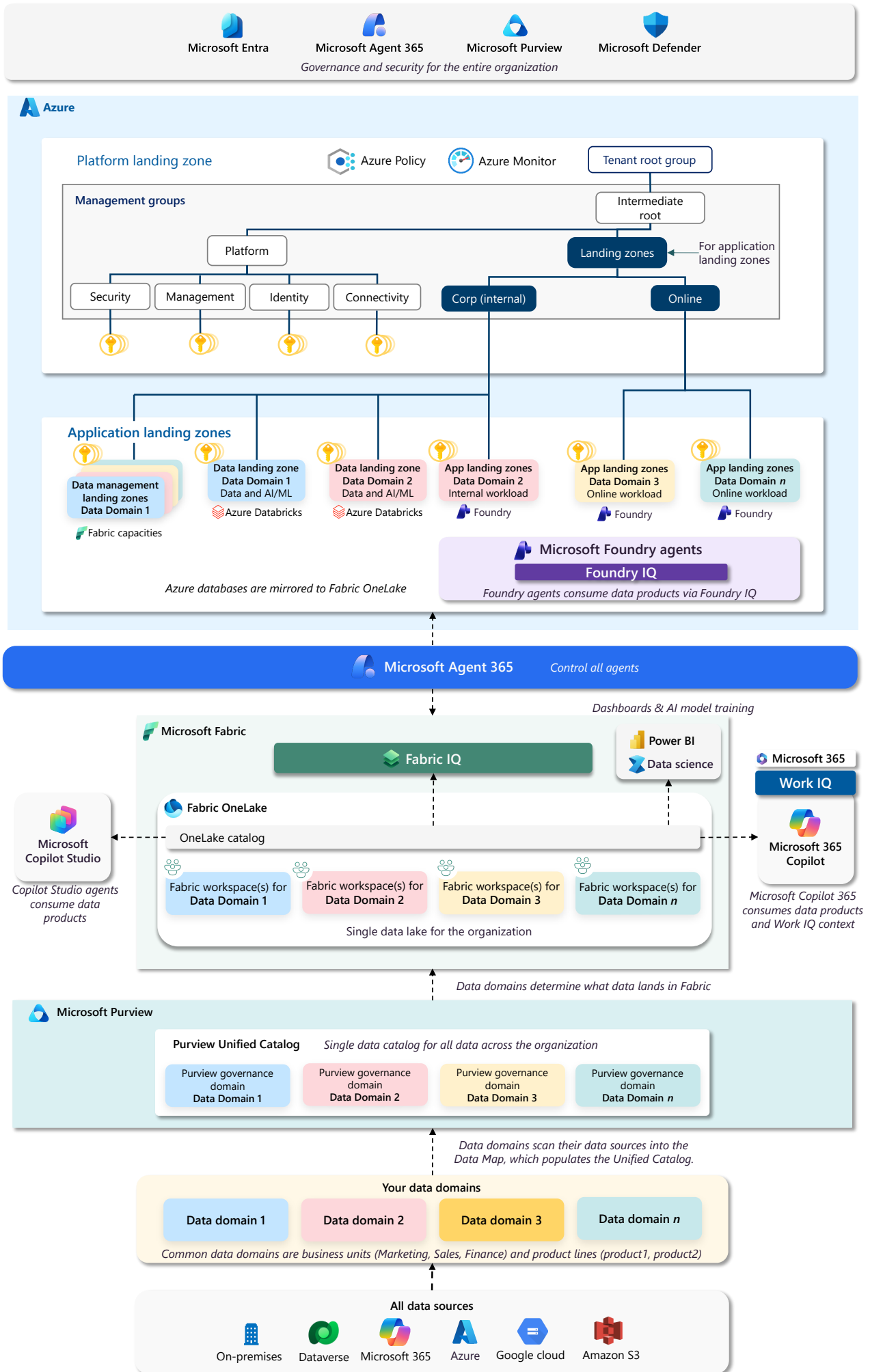


Figure 1. Architecture: Unified data platform for AI and analytics.

1. Plan for Fabric compute

Fabric compute runs on capacity, which defines available resources, performance, and concurrency for all workloads that share that capacity. Capacity choices directly affect cost control and service reliability. **Best practices:** Allocate budgets to data domain teams and allow those teams to own Fabric capacities that match their business criticality. Create each Fabric capacity in a data management landing zone. To learn more, see [Data management landing zones](#).

2. Plan your Fabric regions

Each Fabric capacity runs in a single Azure region. That region determines where compute and OneLake data for workspaces on that capacity reside. Region decisions affect data residency, latency, and service availability. **Best practices:** treat region selection as a governance decision. Publish a short list of supported regions and enforce that list through policy. **Decision guidance:** Decide whether to use [Multi-Geo](#). Choose Multi-Geo when the organization needs formal governance across regions and clear rules for why regions exist. Multi-Geo is unnecessary when operations remain confined to a single geography.

3. Plan your Fabric workspaces

A [Fabric workspace](#) groups data assets and acts as the primary boundary for security, administration, and cost visibility. Workspace design shapes how domains operate inside OneLake. **Best practices:** Give each data domain one or more dedicated workspaces to manage its data products. Apply access control, cost tracking, and operational policies at the workspace level. Use [Fabric domains](#) to apply shared governance across related workspaces (see Figure 2).

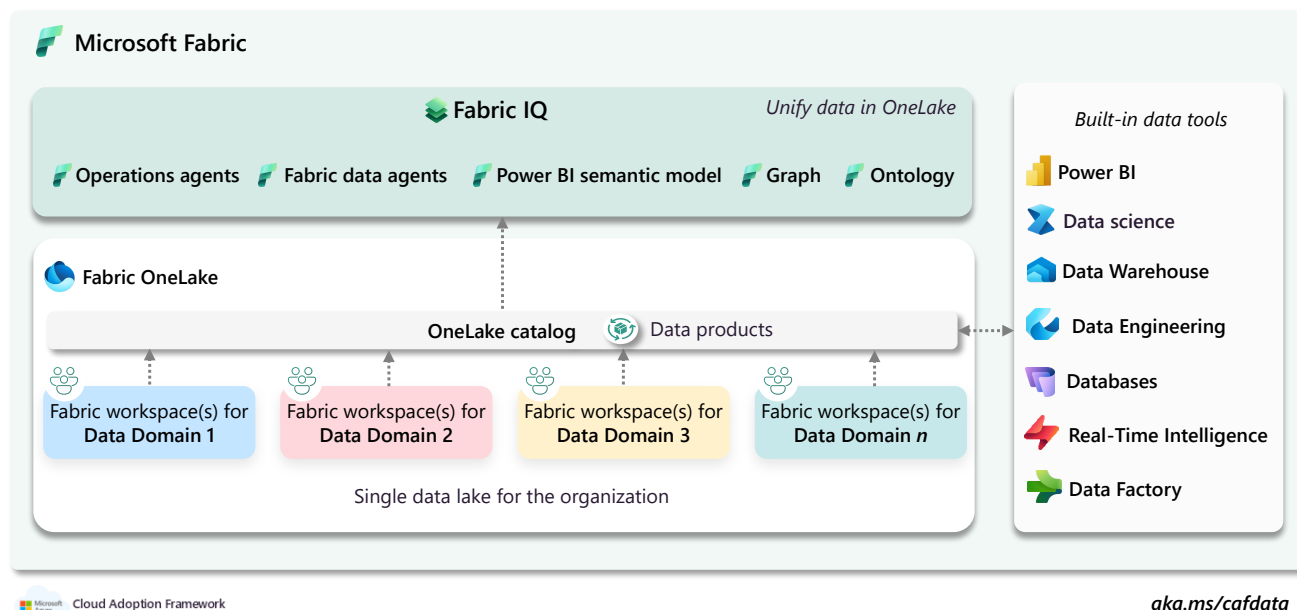


Figure 2. Microsoft Fabric architecture.

4. Plan for Fabric IQ

Fabric IQ provides a semantic intelligence layer over OneLake. This layer defines shared business concepts that analytics tools and AI agents can interpret consistently. **Best practices:** Use Fabric IQ to provide shared meaning across data domains. Focus on scenarios where shared definitions reduce reporting risk or improve AI outcomes. **Decision guidance:** See the following guidance to [choose the right Fabric IQ capability](#).

5. Plan for Fabric disaster recovery

Fabric disaster recovery copies OneLake data to a paired Azure region. This capability supports workload recovery after a regional outage. **Best practices:** Treat Fabric disaster recovery as a business continuity decision. Enable disaster recovery where business impact justifies. **Decision guidance:** Decide to enable disaster recovery when downtime creates material risk. Choose not to enable it for exploratory or noncritical workloads. To learn more, see [OneLake Disaster Recovery and Data Protection](#).

Next step

Azure architecture for a unified data platform

Data governance and security baselines with Microsoft Purview

With the foundational strategy and architecture in place, establish strong governance and security practices from the very beginning. This approach makes sure that as data from all corners of the organization flows into your unified platform, it remains well managed, compliant, and secure. **Recommendation:** Set Microsoft Purview as the system of record for data governance and security so your organization can apply consistent policy, accountability, and compliance across the full data estate (see Figure 1). To apply this recommendation, use this article as a checklist:

Microsoft's Cloud Adoption Framework
Data governance and security baselines with Microsoft Purview

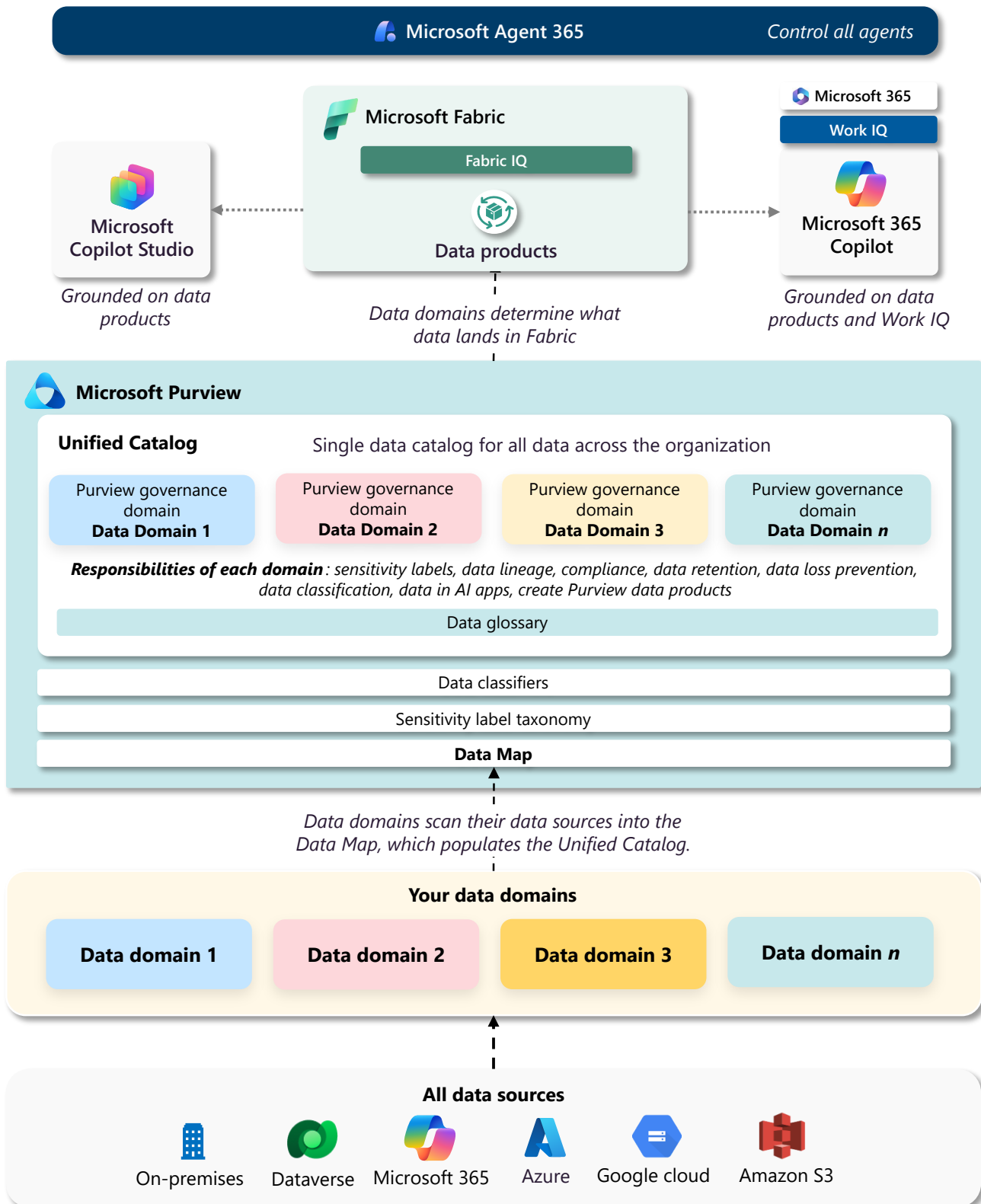


Figure 1. Microsoft Purview's role in data governance and security baselines.

1. Data visibility baseline

Data governance starts with a clear view of all your data assets across the entire estate.

Recommendation: Establish a consistent visibility baseline with Microsoft Purview before data enters OneLake. This approach helps governance decisions remain consistent across platforms and teams. To apply this recommendation, use the following checklist:

1.1. Create a data catalog

A centralized data catalog provides decision makers with a single system of record for understanding what data exists, where it resides, and who owns it. **Recommendation:** Use [Microsoft Purview's Unified Catalog](#) as the authoritative inventory for all enterprise data assets. To apply this recommendation, use the following checklist:

1. **Use governance domains.** Organize the Unified Catalog by using Purview [governance domains](#) that align to existing data domains. Collections, roles, and permissions should reflect these boundaries, so each data domain has authority in its area. The Unified Catalog begins empty, but as data sources are registered and scanned, it grows into an accurate map of the estate. Setting domains early brings clarity as the catalog expands.
2. **Populate the glossary.** With structure in place, create a shared glossary that anchors the organization's most important terms. Terms, such as Customer, Product, Employee, and Location, and measures like Revenue or Headcount, often vary across departments unless harmonized. Document these definitions as [glossary terms](#) in the Unified Catalog and view them in Purview's glossary. Communicate them broadly to help eliminate ambiguity. This clarity supports consistent AI, analytics, and reporting practices across teams.

1.2. Map data sources

The Purview Data Map provides visibility into data assets without copying data or replacing source-level security controls. **Recommendation:** Register and scan all relevant data systems to populate the [Data Map](#). To apply this recommendation, use the following checklist:

1. **Create Purview architecture.** Use Purview collections and [Data Map domains](#) to align permissions and governance with the needs of your data domains. These constructs define boundaries for access control, policy management, and operational responsibility. Follow Purview [domain and collection architecture best practices](#).
2. **Set up Purview for Microsoft 365 data.** Purview has native integration with Microsoft 365 data (such as SharePoint, OneDrive, Exchange, Teams). Make sure that you also govern content in Microsoft 365. When you bring Microsoft 365 documents and messages into OneLake or otherwise use them in analytics, any labels or classifications from Microsoft 365

carry over. In Purview, you can see sensitivity labels and retention labels that you applied in the Microsoft 365 environment. To learn more, see [Microsoft Purview setup guides](#).

3. **Scan Microsoft Fabric OneLake.** Fabric OneLake isn't automatically included in the Purview Data Map. You must explicitly register and scan it. Scanning OneLake enables metadata discovery, lineage tracking, and cataloging of Fabric assets in Purview. To learn more, see [Register and scan Microsoft Fabric](#).

1.3. Scan cloud, SaaS, and on-premises data

Connector-based scanning is required for data stored in Azure services, Microsoft Dataverse, on-premises systems, and other clouds (AWS, Google Cloud, Oracle). You need to [register](#) and scan [supported data sources in Data Map](#). **Recommendation:** Follow Purview's [scanning best practices](#). Choose whether to scan source systems or only scan the Microsoft Fabric OneLake layer. This decision depends on visibility needs, compliance requirements, operational overhead, and the role each system plays in analytics and reporting. To apply this recommendation, review the following options:

Option 1: Scan source systems. Scanning operational systems such as Azure databases or AWS S3 provides end-to-end lineage from the system of record. This information is important for regulated or business-critical data where full provenance supports audits and compliance. It provides the clearest visibility into upstream changes but introduces complexity. Connector configuration, credentials, and scheduling require attention, and some sources require you to [Choose the right integration runtime](#).

Option 2: Scan only the Fabric layer. If deep visibility into upstream systems isn't required, scanning only the Fabric layer simplifies the governance model. Lineage begins when data enters Fabric. This approach reduces integration work. It's best for data where upstream systems are well governed or where compliance obligations don't require full lineage. To learn more, see [Fabric](#) in Microsoft Purview.

1.4. Apply sensitivity labels

Sensitivity labels are a fundamental tool for classifying and protecting data. In Microsoft Purview, there are typically two kinds of sensitivity labels:

- **Metadata-only labels:** These labels are metadata tags in the Purview catalog. For example, labeling an Azure SQL table as "Confidential" in the catalog without affecting the source system directly. These labels help track and manage data sensitivity in Purview for assets outside of Microsoft 365/Fabric.

- **Protective labels:** These labels not only mark data with a classification, like "Confidential," but also enforce protection. They can enforce encryption or restricted access on files and emails. Use these labels heavily in Microsoft 365 and now extend into Fabric as well.

Both contribute to a consistent governance model, and clarity about their purpose helps align the right label type to the right scenario. **Recommendation:** Apply a unified sensitivity labeling strategy across Microsoft 365, Fabric, and the Purview Data Map. To apply this recommendation, use the following checklist:

1. **Define a sensitivity label taxonomy.** Define an organizational labeling schema. A common schema is Non-business, Public, General, Confidential, and Highly Confidential. Make sure everyone understands what each label means and what types of data fall under each category. Get buy-in on this taxonomy from compliance and business stakeholders. See [Get started with sensitivity labels](#).
2. **Label Microsoft 365 data (protective label).** Make sure Microsoft 365 information is labeled by using Microsoft Purview Information Protection. These labels impose security controls (encryption, restricted access). They persist with the data when brought into OneLake or shared via other tools. **Best practices:** Many organizations set up auto-labeling policies to detect sensitive info, like credit card numbers or personal data, and automatically apply a label. See [automatically apply sensitivity labels to Microsoft 365 data](#).
3. **Label Microsoft Fabric data (protective label).** Microsoft Fabric supports [protected sensitivity labels](#) on its own assets, like tables in a Lakehouse, and in Power BI. Configure [default label policies](#) in Fabric so that new data in OneLake is labeled appropriately from creation. For instance, you could specify that any new dataset in certain workspaces is by default labeled as internal or confidential unless changed. This approach makes sure no data enters the lake without classification. Adjust these defaults for areas that handle sensitive data. See [Govern Fabric](#).
4. **Label entries in Purview Data Map (metadata-only label).** For data sources that are scanned into Purview (like an Amazon S3 bucket or an on-premises database), apply metadata labels in the Purview Data Map. While these labels don't encrypt or restrict the data at the source, they do inform users and systems that the data is, say, confidential. They can also trigger other governance workflows. Everything in your catalog should have a sensitivity designation. See [Use autolabeling policies](#) to detect data assets and apply these metadata-only sensitivity labels automatically.

1.5. Capture data lineage

Data lineage provides visibility into how data moves and changes across systems.

Recommendation: Enable automated lineage where available and close gaps manually where required. **Best practices:** Use Microsoft Purview to [automate](#) lineage capture for many assets. Where automation isn't available, add lineage manually in Purview to fill gaps. See [Manual lineage setup](#). Configure data lineage in Purview for [Fabric](#) and, as needed, for [Azure Databricks](#).

2. Data estate compliance baseline

Compliance defines the legal, regulatory, and internal obligations that apply to data across platforms. Decision makers must understand these obligations before enforcing controls. This approach makes sure governance actions remain defensible and auditable across Microsoft Fabric and the broader data estate. **Recommendation:** Define compliance requirements centrally and monitor alignment continuously by using Microsoft Purview. Data governance decisions should remain consistent across Microsoft 365, Azure, other clouds, and on-premises systems. To apply this recommendation, use the following checklist:

1. **Define your compliance requirements.** Compliance requirements vary by industry, region, and workload. The governance model must reflect them before you apply any protective or technical controls. Identify which regulatory frameworks or industry standards are relevant to your organization. **Best practices:** Use Microsoft Purview [regulatory templates](#) and [assessments](#). They represent standards such as, [HIPAA and HITECH](#), [PCI DSS v4.0](#), [Sarbanes-Oxley Act](#), and [ISO 27001](#). Review these templates to understand which rules apply across Microsoft 365, Azure, AWS, and Google Cloud. These templates provide a checklist of controls and practices you should have in place. Use the Compliance Manager's scores and recommendations to gauge your current posture, identify gaps, and prioritize what to address first.
2. **Monitor data compliance.** After you set up Purview's compliance templates, regularly review the **compliance score** and reports. Purview automatically assesses aspects of compliance for [Microsoft 365 data](#) and data across [Azure, AWS, and Google Cloud](#). It highlights issues and suggested actions to improve compliance. Set up alerts for any critical compliance policy violations. **Best practices:** If Purview finds sensitive data in an unapproved location or if a retention policy is violated, notify responsible teams immediately so they can take action. Steady monitoring and incremental improvements make sure you're not caught off guard by audits or incidents.
3. **Configure Microsoft 365 data retention.** Decide how long to keep different types of data and when to delete or archive it. Ambiguity in retention can either lead to keeping data too long (risking compliance breaches or unnecessary storage costs) or deleting too soon

(losing valuable history). **Best practices:** Use Microsoft Purview's data lifecycle management for [Microsoft 365 data](#) to set retention or deletion policies on emails, documents, and Teams messages.

4. **Configure Azure data retention.** Azure services require service-specific retention and backup configuration. **Best practices:** Configure backup retention for services, such as Azure SQL, Cosmos DB, and MySQL. Use Azure Storage [lifecycle management rules](#) to archive or delete data. Reference service guidance for [Azure SQL Database](#), [Cosmos DB](#), and [MySQL](#).
5. **Configure on-premises and other clouds data retention.** Data outside Microsoft platforms still requires compliant lifecycle controls. To avoid unmanaged compliance risk, apply intentional retention strategies to all non-Azure and on-premises data sources. **Best practices:** Use Azure Backup or third-party solutions to retain on-premises, AWS, and Google Cloud data. Follow guidance to [Backup cloud and on-premises workloads to cloud](#). Where needed, manually [upload](#) data to Azure Storage and [archive](#) the blob.

3. Data estate security baseline

A consistent security baseline makes sure that sensitive data remains protected across Microsoft 365, Microsoft Fabric, Azure services, and AI workloads. **Recommendation:** Define and enforce a single security posture across the entire data estate so that classification, protection, and enforcement operate uniformly at scale. To apply this recommendation, use the following checklist.

1. **Classify sensitive data.** [Sensitive data classifiers](#) in Microsoft Purview identify regulated and business-critical data so that protection controls can act automatically and consistently across services.
 - **Option 1. Automated pattern-matching.** Use pattern-matching mechanisms, such as sensitive information types, document fingerprinting, and exact data match, when regulatory alignment is the primary goal and the data types match standard definitions.
 - **Option 2. Trainable classifiers.** Add trainable classifiers to the automated pattern-matching capabilities when business-specific data carries risk that standard pattern-matching doesn't detect, such as trade secrets or proprietary research. Tradeoff: Trainable classifiers increase coverage but require governance oversight to remain accurate over time.

2. **Apply data loss prevention (DLP).** Configure DLP policies in Microsoft Purview to prevent leaks of sensitive data through everyday actions. Set up DLP for both Microsoft 365 (to cover emails, Office documents, and Teams chats) and Microsoft Fabric. A DLP policy can block a user from sharing a file externally if it contains confidential data like customer personal data or health information. In Fabric, DLP can prevent analysts from sharing a dataset or report that includes sensitive data. For detailed configuration, refer to the guidance for [Microsoft 365 apps](#), [Copilot](#), and [Microsoft Fabric](#). **Decision guidance:**

- **Monitor-only (audit mode):** Start with DLP in monitor-only mode when you're concerned about disrupting work. It lets you observe and fine-tune policy behavior, but data might still be exposed because enforcement isn't active.
- **Blocking or restricting:** Move to blocking or restricting mode when data leakage would have severe effects and your detection rules are reliable. Some legitimate actions might be blocked initially and require exception handling and policy adjustments.

3. **Protect data in Azure services.** Microsoft Purview catalogs and labels data in Azure services but doesn't replace native security controls for those services. **Best practices:** Apply service-level security controls, such as network isolation and Microsoft Defender coverage for [Azure SQL Database](#), [Azure Cosmos DB](#), and [Azure Storage](#). Align these controls with Purview classifications so that monitoring and alerting reflect data sensitivity and business risk.

4. **Protect data used by AI apps.** AI applications introduce new data exposure paths that require explicit alignment with enterprise data governance and security policies. Microsoft Purview integrates with Microsoft Foundry and other AI platforms to provide this control.

Best practices:

- Use [Microsoft Purview Data Security Posture Management \(DSPM\)](#). It provides centralized visibility and protection for AI-related data risk. For onboarding and governance automation of DSPM for AI, see [Data Agent Governance and Security Accelerator](#) .
- Use Purview APIs and AI-specific security features to pass sensitivity context into AI workflows so applications can apply masking or response restrictions when required. See [Microsoft Purview and AI](#) and [Purview with Foundry](#).
- Establish review checkpoints for high-impact AI scenarios to confirm that data access aligns with governance standards.

Next step

Fabric governance and security baselines

Last updated on 06/23/2026

Fabric governance and security baselines



Summarize this article for me

Microsoft Fabric centralizes data in OneLake, so governance and security are essential for risk control, regulatory compliance, and operational trust. Decision makers should tailor the principles established for the broader [data estate](#) to the Fabric context.

Recommendation: Establish consistent governance and security baselines for all Microsoft Fabric environments to protect OneLake and ensure alignment with data standards. To apply this recommendation, use this article as a checklist:

1. Set Fabric environment governance baseline

A core governance decision is how you define and structure environments in Microsoft Fabric. Fabric workspaces define security boundaries, capacity allocation, and operational ownership in Microsoft Fabric, so environment structure directly affects governance, risk, and cost control for the organization. **Recommendation:** Establish a default workspace pattern that aligns to data domains and ownership. Allow exceptions when isolation or constraints require them. To apply this recommendation, review the following options:

Option 1: One workspace per data product. In this model, each data product gets one Fabric workspace. **Decision guidance:** Choose this option when one team owns the full pipeline and when moderate isolation meets governance needs. Avoid this option when regulatory or organizational separation requires stricter boundaries. If you're new to medallion architecture, see [Apply medallion architecture](#). Within the workspace, there are two ways to implement the medallion architecture:

- **Single Lakehouse with multiple schemas:** The simplest option is to use one [Lakehouse](#) in the workspace and separate raw (Bronze), cleaned (Silver), and curated (Gold) data by using distinct [schemas](#) or folders. This approach keeps things simple (one place for all data), but it requires strong governance discipline to prevent accidental mixing of raw and curated data (see Figure 1).

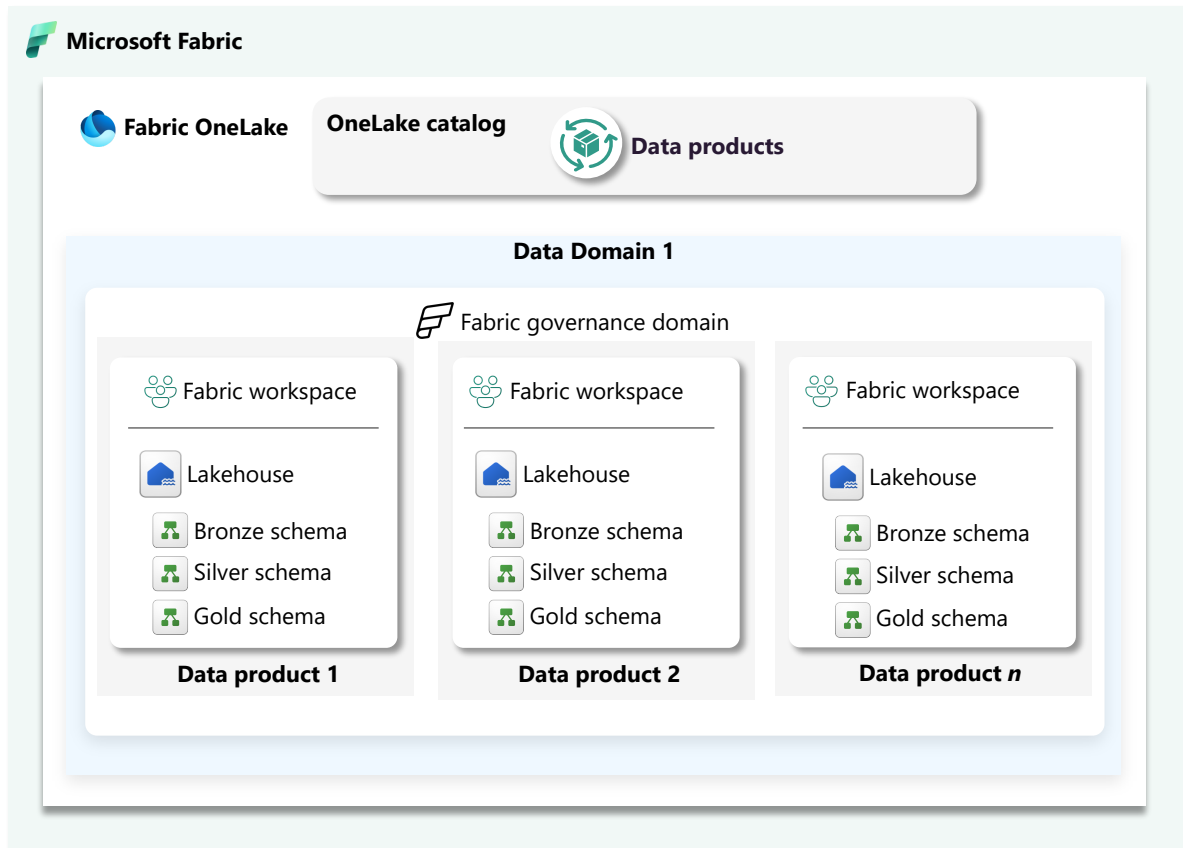
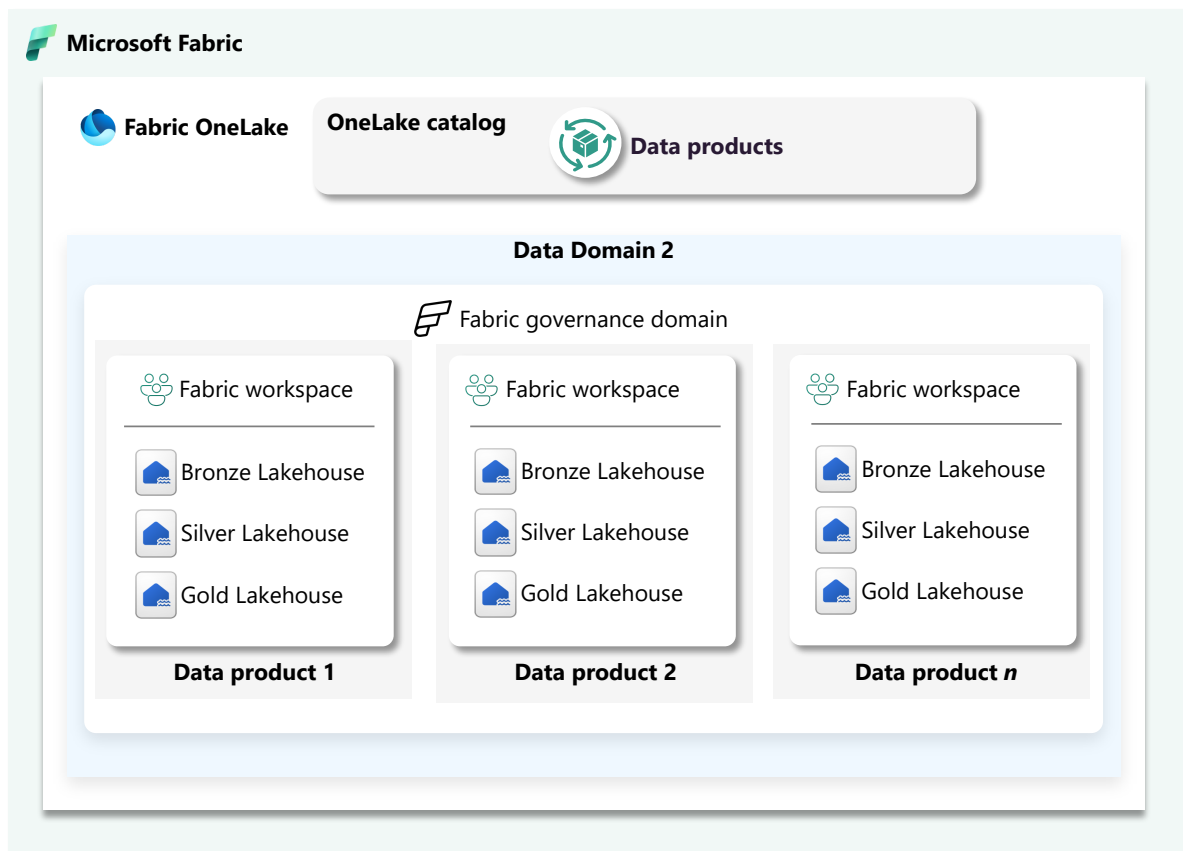


Figure 1. One workspace per data product. Single Lakehouse for each data product. One Lakehouse schema for each layer of the medallion architecture.

- **Multiple Lakehouses for layers:** Alternatively, use multiple Lakehouse objects in the same workspace, dedicating one to each layer. There's one Lakehouse for raw Bronze data, one for Silver, and one for Gold. This approach provides clearer separation and safety between stages. You can give different permissions or even use separate compute for each layer. But it adds more operational complexity when managing the workspace (see Figure 2).



Microsoft Azure Cloud Adoption Framework

Figure 2. One workspace per data product. Multiple Lakehouses for each data product. One Lakehouse for each layer of the medallion architecture.

Option 2: Multiple workspaces per data product (high isolation). This model assigns each medallion layer or lifecycle stage to a separate workspace. **Best practices:** Use this pattern to meet regulatory requirements or to enforce strict separation of duties. Each workspace enforces its own security and capacity boundaries. Data moves across workspaces through governed processes. **Decision guidance:** Choose this option when compliance, risk, or team boundaries require strong isolation. Accept higher coordination cost and operational overhead as a tradeoff (see Figure 3).

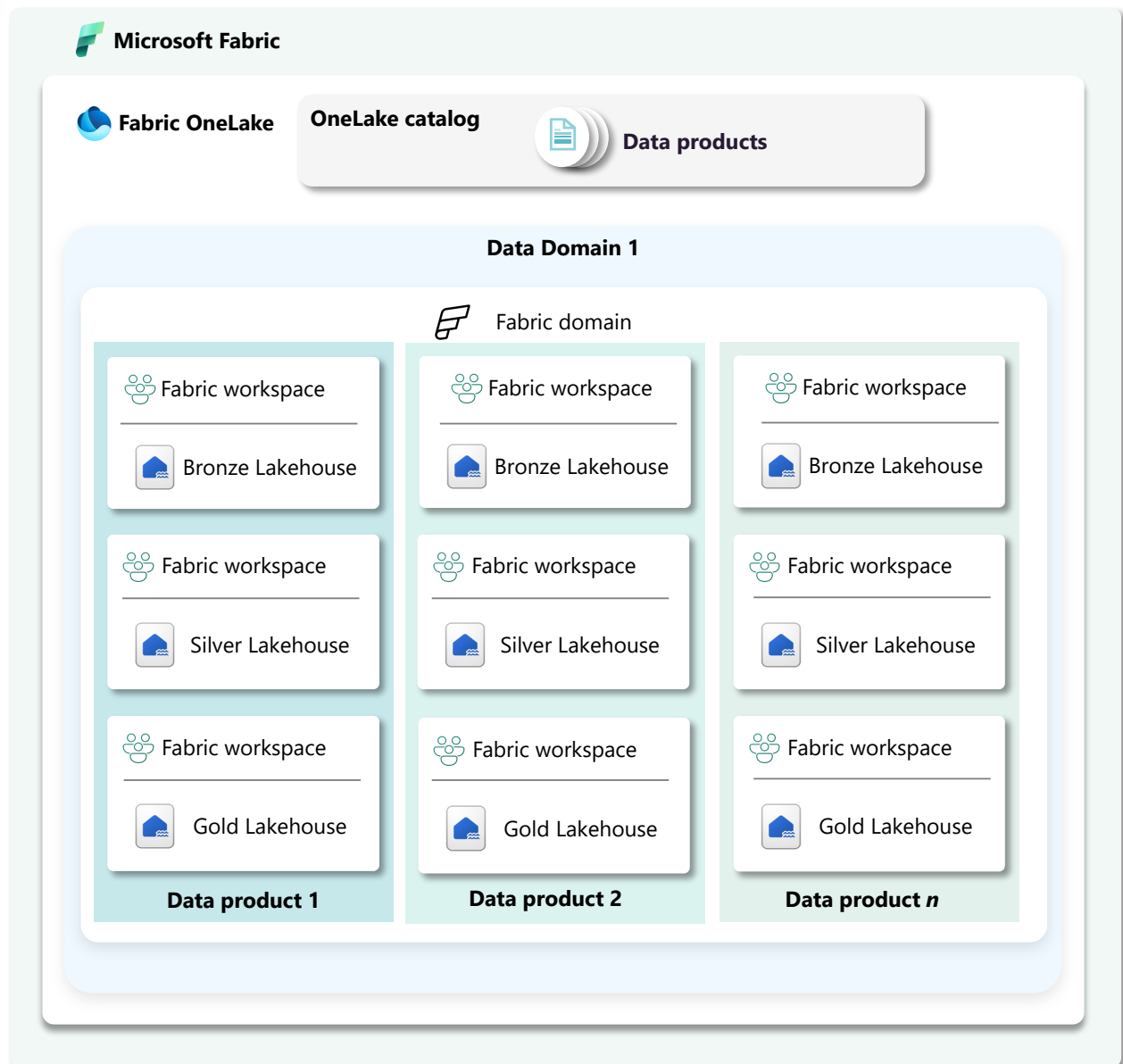


Figure 3. Multiple Fabric workspaces per data product. One workspace per medallion layer.

Option 3: Consolidate multiple data products into one workspace only when constrained.

This model groups smaller or lower-risk data products in a shared workspace due to capacity or licensing limits. **Best practices:** Assign a domain team to manage the workspace and enforce naming and access standards. This approach limits sprawl during early adoption or pilots.

Decision guidance: Choose this option when Fabric capacity or workspace limits prevent a per-product model. Limit use to low-criticality workloads and plan a future move to dedicated workspaces.

Standard to enforce: Tie every workspace model to a data domain and a named owner.

Document the default pattern and the approved exceptions. Educate teams on the governance intent of workspaces, not just the mechanics. Use one workspace per data product as the default unless isolation or constraints justify a different choice.

2. Set cost governance baselines

A cost governance baseline defines how your organization controls spending and optimizes capacity after Microsoft Fabric goes live. Leaders must manage these costs because uncontrolled compute usage reduces business value and erodes trust in the platform.

Recommendation: Define clear cost governance processes that connect Fabric capacity decisions to business priorities and financial accountability. To apply this recommendation, use the following checklist:

1. **Define a Fabric budget.** A Fabric budget is the financial limit that applies to all workloads running on a data domain's Fabric capacities. **Best practices:** Each data domain receives and sets a budget for its Fabric usage before onboarding any workloads. Treat this budget as a hard governance boundary, not a planning estimate. Align spending decisions to it from day one.
2. **Control which workloads consume Fabric capacity.** Workspace assignment controls access to shared compute and shared spend. **Best practices:** Require approval before assigning new workspaces to Fabric capacities. Use this step to confirm business value and budget fit. **Decision guidance:** Limit assignment rights to a few platform owners when governance maturity is low. Broaden access only when cost discipline is proven. Restriction reduces surprise cost. Broader access increases speed.
3. **Standardize permissions that influence capacity consumption.** Certain Fabric permissions allow actions that significantly increase capacity usage. **Best practices:** Restrict advanced creation and scaling permissions on high-cost Fabric capacities. Assign elevated permissions only to trusted operational roles within the data domain. **Decision guidance:** Choose central role assignment when strong guardrails matter. Choose delegated assignment when teams show cost discipline. Central control reduces risk. Delegation improves speed but needs audit discipline.
4. **Control capacity scaling decisions.** Capacity changes directly affect shared budgets and platform stability. Unplanned scaling creates cost risk without guaranteed benefit. **Best practices:** Define a formal approval process for capacity increases. Require business justification based on demand growth or performance evidence. Assign approval authority to a governance body or named role. Tie scaling decisions to agreed service targets rather than reactive fixes. Use [Pause and resume your capacity](#) and [Scale your Fabric capacity](#). **Decision guidance:** Decide how centralized approvals remain. Keep approval centralized for changes that affect shared budgets. Allow limited autonomy only after teams demonstrate consistent cost discipline. Central control reduces financial risk. Selective autonomy improves responsiveness.

5. **Set standards for throttling and surge scenarios.** Usage spikes can overload capacity or trigger throttling. Leaders must plan for these conditions. **Best practices:** Define acceptable thresholds for capacity strain and set alerts before user impact occurs. Schedule non-urgent workloads outside peak hours. Treat recurring throttling as a platform issue that requires capacity or workload optimization. Use [Understand your Fabric capacity throttling](#) and [Surge protection](#) to guide policy. **Decision guidance:** Decide what performance degradation the business tolerates. Accept brief slowdowns for non-critical workloads when cost control matters most. Maintain extra capacity for critical workloads where reliability matters more than cost. Extra capacity improves stability at higher cost. Lean operation reduces spend but increases risk during peaks.
6. **Review platform costs and performance at the leadership level.** Fabric charges compute through capacity units. Leaders can correlate capacity usage with the Azure bill to understand cost drivers and validate spending patterns. **Best practices:** Hold routine leadership reviews that examine platform cost, utilization trends, and business value. Use those meetings to align spending with business priorities and address any inefficiencies. Use [Understand your Fabric capacity Azure bill](#) to review billed usage alongside capacity consumption trends. Use [Evaluate and optimize your Microsoft Fabric capacity](#) to frame the review around overload risk and underuse risk. **Decision guidance:** Decide the review cadence based on volatility. Choose monthly reviews when workloads change often or when you see frequent peaks. Choose quarterly reviews when usage patterns remain stable. A shorter cadence reduces budget surprises but requires more leadership attention.

3. Set data security baseline

A data security baseline defines how leaders control access, classification, and data movement across Microsoft Fabric so governance decisions stay consistent at scale. Decision makers who set this baseline early reduce exposure risk and prevent informal sharing from becoming the operating norm. **Recommendation:** Establish a standard security baseline that applies to all Fabric workspaces and data products before publication so teams follow a single governance model. To apply this recommendation, use the following checklist:

1. **Define workspace security boundaries.** Workspaces in Microsoft Fabric act as the primary security boundary for data and analytics assets. Leaders should determine a policy for who creates workspaces and who controls access within them. **Best practices:** Many organizations designate a primary workspace administrator responsible for managing access and security settings for each workspace. They also establish naming conventions and metadata tags for workspaces indicating their purpose. Follow the [OneLake security best practices](#). **Decision guidance:**

- **Decide who can create new workspaces.** Consider a process or approval for creating Microsoft Fabric workspaces so they align with your governance. Uncontrolled workspace creation could lead to sprawl and data governance gaps.
- **Decide who can access what.** Make sure each workspace has administrators and owners who control membership. Enforce the principle of least privilege. Give each user or group the minimum permissions they need (viewer, contributor, admin). Avoid overly broad access or ad-hoc sharing of workspaces and items, because that approach can inadvertently expose data to the wrong people.

2. **Apply default sensitivity labels.** Sensitivity labels classify data based on business and regulatory impact. Microsoft Fabric uses Microsoft Purview labels and supports default labeling at tenant and workspace scope. **Best practices:** Set a tenant-level default label so every new Fabric item receives a classification if a user takes no action. Configure stricter default labels at the workspace level when that workspace routinely contains sensitive data. Require an intentional action to downgrade a label. This approach prevents unlabeled data from spreading. See [default label policy](#) and [default sensitivity labels](#) in Microsoft Fabric. **Decision guidance:** Choose conservative defaults when data sensitivity is uncertain. Choose lower defaults only in workspaces dedicated to public or broadly shared outputs. Higher defaults increase protection but might add friction for sharing.

3. **Enforce protection policies by label.** Protection policies control what users can do with labeled data. Microsoft Purview enforces these rules across supported Fabric items. **Best practices:** Define protection rules for each label tier. Use Microsoft Purview [protection policy](#) guidance. **Decision guidance:**

- **Label-only policies:** If your organization is starting with data classification, you might begin with labeling-only (no automatic enforcement) to focus on visibility and user education. **Tradeoff:** This approach is simpler to implement initially but relies on users following guidelines.
- **Protection policies:** If you already know you have sensitive data in OneLake that must be protected, configure label-based protection policies from the outset. This configuration ensures a user can't perform disallowed actions, like sharing it publicly or downloading it unencrypted. **Tradeoff:** There's added effort to design and test them, and the need to handle situations where they might block some legitimate access. This situation requires a process to override or refine policies.

4. **Define data loss prevention policies.** Data loss prevention policies detect and respond to risky data sharing or handling. Fabric integrates with Microsoft Purview data loss prevention (DLP) for [supported item type](#). **Best practices:** Configure DLP policies that align with your sensitivity labels and risk tolerance. Start with monitoring to understand real usage patterns. Move to blocking actions once policies show consistent accuracy.

This approach balances insight with control. **Decision guidance:** Choose monitor-only when user behavior is unknown and change resistance is likely. Choose block and restrict when exposure risk is high and policy confidence is strong. Blocking prevents leakage but can interrupt legitimate work.

5. **Use data lineage.** Data lineage shows how data moves and transforms across Fabric and how downstream assets depend on it. Fabric automatically captures lineage and shares it with Microsoft Purview. **Best practices:** Use lineage views to support impact analysis, audits, and data quality investigations. Share lineage insights with governance and risk teams through Purview reporting. Treat lineage as an operational control, not just documentation. Follow [lineage in Microsoft Fabric](#) and [Fabric lineage in Purview](#) guidance. **Decision guidance:** Decide how actively governance teams review lineage data. Regular review improves change management but requires sustained attention. Unplanned review reduces effort but increases the risk of missed dependencies.

For a broader view of how these controls work together, see [Microsoft Purview Information Protection in Microsoft Fabric](#).

4. Set a network security baseline

Control how Microsoft Fabric connects to on-premises systems and external services so data movement follows approved network paths and aligns with organizational risk tolerance. Decision makers who define this baseline reduce exposure risk while keeping connectivity choices consistent across teams. **Recommendation:** Define when teams must use public connectivity and when they must use private connectivity so all Fabric integrations follow a single network security standard. To apply this recommendation, review the following options:

Option 1: Public network connectivity: Public network connectivity allows Fabric to access external services over the internet by using encrypted endpoints. **When to choose:** Allow public connectivity for low-sensitivity data or external services that don't support private endpoints. Use this option for scenarios where speed of setup and operational simplicity matter most. Rely on encryption and Microsoft-managed endpoints to protect data in transit. Accept reduced network control in exchange for simpler configuration and lower operational overhead.

Option 2: Private network connectivity: Private network connectivity keeps traffic within controlled networks by avoiding public internet exposure. Microsoft Fabric supports private connectivity through Azure Private Link, managed virtual network integration for supported connectors, and private data gateways for on-premises sources. **When to choose:** Choose private connectivity when compliance obligations or risk posture require strict control over data paths. Accept added configuration effort and potential cost in exchange for stronger

network isolation and reduced exposure. For all the private networking options, see [Network security in Fabric](#).

5. Use the monitoring baseline

A monitoring baseline defines how leaders maintain platform health, reliability, and trust. Proactive oversight prevents outages and cost surprises. **Recommendation:** Define core metrics and centralized dashboards that provide continuous visibility into Fabric health, reliability, and spend. To apply this recommendation, use the following checklist:

1. **Monitor health metrics.** Health metrics show whether Fabric meets performance and reliability expectations. **Best practices:** Define a standard set of metrics, such as capacity utilization, query performance, and pipeline failure rates. Use a single view, such as the [Fabric Capacity Metrics app](#) to observe all domains together. Review metrics on a regular schedule to identify negative trends early. **Decision guidance:** Decide who owns metric oversight. A central operations team provides consistency and broad visibility but requires dedicated staffing. Data domain-led monitoring supports autonomy but risks missed cross-platform patterns. Many organizations centralize dashboards and standards while data domains act on local issues.
2. **Centralize reliability and performance oversight.** A unified view of reliability exposes systemic issues across domains. **Best practices:** Configure alerts for critical failures, such as refresh failures or processing delays. Route alerts to a central operations channel. Use the [Fabric Monitoring hub](#). **Decision guidance:** Decide who responds to platform-wide incidents. A central reliability team speeds a coordinated response but adds operational overhead. A domain-only response increases ownership but slows detection of cross-domain problems. Many organizations use a hybrid model where central teams handle widespread incidents.

6. Deployment governance

Deployment governance defines how data assets move into production. Consistency protects platform stability and security. **Recommendation:** Treat all data artifacts as code and enforce automated deployment through controlled pipelines. To apply this recommendation, use the following checklist:

1. **Put all data artifacts under version control.** Version control ensures traceability and repeatability across environments. **Best practices:** Require all pipelines, queries, configurations, and reports to live in a source repository. Enforce change history and rollback capability through that repository. **Decision guidance:** Decide whether to mandate this policy organization-wide at once or phase it by domain. Immediate

enforcement improves control but might require training. Phased adoption reduces disruption but extends risk exposure.

2. **Use CI/CD pipelines for releases.** Automated deployment reduces human error and configuration drift. **Best practices:** Require automated pipelines that test changes in pre-production and promote only validated updates to production. Prohibit direct production changes through portals or scripts. **Decision guidance:** Decide how much upfront investment to make. Full automation requires tooling and setup effort but enables faster recovery and consistent environments. Manual deployment appears faster short term but creates long-term operational risk.

Next step

By establishing these governance and security baselines for both your overall data estate and specifically for OneLake, you create a safe environment for data innovation. Good governance is invisible when done well. It operates in the background, making sure people have access to the data they need without compromising security or compliance. The next step is making sure you have robust operational processes so that data flows from sources to insights in a consistent, reliable manner.

Data processing standards

Last updated on 03/10/2026

Data processing standards for AI and analytics

✦ Summarize this article for me

A unified data platform depends on consistent ingestion, transformation, and publishing standards so leaders can trust data for analytics and AI. **Recommendation:** Establish organization-wide standards that control what data enters OneLake, how teams refine that data, and how governed data products reach consumers (see Figure 1). To apply this recommendation, use this article as a checklist:

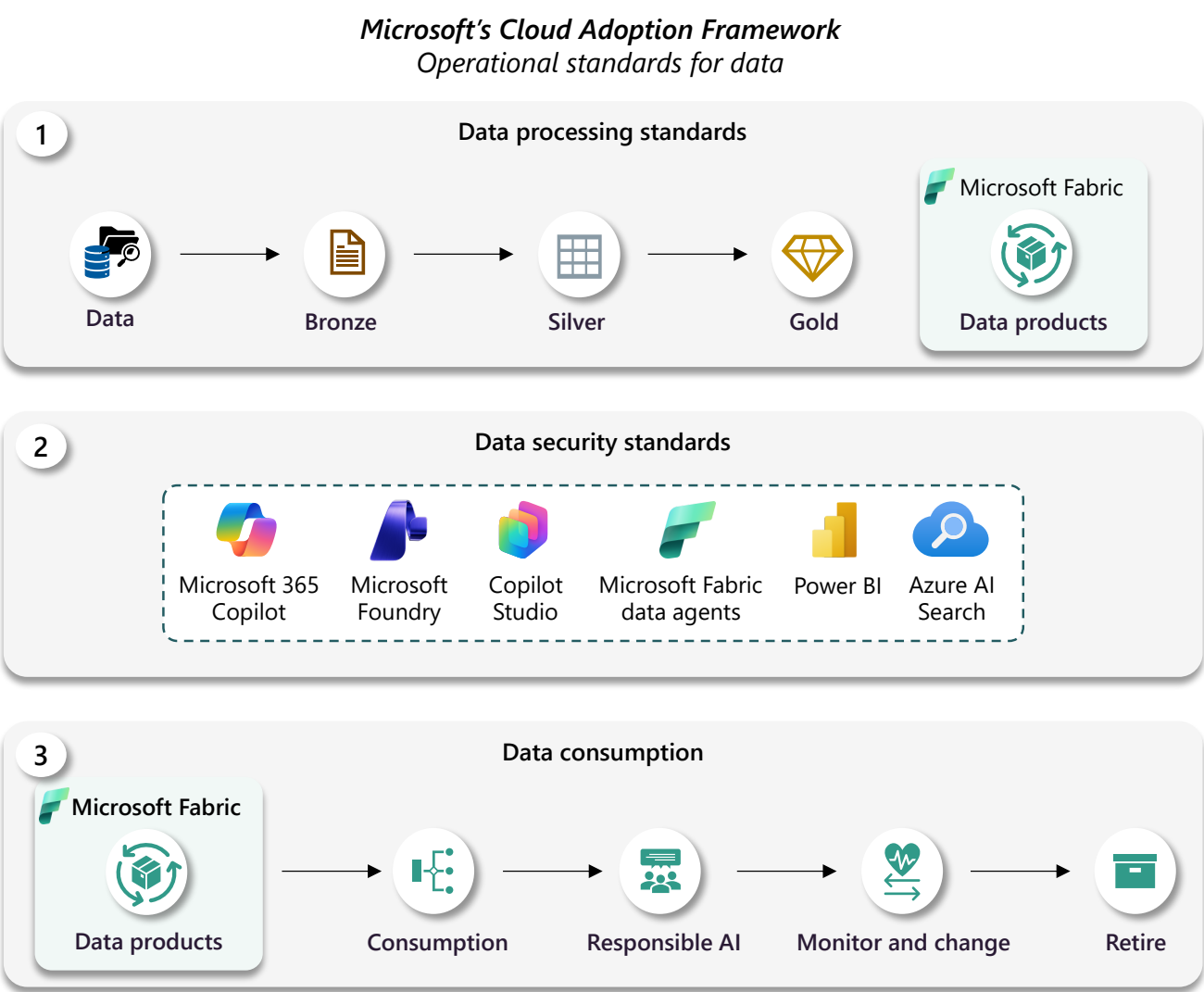


Figure 1. Three steps to setting operational standards for data.

1. Set data ingestion standards

OneLake serves as the central data lake for analytics and AI in Microsoft Fabric, so leaders must control what enters. **Recommendation:** Set clear boundaries so teams ingest only data that

supports defined business outcomes. To apply this recommendation, use the following checklist.

1. **What data do you unify?** Data unification in OneLake means selecting data that supports a data product tied to a measurable business outcome. **Best practices:** Only bring data into OneLake if it supports a data product and adds business value. Treat data ingestion as a product decision rather than a technical default. **Decision guidance:** Decide to ingest data when it explains or measures a business process or outcome that leaders track. Choose to leave data in operational systems or departmental storage when no active use case exists. This choice reduces storage cost and governance overhead.
2. **Keep collaboration and knowledge content in Microsoft 365.** Microsoft 365 data supports Microsoft 365 Copilot agents. These agents retrieve documents, emails, and collaboration artifacts while honoring existing permissions. See [Microsoft 365 Copilot architecture](#). **Best practices:** Use Microsoft 365 as the system of record for content whose primary purpose is collaboration, reference, or knowledge retrieval. Use the [Microsoft 365 adoption guide](#) to prepare this data.
3. **Integrate operational databases through supported patterns.** Operational databases often supply analytics and AI scenarios, but direct access creates risk and instability. **Best practices:** Use Microsoft Fabric-supported patterns, such as [shortcuts](#) for virtual access and [mirroring](#) for replicated access. Azure databases often require mirroring for reliable integration. **Decision guidance:** Choose shortcuts when virtual access meets performance needs. Choose mirroring when analytics performance, isolation, or downstream reuse requires a physical copy in OneLake.
4. **Integrate existing data lakes.** Many organizations already operate data lakes, such as Azure Data Lake Storage (ADLS), Google Cloud Storage, or Amazon S3. **Best practices:** Treat existing lakes as part of the unified data estate rather than forcing immediate migration. Use [shortcuts](#) or [mirroring](#). **Decision guidance:** Decide based on readiness and risk. To avoid duplication, choose shortcuts. Choose mirroring when consistency, performance, or compliance outweighs duplication cost.
5. **Surface Azure Databricks outputs in OneLake.** Azure Databricks often already produces curated analytics data. **Best practices:** Keep Databricks pipelines in place and surface final Gold outputs in OneLake. **Decision guidance:** Choose [shortcuts](#) to avoid replication and when remote access meets needs. Choose [mirroring](#) when governance or consumption patterns demand local data.
6. **Separate internal and external data intake.** Internal analytics data and external-facing data require different controls. **Best practices:** Create separate workspaces or lake areas for external data products. Store only approved external datasets in those locations. **Decision guidance:** Decide early whether data supports internal decisions or external

sharing. Choose physical separation to reduce accidental exposure and simplify security policy enforcement.

2. Set data transformation standards

Getting data into OneLake is only the first step. The real value comes from transforming raw data into high-quality, ready-to-use data products. Leaders don't design pipelines, but they define platforms and architectural standards that prevent fragmentation. **Recommendation:** Standardize transformation platforms and enforce a consistent refinement architecture. To apply this recommendation, use the following checklist:

2.1. Use the right data platform

Your choice of platform sets operational requirements for your data products in Microsoft and Azure environments. **Recommendation:** For each data product, balance simplicity and integration with the need for specialized engineering capabilities. To apply this recommendation, use the following checklist:

1. **Fabric (default).** Fabric provides integrated data engineering, analytics, and BI engines that operate directly on OneLake, which acts as the unified governed data estate for the organization. **Best practices:** Use Fabric for standard analytics, reporting, and data preparation. Favor native Fabric engines, such as Dataflows Gen2, Spark, and SQL to simplify access control, lineage, and cost management. Use OneLake as the single storage layer. **Decision guidance:** Choose Fabric when requirements fit built-in capabilities and when leadership values unified governance and billing. Accept limited customization in exchange for lower operational overhead.
2. **Azure Databricks.** Many people prefer Azure Databricks. It supports large-scale processing and advanced machine learning scenarios. **Best practices:** Continue using Databricks where expertise or scale already exists. Require outputs to land in OneLake or connect through OneLake shortcuts so governance, security, and discovery remain centralized. **Decision guidance:** Choose Databricks when Fabric doesn't meet current requirements. Accept higher integration and skill overhead as the tradeoff.
3. **Enforce platform ownership boundaries.** Clear platform boundaries prevent duplicated cost and inconsistent logic across systems. **Best practices:** Assign responsibility for each class of workload to one platform. Require an architectural review before approving cross-platform processing. **Decision guidance:** Decide which platform owns ingestion, transformation, and analytics outcomes. Prevent duplicate transformations and overlapping pipelines that deliver the same business result.

2.2. Apply medallion architecture

The medallion architecture establishes trust, consistency, and governance across all data products by defining a clear progression from raw data to business-ready outputs.

Recommendation: Require all data products in OneLake to follow a bronze, silver, and gold structure and prohibit shortcuts that bypass these layers. To apply this recommendation, use the following checklist:

1. **Mandate a bronze layer as the system of record (raw ingestion):** The bronze layer captures data exactly as it arrives in OneLake and preserves original source fidelity. **Best practices:** Store data as append-only and immutable. Prohibit corrections or enrichment at this stage. Require every inbound dataset to land in bronze first. **Decision guidance:** Decide that bronze exists only to preserve truth from source systems. Accept slower usability in exchange for auditability and traceability.
2. **Establish the silver layer as the trusted view.** The silver layer holds validated, standardized, and cleansed data that teams rely on for consistent analysis. **Best practices:** Apply data quality rules, format alignment, and basic business validation. Document silver datasets clearly and manage changes through governance processes. **Decision guidance:** Choose silver as the authoritative cleansed layer. Prohibit teams from recleaning raw data independently, which creates conflicting interpretations.
3. **Gold (business context, data products):** Certify gold datasets as business data products. The gold layer delivers governed data products that leaders use for decisions, performance tracking, and reporting. **Best practices:** Align gold data to approved business definitions and metrics. Optimize structures for consumption. Register every gold dataset as a data product in Microsoft Purview with ownership, purpose, and refresh details. **Decision guidance:** Decide that any dataset used across teams or for decisions must exist in gold. Reject unmanaged or uncertified datasets that bypass governance.
4. **Create sanitized products for external use.** External sharing requires deliberate separation from internal operational data. **Best practices:** Produce curated datasets that remove or mask sensitive fields and reduce detail where required. Assign ownership and apply clear labels, such as Public or External use. Store these datasets in approved locations. **Decision guidance:** Choose to treat external datasets as independent products. Accept added governance steps to reduce legal and security risk.

Fabric supports this model through [materialized lake views](#) that can automatically manage the transformations. See [Medallion Lakehouse Architecture in Fabric](#). For an analytics architecture, see [Analytics End-to-End with Microsoft Fabric](#).

Table. Example medallion architecture. Gold layer combines data from two datasets.

Dataset	Layer	Example Data	What happened
Sales transactions	Bronze	OrderID=984321 · StoreID= 17 · Amount="1,200" · TxnDate="2026-01-05T14:32:09Z"	This record arrived from the sales system exactly as sent. Amount is text. Timestamp follows system format. No meaning is applied.
	Silver	OrderID=984321 · StoreID= 17 · Amount= 1200.00 · TxnDate=2026-01-05	The transaction is standardized and validated. Amount is numeric. Date follows enterprise rules. Data is now trustworthy.
Store reference	Bronze	StoreID="17" · RegionName="EAST"	This record arrived from a location system. Formatting reflects the source.
	Silver	StoreID= 17 · Region=East	Store identifiers align with sales data. Region values are cleaned and consistent.
Daily revenue by region	Gold	Region=East · Date=2026-01-05 · TotalRevenue=425000	This value combines Silver sales transactions with Silver store reference data. Individual records are summarized to answer a business question.

2.3. Consider an adaptive gold layer

Adaptive Gold is included here as a forward-looking consideration. The idea is that you use AI agents to create gold layers. Agents can observe patterns that you might not be able to. If users frequently ask for "top customer issues by region per month," AI agents can materialize that dataset. This capability isn't provided out of the box in Microsoft Fabric today. It would require building a custom AI agent that operates on Fabric and Power BI telemetry.

3. Set data product publishing standards

Publishing standards define how your organization exposes trusted data products through Microsoft Fabric OneLake and Microsoft Purview. The goal is to scale reuse, enforce governance, and reduce risk across analytics and AI workloads. **Recommendation:** Establish a single publishing standard that makes every approved data product discoverable, governed, and clearly intended for a defined audience before broad use. To apply this recommendation, use the following checklist:

1. **Standardize publication through OneLake Catalog.** OneLake Catalog provides a unified access surface for data products across Fabric and external processing platforms, such as Databricks. **Best practices:** Use OneLake as the default execution and consumption layer

for all approved data products. Treat Microsoft Purview as the system of record for governance and business definitions. This alignment allows Power BI, Fabric data agents, and Azure AI Search to consume data consistently while enabling centralized governance visibility.

2. **Ensure discoverability.** Discoverability ensures decision makers and consumers can find trusted data products without relying on informal knowledge. **Best practices:** Configure Fabric workspace visibility so relevant audiences can discover items. They don't need access, just the ability to request access. Enable [Purview access request workflows](#) so users can request permission directly from the catalog. **Decision guidance:** Choose broad discoverability when the goal is reuse across domains. Choose limited discoverability when regulatory or confidentiality constraints apply. Balance visibility with access controls rather than hiding assets.
3. **Mandate metadata enrichment at publication.** Metadata provides context that allows leaders to assess fitness, trust, and reuse of a data product. **Best practices:** Require descriptive metadata at publication. Use [tags in Fabric](#) to classify products by business domain or initiative. Ensure descriptions explain the purpose and data scope. This practice supports catalog search and improves confidence in reuse. **Decision guidance:** Decide whether metadata requirements remain minimal or enforce a standard schema. Choose a standard schema when the organization operates multiple domains and needs consistency. Choose a lighter approach for early maturity stages.
4. **Endorse and certify when appropriate.** Define endorsement and certification criteria. Endorsement signals the trust level and governance maturity to the organization. **Best practices:** Use Promoted to indicate domain-recommended products. Use Certified to indicate products that passed a formal governance review. Apply certification to Gold datasets that support executive reporting or critical analytics. Reference Fabric endorsement guidance at [Endorsement](#). **Decision guidance:** Decide which products require certification. Choose mandatory certification for executive or regulatory workloads. Choose optional certification when speed and experimentation take priority. Accept slower onboarding as the tradeoff for higher trust.
5. **Publish as a data product in Purview.** Purview data products provide a higher-level view that groups assets into a managed product lifecycle. **Best practices:** Create a Purview data product entry for each published data product. Include product name, description, owners, quality status, and related assets, such as tables, models, and reports. Reference [Data Products in Unified Catalog](#). **Decision guidance:** Decide whether Purview data products are required for all published assets or only for strategic products. Choose full coverage when portfolio visibility matters. Choose selective coverage when governance capacity is limited.

6. **Declare intended audience and usage.** Clear intent prevents misuse and supports compliance across analytics and AI scenarios. **Best practices:** Require each data product to state its intended audience and supported workload type. Specify internal, partner, or public use. Identify AI, analytics, BI, or public web scenarios. Use Purview [metadata](#), [glossary terms](#), and [sensitivity labels](#) together to express this intent consistently. **Decision guidance:** Decide whether external or agent use requires extra approval. Choose stricter approval when data leaves organizational boundaries. Accept slower publication as the tradeoff for reduced risk.

Next step

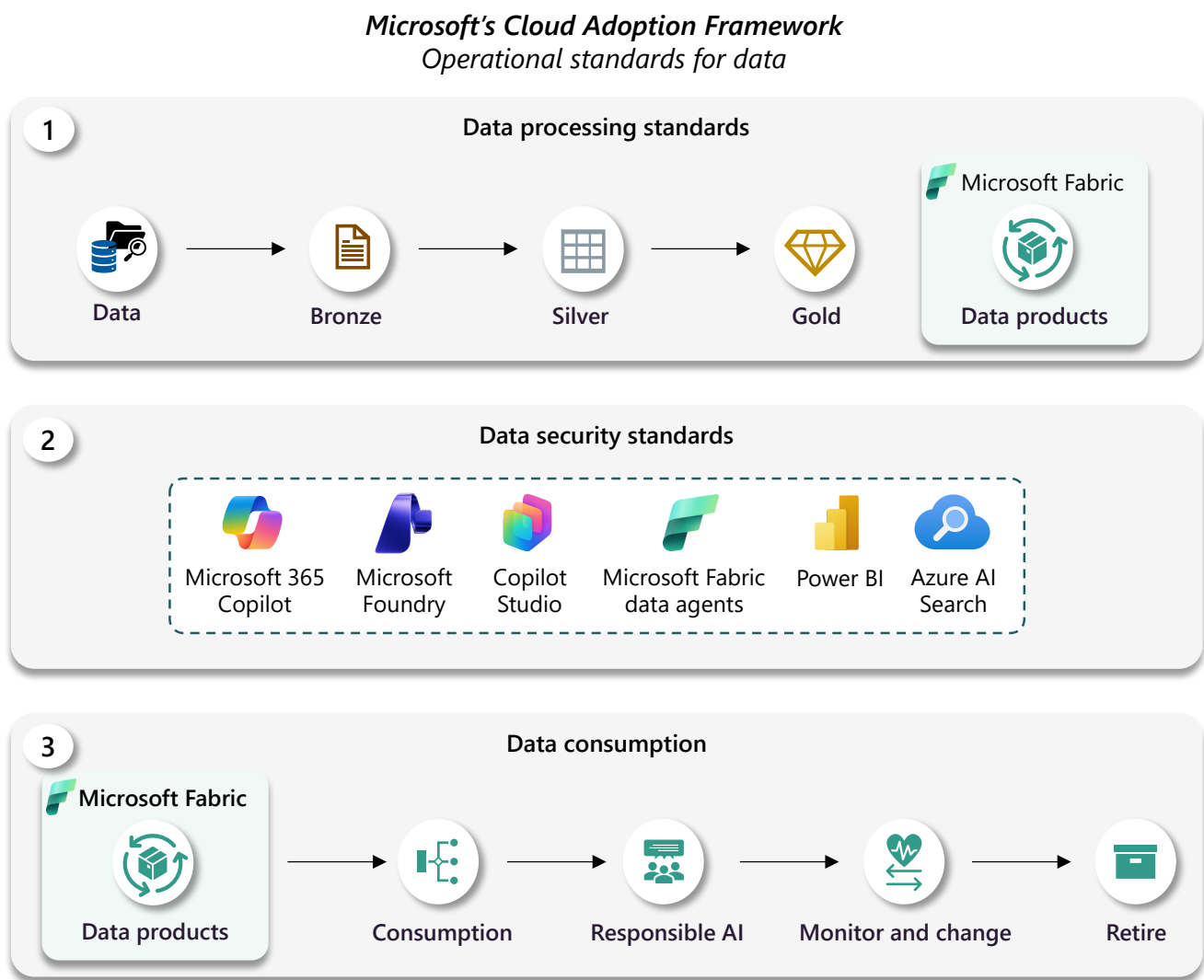
Data security standards

Last updated on 03/10/2026

Data security standards for AI and analytics

Summarize this article for me

AI and analytics consume data products in a unified data platform through multiple Microsoft services. Each service enforces security in a specific way. **Recommendation:** For every Microsoft product that can access data products, first understand its native access and security model. Then, decide whether to inherit it as-is or define standards where your requirements aren't met (see Figure 1). To apply this recommendation, use this article as a checklist:



Microsoft Azure Cloud Adoption Framework aka.ms/cafddata

Figure 1. Three steps to setting operational standards for data.

Business data in AI standard

Successful organizations publish a policy that limits business data use to services that honor tenant controls and don't reuse prompts for training. They also set an approval gate for any tool that can capture prompts or outputs. They train leaders to treat prompts as business

records. **Best practices:** Microsoft doesn't use that data to train foundation models. Treat the Microsoft boundary as a required security baseline for business data.

Microsoft 365 Copilot security standard

Access model in Microsoft 365 apps: Microsoft 365 Copilot can access only the data already available to a user within Microsoft 365. This data includes files, emails, chats, calendars, and other information exposed through Microsoft Graph and the [Semantic Index](#). Microsoft 365 Copilot uses the user's existing identity and respects all current access permissions. Copilot operates entirely within Microsoft's enterprise boundary, and prompt data isn't used for model training. **Best practices:** Continue to rely on Microsoft 365's built-in security model. [How does Microsoft 365 Copilot protect organizational data?](#)

Access model with Fabric data agents: When Microsoft 365 Copilot connects to a Fabric data agent, it continues to operate under the user's identity. Fabric remains responsible for enforcing workspace permissions, item permissions, and row- or column-level security. This security model makes sure users never see more data through Copilot than they're already entitled to view in Fabric. To learn more, see [Microsoft 365 Copilot and Fabric data agent](#).

Fabric data agent security standard

Access model: Fabric Data Agents are a governed interface to your data in Microsoft Fabric's OneLake. When a Copilot or Foundry queries a Fabric Data Agent, it must authenticate as a user. Fabric applies all its normal security filters on the data. In short, the Fabric data agent doesn't return data that the user can't retrieve in Fabric. Any sensitive-data policies down to rows and columns remain in force. Using a Fabric Data Agent doesn't expose any data by itself. It's another entry point that fully respects Fabric's governance settings. To learn more, see [Fabric data agent sharing and permission management](#). **Best practices:**

- **Use Fabric agents for internal data.** If your AI or reporting solution needs to pull structured data, require it to go through a Fabric Data Agent instead of bypassing with direct connectors. This approach guarantees unified enforcement of all data policies during AI consumption. For example, if an AI assistant in Teams needs sales figures, have it query a published Fabric agent so that the security policy automatically applies.
- **One agent per audience.** You can publish multiple Fabric Data Agents with different scopes. If different user groups should see different slices of data, create separate agents with appropriate security filters. Avoid a one-size-fits-all agent. This way, each agent experience delivers results appropriate to the caller's role.

- **Govern publishing.** You should govern how agents are published and shared to ensure quality and operational consistency. As a decision maker, you might require an approval step before publishing Fabric agents to the Microsoft 365 Copilot Agent Catalog. This step helps prevent low-quality agents from circulating, even though they can't bypass data access controls.

Azure AI Search security standard

Access model: Azure AI Search indexes data so applications and AI systems can retrieve it. It can read data from [Fabric OneLake](#) and other locations only when [granted explicit permission](#). However, by default, an Azure AI Search index doesn't automatically enforce your source permissions. Once data is indexed, any user or app with query access to the index could retrieve that content. **Best practices:**

1. **Enforce either labels or ACLs (or both).** Require [ACL-based filtering](#) and/or [Microsoft Purview sensitivity-label enforcement](#) for sensitive content. Avoid indexing sensitive files unless you have strong label or access control list (ACL) enforcement in place. **Decision guidance:** Choose Purview sensitivity labels when you want centralized policy control, such as automatically denying all "Secret" data. **Tradeoff:** You accept dependency on correct labeling and label governance. Choose ACL filtering when access needs to mirror file-by-file permissions and you can manage the metadata overhead. **Tradeoff:** You accept higher metadata-management cost. Choose both when you need defense in depth. **Tradeoff:** You accept more complexity in troubleshooting and validation.
2. **Index only what's approved:** Include only content that your organization is comfortable exposing broadly or can be secured through metadata. Avoid indexing sensitive files unless you have strong label or ACL enforcement in place. Maintain an inventory of search indexes and designate owners for them, so you know what data is being made searchable.
3. **Test with real user accounts.** Before deploying an AI solution that uses Azure Search, test the search queries with accounts of different roles to verify that restricted content truly stays filtered out. Catching a misconfigured index (one missing certain ACL entries) in testing prevents an accidental exposure later. This guidance applies to all platforms that use Azure AI Search. It includes Microsoft Foundry, Microsoft Copilot Studio, and any other system that queries Azure AI Search indexes.

Microsoft Foundry security standard

Access model: Microsoft Foundry enables custom AI agents to connect to data through multiple access paths. These options create material governance and security decisions for

leaders responsible for platform standards. **Recommendation:** Set an organization-wide access model that defines approved data integration paths for Foundry agents. To apply this recommendation, use the following checklist:

- **Use Foundry IQ.** Foundry IQ is a managed knowledge layer built on Azure AI Search. It enforces access through indexed metadata, such as sensitivity labels or access control lists, when those signals are present in the source and supported by the connector. **Best practices:** Treat Foundry IQ the same way you treat Azure AI Search. Require that all indexed content comes from sources with well-managed permissions or from data that has sensitivity labels in Microsoft Purview. Make metadata quality a prerequisite for approval. **Decision guidance:** Choose this option when you need consistent access enforcement across agents. Avoid it only when the data source can't provide reliable metadata. The tradeoff is broader coverage versus predictable security behavior. See [Connect Agents to Foundry IQ Knowledge Bases](#).
- **Use Fabric data agents.** Foundry agents can call Fabric Data Agents as tools. Fabric enforces its own fine-grained security and applies it consistently across analytics and AI experiences. **Best practices:** Treat Fabric Data Agents as the preferred access path for high-value or sensitive organizational data. This approach preserves one security model across Power BI, Copilot, and custom agents. **Decision guidance:** Decide which data categories must be accessed through Fabric only. Choose this option for governed data products and shared analytics assets. The tradeoff is architectural consistency versus limited integration paths. See [Microsoft Foundry](#) and [Microsoft Fabric](#).
- **Use Azure AI Search indexes.** Foundry agents can query an Azure AI Search index directly instead of using Foundry IQ. This path bypasses any other guardrails that a curated knowledge base might provide. **Best practices:** Require that any directly connected index enforces access through labels or access control lists. Assume that an unsecured index exposes all documents to all users. **Decision guidance:** Choose direct index access for scenarios where security controls are enforced at the index level. Prefer Foundry IQ when governance consistency matters. The tradeoff is flexibility versus risk of unintended data exposure. See [Connect an Azure AI Search index to Foundry agents](#).
- **Use MCP servers.** MCP servers enable Foundry agents to invoke tools that interact with external APIs, services, and data sources, including databases. **Best practices:** Require user-identity passthrough when an MCP tool accesses sensitive data or systems with user-specific permissions. Avoid MCP connections to systems that can't restrict access per user. **Decision guidance:** Decide if shared credentials are acceptable. Require user-based authentication for business data and operational systems. The tradeoff is operational simplicity versus strong access control. See [MCP server authentication](#).

Decision guidance:

- **Define an AI architecture standard:** Foundry supports many integration patterns, which increases flexibility but also increases risk without clear boundaries. **Best practices:** Define which connector types are approved for internal data, which require review, and which are disallowed. Align this standard with your existing data and security policies. **Decision guidance:** Decide which access paths are mandatory for internal data. Choose Fabric or approved knowledge bases when security consistency matters most. Restrict unvetted sources. The tradeoff is innovation speed versus predictable security outcomes.
- **Require oversight for high-risk agents:** Custom agents can combine multiple tools, which increases exposure when agents access sensitive systems or perform actions. **Best practices:** Require formal review for any agent that connects to sensitive systems or performs actions. Validate that every tool enforces authentication and authorization correctly. **Decision guidance:** Decide which agent classes require strict review. Apply lighter review to low-risk and read-only scenarios. The tradeoff is governance effort versus exposure reduction.
- **Require security competency as a condition for AI development:** Each Foundry connector enforces security differently, and misuse can bypass user-level controls without intent. **Best practices:** Require teams to understand the security model of every approved connector. Standardize on supported identity frameworks, such as Microsoft Entra ID. **Decision guidance:** Decide how security competency is enforced. Use gated access or formal validation for teams handling sensitive data. Apply lighter requirements for low-risk use. The tradeoff is enablement speed versus reduced misconfiguration risk.

Microsoft Copilot Studio security standard

Access model: Microsoft Copilot Studio enables your organization to build custom copilots (agents) that run within your Microsoft 365 tenant environment. These custom copilots can connect to various knowledge sources, such as SharePoint, OneDrive, databases, or even Fabric Data Agents, to retrieve information. Copilot Studio runs in Microsoft's cloud and processes prompts and responses inside the Microsoft 365 service boundary. Prompts and responses remain in the tenant, and Microsoft doesn't use them for model training, consistent with Microsoft 365 Copilot behavior.

Copilot Studio agents access organizational data by using the current user's Microsoft Entra identity. The agent can only retrieve content that the user already has permission to view in systems, such as SharePoint or OneDrive.

Best practices: To help govern outputs, use Purview [sensitivity labels](#) flow through responses. To block knowledge sources, administrators can apply [data policies](#) to limit which knowledge sources are available.

Copilot Studio can retrieve data products through Azure AI Search and Fabric data agents. Each option enforces security differently and must align with organizational standards. **Decision guidance:**

- **Azure AI Search:** Choose Azure AI Search when indexed content already uses ACLs or sensitivity labels. See [Azure AI Search](#).
- **Fabric data agents:** Choose Fabric data agents when you need native enforcement of workspace, item, row-level, and column-level security defined in Fabric. See [Fabric data agents](#) and [Consuming a data agent in Microsoft Copilot Studio](#).

Microsoft Power BI security standard

Access model: Power BI transforms OneLake data into reports and dashboards while preserving Microsoft Purview sensitivity labels. These protections extend to exported content when supported by upstream systems. **Decision guidance:** For leaders, Power BI represents the final mile of data exposure. Making sure labels are applied upstream guarantees that protections remain intact even when content is shared or exported. See [Power BI implementation planning](#).

Copilot in Fabric security standard

Access model: Copilot in Fabric operates entirely within Fabric and acts only on behalf of the signed-in user. It doesn't introduce a new access path and can't exceed existing permissions. Outputs remain private until explicitly shared. See [Privacy, Security, and Responsible AI Use of Copilot in Fabric](#). See [diagram of Copilot in Fabric](#).

Next step

Data consumption standards

Last updated on 03/10/2026

Data consumption standards for AI and analytics



Summarize this article for me

Data product consumption defines how data products in Microsoft Fabric are used across the organization. This process affects trust, security, and long-term platform success.

Recommendation: Establish organization-wide standards for how teams consume and manage all published data products in Fabric over time. This approach helps teams safely reuse data without creating risk or instability (see Figure 1). To apply this recommendation, use this article as a checklist:

Microsoft's Cloud Adoption Framework Operational standards for data

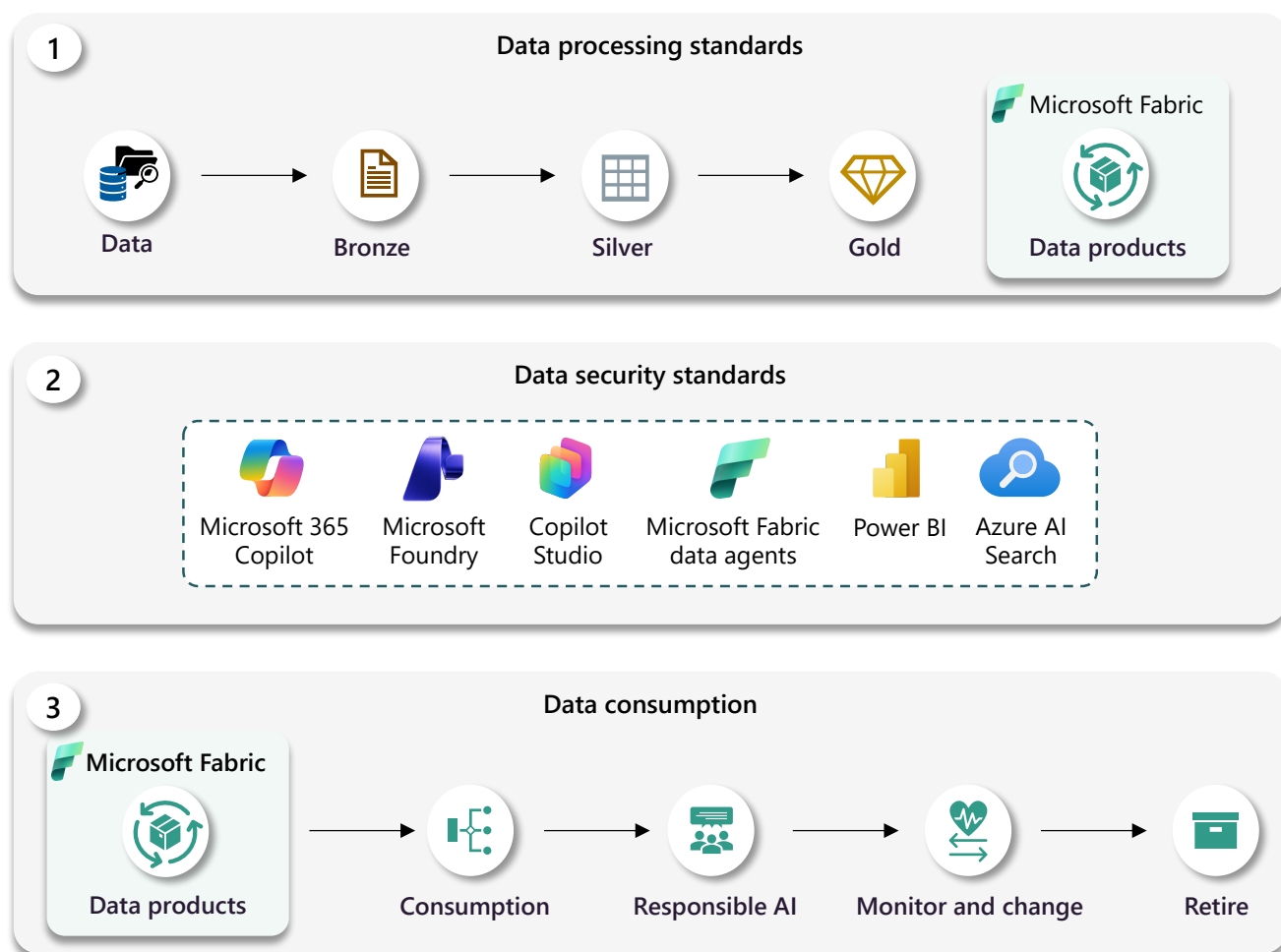


Figure 1. Three steps to setting operational standards for data.

Data consumption standards

Data consumption standards define when and how teams might rely on shared data products for business decisions, reporting, and automated systems. Leaders who set these standards reduce risk, improve trust, and create predictable reuse across Microsoft Fabric and connected services. **Recommendation:** Establish clear rules that control which data products teams might consume, how teams approve new uses, how access is granted, and how consumption paths are validated before production use. To apply this recommendation, use the following checklist:

- 1. Require official status before broad consumption.** Official status means a data product is approved for reuse beyond the owning team and is recognized as trusted at the organization level. **Best practices:** Require all shared data products to have an explicit official status before broad consumption. Use consistent signals to show which products are approved for reuse. Make sure each official product has a clear purpose and accountable ownership. Treat official status as the entry point for lifecycle governance. **Decision guidance:** Decide which endorsement level applies to each product category, such as [Fabric endorsement levels](#). Use lighter endorsement for internal reuse. Use certification for cross-domain or regulated use. Higher endorsement increases confidence but requires more reviews.
- 2. Approve significant new uses of data products.** Don't assume a data product is fit for every purpose without review. **Best practices:** Require an approval step when a team wants to use a data product in a critical scenario. Examples include external customer exposure or AI decision support. Keep the step light. Make it fast. Keep records in the product's documentation. Use [data lineage](#) to understand upstream and downstream dependencies before you approve a new use. This approach helps you see what the product depends on and what depends on it. **Decision guidance:** Define which scenarios need formal approval. Internal business-as-usual analyses can be exempt or covered by broad policy for agility. External uses, regulatory-sensitive scenarios, or uses of data in AI systems that make autonomous decisions should trigger a formal review and sign-off. While an approval step can introduce a short delay, it helps prevent misuse of data.
- 3. Enforce least-privilege access for consumers.** Only the required data should be accessible to any consumer, whether a user or an application. **Best practices:** When granting access to a data product, scope it to the minimum necessary for the task. If a particular report or application only needs aggregated data, provide access to an aggregated view rather than the detailed underlying data. Set access standards by using the Fabric [permission model](#) and [OneLake security](#). Favor item-level access over workspace-wide access. Require periodic access review for official products. **Decision guidance:** Decide which access scope applies to each product tier. Broader access reduces friction but increases exposure. Narrow access improves security but adds governance effort.

4. **Validate consumption paths before production use.** Validation confirms that data access and protection behave as intended when consumers use the product. **Best practices:** Before any new report, application, or AI agent goes live by using a data product, require a testing phase. In this phase, the team should simulate how the data is used and confirm security is working correctly. For example, use a test user account with limited permissions to see if they can only retrieve what they should. To test protection without enforcement, use [Purview's data loss prevention \(DLP\) simulation](#). **Decision guidance:** Decide which validation depth applies per product tier. Lightweight validation suits low-risk products. Deeper validation suits sensitive products. More validation reduces incidents but increases lead time.

Responsible AI standards

AI consumption means a data product is used to train or ground AI systems. **Best practices:** If the data product feeds into an AI system, incorporate a Responsible AI review. This review checks the data for biases. Does it over-represent or under-represent certain groups? Make sure it has appropriate sensitivity labels, so the AI knows what's sensitive. For example, if you have an AI agent answering questions about HR data, verify that the data is accurate and shared appropriately. If you train a model on the data, document what time period or population the data covers, so the AI's results are interpreted correctly. **Decision guidance:** Decide which AI use cases require enhanced review. Apply stricter review when AI influences decisions or automates actions. Stronger review reduces risk but limits speed.

Monitoring and change management standards

Monitoring and change standards protect trust in data products at scale and reduce operational risk for the organization. Decision makers set these standards to balance reliability, cost, and agility across all governed data products. **Recommendation:** Establish clear monitoring and change expectations for all official data products and scale those expectations based on business criticality. To apply this recommendation, use the following checklist:

1. **Monitor usage and operational health.** Define monitoring as the ongoing oversight of usage patterns that affect trust in a data product. Decision makers rely on monitoring to detect issues early and to understand business impact. **Best practices:** Require ongoing monitoring for all official data products. Use [Fabric monitoring](#) to track refresh health and failures. Use lineage views to understand downstream impact when an issue occurs. Treat monitoring as a standard control, not an optional feature. **Decision guidance:** Decide which data products require enhanced monitoring. Choose enhanced monitoring for products that support critical business decisions or that have broad adoption. Choose

baseline monitoring for low-impact or limited-scope products. Enhanced monitoring improves resilience but consumes more capacity and operational effort.

2. **Set change management standards.** Define change management as the process that governs how updates to a data product affect consumers and dependent systems. Decision makers use change management standards to protect consumers from unexpected disruption. **Best practices:** When you need to change a data product, such as altering its schema, recalculating a metric, or changing its refresh schedule, inform all stakeholders well in advance. Use tools to help identify the effects. For changes to a Power BI semantic model, use [impact analysis](#). Again, use [data lineage](#) to understand upstream and downstream dependencies. Use multiple channels if necessary. Make updates in the Microsoft Purview Unified Catalog with notes on the upcoming change. Send out notifications or release notes to known users of the data product. If applicable, announce it in relevant team meetings or internal forums. Clearly explain what is changing, when it happens, and the reason for the change. Explain whether consumers need to do anything, such as update their queries or reports. **Decision guidance:** Decide which change types require longer notice. Breaking changes need more notice. Nonbreaking changes need less. Longer notice protects consumers but reduces agility.

Data product retirement standards

Retire data products in a controlled manner. Retirement removes a data product from active use without breaking dependent workloads. **Best practices:** Require a documented retirement plan for all official data products. Remove endorsement before retirement. Preserve Purview metadata and lineage for audit and investigation. **Decision guidance:** Decide retirement timelines based on dependency depth. Short timelines reduce cost. Longer timelines reduce disruption. Choose based on business needs.

Next step

The next step is to establish standards for how AI apps and agents consume your data products.

General AI adoption

AI Agent adoption

Microsoft Fabric Well-Architected documentation

Learn about designing workloads on Microsoft Fabric with reliability, security, cost optimization, operational excellence, and performance efficiency in mind to build sustainable, scalable analytics platforms.

Get started

OVERVIEW

[What is a Well-Architected Microsoft Fabric workload?](#)

[Common architectural challenges](#)

[Architecture patterns](#)

CONCEPT

[Design methodology for Fabric workloads](#)

Optimize reliability

CONCEPT

[Know your constraints and boundaries](#)

[Understand where failures happen](#)

[Define reliability goals for your workloads](#)

[Build self-healing through redundancy](#)

[Plan for disaster recovery](#)

Secure your workloads

CONCEPT

[Start with a security baseline](#)

Design isolation boundaries

Use identity as the primary security perimeter

Secure network communications

Harden workload assets

Optimize costs

CONCEPT

Identify key cost drivers

Model costs effectively

Monitor costs and implement governance

Optimize environment costs

Reduce costs through automation

Achieve operational excellence

CONCEPT

Prepare your team

Deploy changes safely

Automate operations

Monitor your environment

Have an incident response plan

Improve performance efficiency

CONCEPT

Plan capacity deliberately

Design scaling strategies

Monitor performance issues

Do performance testing

Optimize performance

Understand medallion lakehouse architecture for Fabric with OneLake

The medallion lakehouse architecture, commonly known as *medallion architecture*, is a design pattern that's used to organize data in a lakehouse. It's the recommended design approach for Fabric. Since OneLake is the data lake for Fabric, medallion architecture is implemented by creating lakehouses in OneLake.

Medallion architecture comprises three distinct layers. The three medallion layers are: bronze (raw data), silver (enriched data), and gold (curated data). Each layer indicates the quality of data stored in the lakehouse, with higher levels representing higher quality.

Medallion architecture helps your data stay accurate and reliable according to the principles of atomicity, consistency, isolation, and durability (ACID). Your data starts in its raw form, and the original copies are preserved as a source of truth while your pipelines of validations and transformations prepares the data for analytics.

For more information, see [What is the medallion lakehouse architecture?](#).

Audience

This article introduces medallion lake architecture and describes how you can implement the design pattern in Fabric. It's targeted at multiple audiences:

- **Data engineers:** Technical staff who design, build, and maintain infrastructures and systems that enable their organization to collect, store, process, and analyze large volumes of data.
- **Center of Excellence, IT, and BI teams:** The teams that are responsible for overseeing analytics throughout the organization.
- **Fabric administrators:** The administrators who are responsible for overseeing Fabric in the organization.

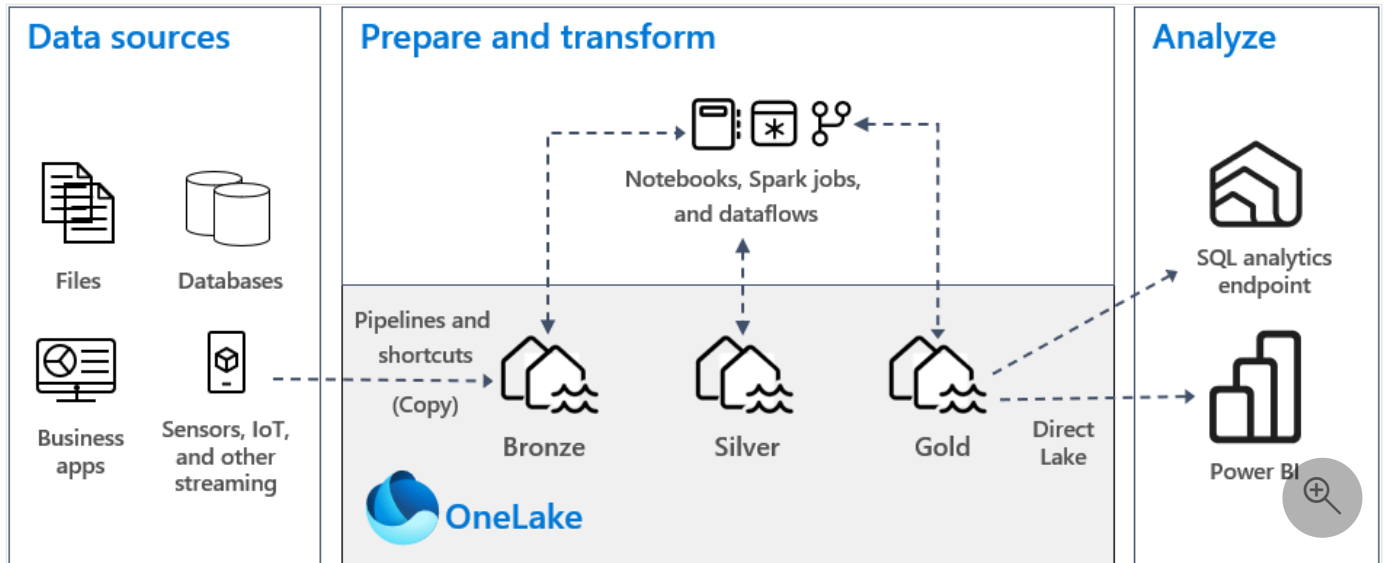
What is medallion architecture?

The goal of medallion architecture is to incrementally improve the structure and quality of data. Think of medallion architecture as a three-stage cleaning and organizing process for your data. Each layer makes your data more reliable and easier to use.

1. **Bronze (Raw):** Store everything exactly as it arrives. No changes are allowed.

2. **Silver (Enriched)**: Fix errors, standardize formats, and remove duplicates.
3. **Gold (Curated)**: Organize for reports and dashboards.

Keep each layer separated in its own lakehouse or warehouse in OneLake, with data moving between the layers as it's transformed and refined.



In a typical medallion architecture implementation in Fabric, the bronze layer stores the data in the same format as the data source. When the data source is a relational database, Delta tables are a good choice. The silver and gold layers should contain Delta tables.

💡 Tip

To learn how to create a lakehouse, work through the [Lakehouse end-to-end scenario](#) tutorial.

Real-world example

Consider the following example of an e-commerce company that applies medallion architecture to its data:

Bronze Layer:

- Store raw sales data from website (JSON)
- Store raw inventory data from warehouse (CSV)
- Store raw customer data from CRM (SQL export)

Silver Layer:

- Standardize date formats across all sources

- Convert all currency to USD
- Remove test transactions
- Match customer records across systems

Gold Layer:

- Create daily sales dashboard table
- Build customer lifetime value table
- Generate inventory forecasting table

Medallion architecture in OneLake

The basis of a modern data warehouse is a data lake. OneLake is a single, unified, logical data lake for your entire organization. It comes automatically provisioned with every Fabric tenant, and it's the single location for all your analytics data.

To store data in OneLake, you create a *lakehouse* in Fabric. A lakehouse is a data architecture platform for storing, managing, and analyzing structured and unstructured data in a single location. It can scale to large data volumes of all file types and sizes, and because the data is stored in a single location, it can be shared and reused across the organization.

For more information, see [What is a lakehouse in Fabric?](#).

Tables and files

When you create a lakehouse in OneLake, two physical storage locations are provisioned automatically:

- **Tables** stores tables of all formats in Apache Spark (CSV, Parquet, or Delta).
- **Files** stores data in any file format. If you want to create a table based on data in the files area, you can create a [shortcut](#) that points to the folder that contains the table files.

In the bronze layer, you store data in its original format, which might be either tables or files. If the source data is from OneLake, Azure Data Lake Store Gen2 (ADLS Gen2), Amazon S3, or Google, create a shortcut in the bronze layer instead of copying the data across.

In the silver and gold layers, you typically store data in Delta tables. However, you can also store data in Parquet or CSV files. If you do that, you must explicitly create a shortcut or an external table with a location that points to the unmanaged folder that contains the Delta Lake files in Apache Spark.

In Fabric, the [Lakehouse explorer](#) provides a unified graphical representation of the whole lakehouse for users to navigate, access, and update their data.

Delta Lake storage

Delta Lake is an optimized storage layer that provides the foundation for storing data and tables. It supports ACID transactions for big data workloads, and for this reason it's the default storage format in a Fabric lakehouse.

Delta Lake delivers reliability, security, and performance in the lakehouse for both streaming and batch operations. Internally, it stores data in Parquet file format, however, it also maintains transaction logs and statistics that provide features and performance improvement over the standard Parquet format.

Delta Lake format delivers the following benefits compared to generic file formats:

- Support for ACID properties, especially durability to prevent data corruption.
- Faster read queries.
- Increased data freshness.
- Support for both batch and streaming workloads.
- Support for data rollback by using [Delta Lake time travel](#).
- Enhanced regulatory compliance and audit by using [Delta Lake table history](#).

Fabric standardizes storage file format with Delta Lake. By default, every workload engine in Fabric creates Delta tables when you write data to a new table. For more information, see [Lakehouse and Delta Lake tables](#).

Deployment model

To implement medallion architecture in Fabric, you can either use lakehouses (one for each layer), a warehouse, or combination of both. Your decision should be based on your preference and the expertise of your team. With Fabric, you can use different analytic engines that work on the one copy of your data in OneLake.

Here are two patterns to consider:

- **Pattern 1:** Create each layer as a lakehouse. In this case, business users access data by using the SQL analytics endpoint.
- **Pattern 2:** Create the bronze and silver layers as lakehouses, and the gold layer as a data warehouse. In this case, business users access data by using the data warehouse endpoint.

While you can create all lakehouses in a single [Fabric workspace](#), we recommend that you create each lakehouse in its own, separate workspace. This approach provides you with more control and better governance at the layer level.

For the bronze layer, we recommend that you store the data in its original format, or use Parquet or Delta Lake. Whenever possible, keep the data in its original format. If the source data is from OneLake, Azure Data Lake Store Gen2 (ADLS Gen2), Amazon S3, or Google, create a [shortcut](#) in the bronze layer instead of copying the data across.

For the silver and gold layers, we recommend that you use Delta tables because of the extra capabilities and performance enhancements they provide. Fabric standardizes on Delta Lake format, and by default every engine in Fabric writes data in this format. Further, these engines use V-order write-time optimization to the Parquet file format. That optimization enables fast reads by Fabric compute engines, such as Power BI, SQL, Apache Spark, and others. For more information, see [Delta Lake table optimization and V-order](#).

Lastly, today many organizations face massive growth in data volumes, together with an increasing need to organize and manage that data in a logical way while facilitating more targeted and efficient use and governance. That can lead you to establish and manage a decentralized or federated data organization with governance. To meet this objective, consider implementing a *data mesh architecture*. [Data mesh](#) is an architectural pattern that focuses on creating data domains that offer data as a product.

You can create a data mesh architecture for your data estate in Fabric by creating data domains. You might create domains that map to your business domains, for example, marketing, sales, inventory, human resources, and others. You can then implement medallion architecture by setting up data layers within each of your domains. For more information about domains, see [Domains](#).

Use materialized lake views for medallion architecture

[Materialized lake views](#) in Fabric help you to implement medallion architecture in your lakehouse. Rather than building complex pipelines to transform data between bronze, silver, and gold layers, you can define materialized lake views that automatically manage the transformations.

Key benefits of using materialized lake views for medallion architecture include:

- **Declarative pipelines:** Define data transformations using SQL statements rather than building manual pipelines between layers.

- **Automatic dependency management:** Fabric automatically determines the correct execution order based on view dependencies.
- **Data quality rules:** Built-in support for defining and enforcing data quality constraints as data moves through layers.
- **Optimal refresh:** The system automatically determines whether to perform incremental, full, or no refresh for each view.
- **Visualization and monitoring:** View lineage across all layers and track execution progress.

For example, you can create a silver layer view that cleanses and joins data from bronze tables, and then create gold layer views that aggregate the silver layer data for reporting. The system handles the refresh orchestration automatically.

For more information, see [Implement medallion architecture with materialized lake views](#).

Understand Delta table data storage

This section describes other guidance related to implementing a medallion lakehouse architecture in Fabric.

File size

Generally, a big data platform performs better when it has a few large files rather than many small files. Performance degradation occurs when the compute engine has many metadata and file operations to manage. For better query performance, target file sizes between 128 MB and 1 GB, depending on the table size and the layer's consumption pattern.

Different layers of the medallion architecture have different requirements for file size based on which consumption engine is used. In the bronze layer, you can use smaller files because of the raw nature of the data, as long as you focus data modification and preparation on Spark. In the silver layer, use moderate file sizes to balance write and read performance. In the gold layer, optimize for larger file sizes and larger row groups to improve query performance for consumption engines. To learn more about optimizing file sizes for different layers, see [Cross-workload table maintenance and optimization](#).

Historical retention

By default, Delta Lake maintains a history of all changes made, so the size of historical metadata grows over time. Based on your business requirements, keep historical data only for a certain period of time to reduce your storage costs. Consider retaining historical data for only the last month, or other appropriate period of time.

You can remove older historical data from a Delta table by using the [VACUUM command](#). However, by default you can't delete historical data within the last seven days. That restriction maintains the consistency in data. Configure the default number of days with the table property `delta.deletedFileRetentionDuration = "interval <interval>"`. That property determines the period of time that a file must be deleted before it can be considered a candidate for a vacuum operation.

Table partitions and clustering

When you store data in each layer, use a partitioned folder structure wherever applicable. This technique improves data manageability and query performance. Generally, partitioned data in a folder structure results in faster search for specific data entries because of partition pruning or elimination. In the bronze layer, partitioning is acceptable but discouraged for new implementations. For silver and gold layers, use Liquid Clustering instead of partitioning to optimize query performance. To learn more about optimizing for different layers, see [Cross-workload table maintenance and optimization](#).

Typically, you append data to your target table as new data arrives. However, in some cases you might merge data because you need to update existing data at the same time. In that case, you can perform an *upsert* operation by using the [MERGE command](#). When your target table is partitioned, be sure to use a partition filter to speed up the operation. That way, the engine can eliminate partitions that don't require updating.

Data access

You should plan and control who needs access to specific data in the lakehouse. You should also understand the various transaction patterns they're going to use while accessing this data for each layer.

Tip

Each medallion layer has different optimization requirements. For comprehensive guidance on table maintenance strategies for bronze, silver, and gold layers, including when to enable V-order and optimal file sizes, see [Cross-workload table maintenance and optimization](#).

Related content

For more information about implementing medallion lakehouse architecture, see the following resources.

- [Cross-workload table maintenance and optimization](#)
- [Delta Lake table optimization and V-Order](#)
- [Tutorial: Lakehouse end-to-end scenario](#)
- [Tutorial: Implement medallion architecture with materialized lake views](#)
- [Lakehouse and Delta Lake tables](#)
- [Fabric decision guide: choose a data store](#)
- [The need for optimize write on Apache Spark](#)
- Questions? Try asking the [Fabric community](#) [↗](#).
- Suggestions? [Contribute ideas to improve Fabric](#) [↗](#).

ⓘ **Note:** The author created this article with assistance from AI. [Learn more](#)

Last updated on 07/17/2026

Architecture best practices for Azure Databricks



Summarize this article for me

Azure Databricks is an Apache Spark-based analytics platform that data teams can use to collaborate efficiently. Use it to build and deploy data engineering, machine learning, and analytics workloads at scale. This article covers common considerations and best practices for Azure Databricks and gives architectural recommendations mapped to the [Azure Well-Architected Framework pillars](#).

It's assumed that as an architect, you've reviewed [Choose an analytical data store](#) and chose Azure Databricks as the analytics platform for your workload.

Technology scope

This review focuses on the interrelated decisions for the following Azure resources:

- Azure Databricks
- Spark
- Delta Lake
- Unity Catalog
- MLflow

Reliability

The purpose of the Reliability pillar is to provide continued functionality by **building enough resilience and the ability to recover fast from failures**.

[Reliability design principles](#) provide a high-level design strategy applied for individual components, system flows, and the system as a whole.

Workload design checklist

Start your design strategy based on the [design review checklist for Reliability](#). Determine its relevance to your business requirements while keeping in mind the nature of your application and the criticality of its components. Extend the strategy to include more approaches as needed.

- ✓ **Understand service limits and quotas.** Azure Databricks [service limits](#) directly constrain workload reliability through compute clusters, workspace capacity, storage throughput, and network bandwidth restrictions. Your architecture design must proactively incorporate

quotas to prevent unexpected service disruptions that can halt scaling operations during peak demand. These quotas include a 1000-node cluster limit, workspace cluster maximums, and regional capacity constraints.

- ✔ **Use failure mode analysis (FMA) to anticipate potential failures.** Systematic [FMA](#) identifies potential system failures and establishes corresponding mitigation strategies to maintain distributed computing resilience.

The following table includes common failure scenarios and their proven mitigation approaches.

 [Expand table](#)

Failure	Mitigation
Cluster driver node failure	Use cluster automatic-restart policies and implement checkpointing for Spark applications. Use structured streaming with fault-tolerant state management.
Job execution failures	Implement retry policies with exponential backoff. Use Azure Databricks job orchestration with error handling. Set up appropriate timeout settings.
Data corruption or inconsistency	Use Delta Lake atomicity, consistency, isolation, and durability (ACID) transactions, time travel capabilities, and data expectations in Lakeflow Spark Declarative Pipelines. Implement data validation checks and monitoring.
Workspace or region unavailability	Implement multiregion deployment strategies. Use workspace backup and restore procedures. Set up cross-region data replication.

These mitigation strategies use native Azure Databricks capabilities like automatic restart, automatic scaling, Delta Lake consistency guarantees, and Unity Catalog security features for fault tolerance.

- ✔ **Design to support redundancy across the critical layers.** Redundancy in the critical architectural layers is key to maintaining workload continuity.

For example, distribute clusters across availability zones by using diverse instance types and cluster pools and by implementing automatic node replacement policies. Reliable network design also protects against connectivity failures that can disrupt control plane reachability, data access, and communication with dependencies. Use redundant network paths, diverse private endpoint configurations, Domain Name System (DNS) failover mechanisms, and virtual network injection to achieve network resilience. Metadata resilience is important for maintaining compliance and data accessibility during service disruptions because governance failures can halt data access and compromise compliance requirements.

For higher availability, consider using multiregion Azure Databricks deployments for geographic redundancy. This approach helps protect against regional outages and ensures business continuity (BC) during extended service disruptions. Multiregion setup is also a viable solution for disaster recovery (DR).

- ✓ **Implement scaling strategies.** Use automatic scaling to handle demand fluctuations while maintaining consistent performance. Plan for resource provisioning delays and regional capacity limits. Balance the trade-off between scaling responsiveness and cluster startup latency during peak demand.
- ✓ **Adopt serverless compute for improved reliability.** Serverless compute options reduce operational complexity and improve reliability by shifting infrastructure management to Microsoft. This approach provides automatic scaling, built-in fault tolerance, and consistent availability without cluster life-cycle management overhead.
- ✓ **Implement comprehensive health monitoring and alerting.** Use [comprehensive monitoring](#) across all Azure Databricks components to proactively detect problems and respond before they affect availability. Implement automated escalation workflows for workspace health, cluster status, job execution patterns, and data pipeline performance.
- ✓ **Protect data by using Delta Lake reliability features.** Delta Lake provides essential data protection through ACID transactions, automatic versioning, time travel capabilities, and schema enforcement. These features prevent corruption and help you recover from data problems.
- ✓ **Set up job reliability and retry mechanisms.** Job reliability configurations establish resilient data processing by using intelligent retry policies, timeout management, and failure handling mechanisms that distinguish between transient problems and permanent errors.
- ✓ **Build data pipeline resilience and fault tolerance.** Data pipeline resilience addresses the critical reliability challenges of distributed data processing where failures can cascade throughout interconnected data systems and disrupt business analytics workflows.

Advanced resilience strategies use Lakeflow Spark Declarative Pipelines, structured streaming checkpoints, Auto Loader rescued data capabilities, and Lakeflow Spark Declarative Pipelines quality constraints to provide automatic error handling, data quality enforcement, and graceful degradation during infrastructure disruptions.

- ✓ **Establish backup and DR procedures.** Effective [DR](#) requires you to align recovery time objectives (RTOs) with business requirements and establish automated backup processes for workspace metadata, notebook repositories, job definitions, cluster configurations, and integrated data storage systems.

If you use a secondary region for recovery, account for workspace metadata synchronization, code repository replication, and coordinated integration with dependent Azure services to maintain operational continuity across geographic boundaries.

- ✓ **Implement reliability testing and chaos engineering.** Systematic reliability testing validates that failure recovery mechanisms function correctly in real-world scenarios. Apply chaos engineering principles in your tests to identify resilience gaps before they affect production environments.

Recommendations

 Expand table

Recommendation	Benefit
Set up cluster autoscaling with a minimum node count of two and a maximum node count that aligns with workspace quota limits. Set target utilization thresholds between 70% and 80% to balance cost efficiency with the ability to increase performance.	Automatic scaling uses dynamic node allocation to prevent cluster resource exhaustion while maintaining cost efficiency. Set up appropriate limits to ensure that workloads remain within service quotas. This approach helps you avoid jobs that fail because they exceed workspace capacity constraints.
Deploy Azure Databricks workspaces across multiple Azure regions for mission-critical workloads. Set up workspace replication with automated backup of source code, job definitions, and cluster configurations by using Databricks Asset Bundles and Azure DevOps or Azure Data Factory pipelines.	Multiregion deployments provide geographic redundancy that maintains data processing capabilities during regional outages or disasters. Automated workspace replication reduces RTOs from hours to minutes by ensuring consistent configurations across regions. This approach supports BCuity requirements and minimizes operational effects during extended regional service disruptions.
Establish cluster pools that have prewarmed instances by using different virtual machine (VM) sizes within the same family. Set up pool sizes to maintain 20% to 30% more overhead capacity than typical workload requirements.	Prewarmed cluster pools reduce cluster startup time from 5 to 10 minutes to less than 60 seconds, which helps your workload recover faster from node failures. Different VM sizing within pools ensures that cluster provisioning succeeds even when specific instance types face capacity constraints.
Activate Delta Lake time travel features by setting up automatic table versioning and retention policies. Set retention periods based on recovery requirements, which are typically 7 to 30 days for production tables.	Time travel capabilities provide point-in-time recovery (PITR) without requiring external backup systems or complex restore procedures. Automatic versioning protects against data corruption and accidental modifications while maintaining complete data lineage for compliance and debugging purposes. This approach eliminates the need for separate backup

Recommendation	Benefit
	infrastructure while ensuring rapid recovery from data problems.
Integrate Azure Databricks with Azure Monitor by enabling diagnostic logs for cluster events, job execution, and data analytics. Set up custom metrics and alerts for cluster health, job failure rates, and resource utilization thresholds.	Centralized monitoring provides unified observability across all Azure Databricks components so that you can proactively detect problems before they affect production workloads. Custom alerting reduces mean time to resolution (MTTR) by automatically notifying teams when clusters experience performance degradation or job failures exceed acceptable thresholds.
Deploy serverless SQL warehouses for unplanned analytics and reporting workloads that require consistent availability without cluster management overhead.	Serverless infrastructure eliminates cluster provisioning delays and provides automatic scaling with built-in high availability guarantees. Microsoft manages all infrastructure patching, updates, and failure recovery, which reduces operational complexity and helps ensure consistent performance.
Set up Azure Databricks job retry policies to start exponential backoff at 30 seconds, and set the maximum retry count to 3. Set different retry strategies for transient failures versus configuration errors to avoid unnecessary resource consumption.	Intelligent retry mechanisms automatically recover from transient failures like network timeouts or temporary resource unavailability without manual intervention. Exponential backoff prevents overwhelming downstream services during outages. It also distinguishes between recoverable transient problems and permanent configuration problems. This approach reduces operational overhead and improves overall system resilience through automated failure recovery.
Implement virtual network injection for Azure Databricks workspaces to allow custom network routing and private connectivity. Set up network security groups (NSGs) and Azure Firewall rules to control traffic flow and integrate with existing enterprise networking infrastructure.	Virtual network injection provides network-level redundancy through custom routing options and eliminates dependency on default Azure networking paths. Private connectivity allows integration with on-premises networks and other Azure services while maintaining security isolation. This configuration supports multiple availability zones and custom load balancing strategies that improve network reliability.
Activate Unity Catalog with automated metastore backup and cross-region metadata synchronization. Set up external metastore locations in separate storage accounts to ensure metadata persistence during workspace failures.	Unity Catalog backup preserves governance policies and data lineage information during workspace disasters. Cross-region synchronization reduces metadata recovery time from hours to minutes while preserving centralized access control policies across all environments.

Recommendation	Benefit
Deploy Lakeflow Spark Declarative Pipelines for production data pipelines that require automatic quality enforcement and fault tolerance. Set up pipeline restart policies and expectation handling to ensure data quality and maintain processing continuity.	Lakeflow Spark Declarative Pipelines automatically handles transient failures, data quality violations, and infrastructure problems without manual intervention. Built-in quality enforcement prevents corrupted data from propagating downstream, and automatic retry capabilities ensure pipeline completion during temporary resource constraints. This managed approach reduces operational overhead while maintaining data integrity standards.
Create automated workspace backup procedures by using Azure REST APIs or Databricks CLI to export source code, job and pipeline configurations, cluster settings, and workspace metadata. Schedule regular backups to Azure Storage accounts with cross-region replication turned on.	Comprehensive workspace backups allow complete environment restoration during disaster scenarios, which preserves all development work and operational configurations. Automated procedures eliminate human error and ensure backup consistency. Cross-region storage replication protects against regional outages. These approaches reduce RTOs and maintain BCuity for data teams and their analytical workflows.
Implement structured streaming and store checkpoint locations in highly available Storage accounts that have zone-redundant storage (ZRS). Set checkpoint intervals between 10 and 60 seconds based on throughput requirements and failure recovery objectives.	Checkpointing provides exactly-once processing guarantees and allows automatic recovery from cluster failures without data loss or duplicate processing. ZRS ensures checkpoint persistence across availability zone failures to maintain streaming job continuity during infrastructure disruptions.
Activate automatic cluster restart policies for sustained compute workloads. Set appropriate restart timeouts and maximum restart attempts. Turn on cluster termination detection and automatic job rescheduling for mission-critical data processing workflows.	Automatic restart policies ensure workload continuity during planned maintenance events and unexpected cluster failures without requiring manual intervention. Intelligent restart logic distinguishes between recoverable failures and permanent problems to prevent infinite restart loops while maintaining service availability for critical data processing tasks.
Set up instance pools with multiple VM families and sizes within the same compute category to provide allocation flexibility during capacity constraints.	Diverse instance type configurations ensure that cluster provisioning succeeds even when specific VM sizes experience regional capacity limitations. Mixed VM families within pools provide cost optimization opportunities and maintain performance for workload requirements. This approach reduces the risk of provisioning failures during peak demand periods.
Establish chaos engineering practices by deliberately introducing cluster failures, network partitions, and resource constraints in nonproduction environments. Automate	Proactive failure testing validates DR procedures and automatic recovery capabilities before production incidents occur.

Recommendation	Benefit
failure injection by using Azure Chaos Studio to validate recovery procedures and identify resilience gaps.	Systematic chaos engineering identifies weak points in pipeline dependencies, cluster configurations, and monitoring systems that might not be apparent during normal operations. This approach builds confidence in system resilience while ensuring that recovery procedures work as designed during actual outages.

Security

The purpose of the Security pillar is to provide **confidentiality, integrity, and availability** guarantees to the workload.

The [Security design principles](#) provide a high-level design strategy for achieving those goals by applying approaches to the technical design of Azure Databricks.

Workload design checklist

Start your design strategy based on the [design review checklist for Security](#) and identify vulnerabilities and controls to improve the security posture. Extend the strategy to include more approaches as needed.

- ✓ **Review security baselines.** The [Azure Databricks security baseline](#) provides procedural guidance and resources for implementing the security recommendations specified in the Microsoft cloud security benchmark.
- ✓ **Integrate secure development life cycle (SLC).** Implement security code scanning for source code and MLflow model security validation to identify vulnerabilities early in the development life cycle.

Use infrastructure as code (IaC) validation to enforce secure configurations of Azure Databricks resources.

Protect the development environment by implementing secure source code management, managing credentials safely within development workflows, and integrating automated security testing into continuous integration and continuous delivery (CI/CD) pipelines that you use for data processing and machine learning model deployment.

- ✓ **Provide centralized governance.** Add traceability and auditing for data sources through Azure Databricks pipelines. Unity Catalog provides a centralized metadata catalog that supports data discovery and lineage tracking across workspaces with fine-grained access controls and validation.

Unity Catalog can integrate with external data sources.

- ✓ **Introduce intentional resource segmentation.** Enforce segmentation at different scopes by using separate workspaces and subscriptions. Use separate segments for production, development, and sandbox environments to limit the effect of potential breaches.

To apply segmentation, take the following actions:

- Isolate sensitive data workloads in dedicated workspaces that have stricter access controls.
 - Use sandbox environments that have limited privileges and no production data access for exploratory work.
- ✓ **Implement secure network access.** Azure Databricks data plane resources, like Spark clusters and VMs, are deployed into subnets within Azure Virtual Network through virtual network injection. The control plane, which the Databricks platform manages, is isolated from the data plane, which prevents unauthorized access. The control plane communicates securely with the data plane to manage the workload, while all data processing remains within your network.

Virtual network injection gives you control over configuration, routing, and security by using private networking capabilities in Azure. For example, you can use Azure Private Link to secure the connection to the control plane without using the public internet. You can use NSGs to control egress and ingress traffic between subnets and route traffic through Azure Firewall, NAT Gateway, or network virtual appliances for inspection and control. You can also peer the virtual network with your on-premises network, if needed.

- ✓ **Implement authorization and authentication mechanisms.** Consider [identity and access management](#) across both the control and data planes. The Azure Databricks runtime enforces its own security features and access controls while jobs run, which creates a layered security model. Azure Databricks components, like Unity Catalog and Spark clusters, integrate with Microsoft Entra ID, so you can manage access by using Azure role-based access control (Azure RBAC) policies. This integration also provides enterprise authentication through single sign-on (SSO), multifactor authentication, and conditional access policies.

Know where your architecture relies on Databricks-native security and where it intersects with Microsoft Entra ID. This layered approach might require separate identity management and maintenance strategies.

- ✓ **Encrypt data at rest.** Azure Databricks integrates with Azure Key Vault to manage encryption keys. This integration supports customer-managed keys, so you can control the

operation of your encryption keys, including revocation, auditing, and compliance with security policies.

- ✓ **Protect workload secrets.** To run data workflows, you typically need to store secrets like database connection strings, API keys, and other sensitive information. Azure Databricks natively supports secret scopes to store secrets within a workspace that you can securely access from source code and jobs.

Secret scopes integrate with Key Vault, so you can reference secrets and manage them centrally. Enterprise teams typically need Key Vault-backed secret scopes for compliance, security, and policy enforcement.

- ✓ **Implement security monitoring.** Azure Databricks natively supports audit logging, like login attempts, notebook access, and changes to permissions. Use these logs to see admin activities in a workspace. Also, Unity Catalog access logs track who accesses what data, when they access it, and how they access it.

Use Azure Databricks to view logs in Azure Monitor.

The Databricks Security Analysis Tool (SAT) is also compatible with Azure Databricks workspaces.

Recommendations

 Expand table

Recommendation	Benefit
Deploy Azure Databricks workspaces by using virtual network injection to establish network isolation and allow integration with corporate networking infrastructure. Set up custom network security groups, route tables, and subnet delegation to control traffic flow and enforce enterprise security policies.	Virtual network injection eliminates public internet exposure for cluster nodes and provides granular network control through custom routing and firewall rules. Integration with on-premises networks allows secure hybrid connectivity while maintaining compliance with corporate security standards.
Set up Microsoft Entra ID SSO integration with multifactor authentication and conditional access policies for workspace access. Turn on automatic user provisioning and group synchronization to streamline identity management and enforce enterprise authentication standards.	SSO integration eliminates password-related security risks and provides centralized identity management through enterprise authentication systems. Conditional access policies add context-aware security controls that evaluate user location, device compliance, and risk factors before they grant workspace access. This layered approach significantly reduces authentication-related

Recommendation	Benefit
	security vulnerabilities and improves user experience.
Deploy Unity Catalog with centralized metastore configuration to establish unified data governance across all Azure Databricks workspaces. Set up hierarchical permission structures by using catalogs, schemas, and table-level access controls that have regular permission audits.	Unity Catalog provides centralized data governance that eliminates inconsistent access controls and reduces security gaps across multiple workspaces. Fine-grained permissions allow least-privilege access while audit logging supports compliance requirements and security investigations.
Activate customer-managed keys for workspace storage encryption by using Key Vault integration with automatic key rotation policies. Set up separate encryption keys for different environments and implement appropriate access controls for key management operations.	Customer-managed keys provide complete control over encryption key life cycle management and support regulatory compliance requirements for data sovereignty. Key separation across environments reduces security exposure. Automatic rotation policies maintain cryptographic hygiene without operational overhead. This approach helps you meet stringent compliance requirements like FIPS 140-2 Level 3 or Common Criteria standards.
Establish Key Vault-backed secret scopes for centralized credential management with RBAC. Implement secret rotation policies and avoid storing credentials in source code or cluster configurations.	Key Vault integration centralizes secrets management and provides enterprise-grade security controls like access logging and automatic rotation capabilities. This approach eliminates credential exposure in code and configuration files while enabling secure access to external systems and databases.
Create IP access lists that have allow-only policies for trusted corporate networks and deny rules for known threat sources. Set up different access policies for production and development environments based on security requirements.	IP address-based access controls provide an extra security layer that prevents unauthorized access from untrusted networks, which reduces the attack surface. Environment-specific policies enforce appropriate security levels while supporting compliance requirements for network-based access restrictions.
Set up all clusters to use secure cluster connectivity so that public IP addresses can't access them and turn off Secure Shell (SSH) access to cluster nodes. Implement cluster access modes and runtime security features to prevent unauthorized code execution.	Secure cluster connectivity eliminates public internet exposure for compute nodes while preventing direct SSH access that might compromise cluster security. Runtime security features provide extra protection against malicious code execution and lateral movement attacks within the cluster environment.

Recommendation	Benefit
Deploy Private Link endpoints for control plane access to eliminate public internet transit for workspace connectivity. Set up private DNS zones and ensure correct network routing for seamless private connectivity integration.	Private Link eliminates public internet exposure for workspace access and ensures that all management traffic remains within the backbone network in Azure. Private connectivity provides enhanced security for sensitive workloads and supports compliance requirements that mandate private network access. This configuration reduces exposure to internet-based threats while maintaining full workspace functionality.
Activate the enhanced security and compliance settings for regulated environments that require Health Insurance Portability and Accountability Act (HIPAA), Payment Card Industry Data Security Standard (PCI DSS), or Systems and Organization Controls 2 (SOC 2) compliance. Set up automatic security updates and turn on compliance security profiles for specific regulatory frameworks.	Enhanced security and compliance features provide specialized security controls, including compliance security profiles, automatic security updates, and enhanced monitoring capabilities. This managed approach ensures continuous compliance with regulatory requirements while reducing operational overhead for security management. Automatic updates maintain security posture without disrupting business operations or requiring manual intervention.
Turn on audit logging by using Unity Catalog system tables and workspace audit logs with automated analysis and alerting. Set up log retention policies and integrate with Security Information and Event Management (SIEM) systems for centralized security monitoring and incident response.	Audit logging provides visibility into user activities, data access patterns, and system changes for security monitoring and compliance reporting. Integration with SIEM systems allows automated threat detection and rapid incident response capabilities through centralized log analysis.
Set up OAuth 2.0 machine-to-machine authentication for API access and automated workloads instead of personal access tokens (PATs). Implement appropriate token scoping and life cycle management to ensure secure programmatic access.	OAuth authentication provides enhanced security through fine-grained permission scoping and improved token life cycle management compared to PATs. This approach allows secure automation while maintaining appropriate access controls and audit trails for programmatic workspace interactions.
Implement workspace isolation strategies by deploying separate workspaces for different environments and establishing network segmentation controls. Set up environment-specific access policies and data boundaries to prevent cross-environment data access.	Workspace isolation prevents data leakage between environments and supports compliance requirements for data segregation and access controls. This architecture reduces the effects of security incidents and enforces environment-specific security policies that match risk profiles.

Recommendation	Benefit
Deploy the SAT for continuous security configuration assessments that provide automated remediation recommendations. Schedule regular security scans and integrate the scans' discoveries into CI/CD pipelines for proactive security management.	Automated security assessment provides continuous monitoring of workspace configurations against security best practices and compliance requirements. Integration with development workflows applies shift-left security practices that identify and address misconfigurations before they reach production environments. This proactive approach reduces security risks while minimizing remediation costs and operational disruption.
Set up service principal authentication for automated workflows and CI/CD pipelines that have minimal required permissions. Implement credential management through Key Vault and turn on certificate-based authentication for enhanced security.	Service principal authentication eliminates dependencies on user credentials for automated processes while providing appropriate access controls and audit trails. Certificate-based authentication enhances security compared to client secrets while supporting appropriate credential life cycle management for production automation scenarios.
Establish network egress controls through virtual network injection by using custom route tables and network security groups to monitor and restrict data transfer. Set up Azure Firewall or network virtual appliances to inspect and control outbound traffic patterns.	Network egress controls prevent unauthorized data exfiltration while providing visibility into data movement patterns through traffic monitoring and analysis. Custom routing and firewall inspection detect unusual data transfer activities that indicate security breaches or insider threats.
Activate Microsoft Entra ID credential passthrough for Azure Data Lake Storage access to eliminate service principal dependencies. Set up user-specific access controls and ensure appropriate permission inheritance from Unity Catalog governance policies.	Credential passthrough simplifies service principal management for data access and integrates with enterprise identity systems. User-specific access controls ensure that data access permissions align with organizational policies and job functions. This approach simplifies credential management while maintaining strong security controls and audit capabilities for data lake tasks.
Implement cluster hardening practices, including SSH restriction, custom image scanning, and runtime security controls. Use approved base images and prevent unauthorized software installation by using cluster policies and init scripts validation.	Cluster hardening uses SSH restrictions to reduce attack surfaces and prevents unauthorized software installation that might compromise cluster security. Custom image scanning ensures that base images meet security standards, and runtime controls block malicious code and lateral movement within the cluster environment.

Recommendation	Benefit
Implement automated security scanning for source code and code artifacts through CI/CD pipeline integration with static analysis tools and vulnerability scanners.	Automated security scanning helps you detect security vulnerabilities in analytical code and infrastructure configurations before they reach production environments.

Cost Optimization

Cost Optimization focuses on **detecting spend patterns, prioritizing investments in critical areas, and optimizing in others** to meet the organization's budget while meeting business requirements.

The [Cost Optimization design principles](#) provide a high-level design strategy for achieving those goals and making trade-offs as necessary in the technical design related to Azure Databricks and its environment.

Workload design checklist

Start your design strategy based on the [design review checklist for Cost Optimization](#). Fine-tune the design so that the workload aligns with the budget that's allocated for the workload. Your design should use the right Azure capabilities, monitor investments, and find opportunities to optimize over time. Define policies and procedures to continuously monitor and optimize costs while meeting your performance requirements.

- ✓ **Determine your cost drivers.** Theoretical capacity planning often leads to overprovisioning and wasted spend. Not investing in enough resources is also risky.

Estimate costs and seek optimization opportunities based on workload behavior. Run pilot workloads, benchmark cluster performance, and analyze automatic scaling behavior. Real usage data can help you rightsize the cluster, set scaling rules, and allocate the right resources.

- ✓ **Set clear accountability for spend.** When you use multiple Azure Databricks workspaces, track which teams or projects are responsible for specific costs. This task requires tagging resources, like clusters or jobs, with project or cost center information, using chargeback models to assign usage-based costs to teams, and setting budget controls to monitor and limit spending.
- ✓ **Choose the appropriate tiers.** We recommend that you use the Standard tier for development and basic production workloads. Use the Premium tier for production workloads because it provides security features, like the Unity Catalog, that are essential for most analytics workloads.

- ✓ **Choose between serverless compute versus VMs.** Serverless compute uses consumption-based pricing, so you only pay for what you use. We recommend that you use serverless compute for workloads that have activity spikes or on-demand jobs because it scales automatically and reduces operational overhead. You don't need to manage infrastructure or pay for idle time.

For predictable or steady usage, choose VM-based clusters. This approach gives you more control but requires operational management and tuning to avoid overprovisioning. If you're sure about long-term usage, use reserved capacity. Databricks Commit Units (DBCUs) are prepaid usage contracts that give discounts in exchange for usage commitments.

Make sure that you analyze historical trends and project future demands to make the best choice.

- ✓ **Optimize cluster utilization.** Reduce Azure Databricks costs by automatically scaling and shutting down clusters when you don't need them.

Evaluate whether your budget allows for cluster pools. Cluster pools can reduce cluster start times, but they're idle resources that accrue infrastructure costs while you're not using them.

Save costs in development and test environments by using scaled down configurations. Encourage cluster sharing among teams to avoid using unnecessary resources. Enforce automatic termination policies to deprovision idle clusters.

- ✓ **Optimize compute for each workload.** Different workloads require different compute configurations. Some jobs might need higher memory and processing power, while other workloads might run lightweight jobs that accrue lower costs.

Instead of using the same large cluster for every job, assign the right cluster to each job. Use Azure Databricks to tailor compute resources to match each workload. This approach helps you reduce costs and improve performance.

- ✓ **Optimize storage costs.** Storing large volumes of data can get expensive. Reduce costs by using Delta Lake capabilities. For example, use data compaction to merge many small files into fewer large files to reduce storage overhead and speed up queries.

Diligently manage old data. You can use retention policies to remove outdated versions. You can also move old, infrequently accessed data to cheaper storage tiers. If applicable, automated life cycle policies, like time-based deletion or tiering rules, help archive or delete data when it becomes less useful.

Different storage formats and compression settings can also reduce the amount of space that you use.

- ✓ **Optimize data processing techniques.** Compute, networking, and querying when you process large volumes of data accrue costs. To reduce these costs, use a combination of strategies for query tuning, data format selection, and Delta Lake and code optimizations.
 - *Minimize data movement.* Evaluate the data processing pipeline to reduce unnecessary data movement and bandwidth costs. Implement incremental processing to avoid reprocessing unchanged data, and use caching to store frequently accessed data closer to compute resources. Reduce overhead when connectors access or integrate with external data sources.
 - *Use efficient file formats.* Formats like Parquet and compression algorithms like Zstandard that are native to Databricks lead to faster read times and lower data costs because less data needs to move.
 - *Make your queries efficient.* Avoid full-table scans to reduce compute costs. Instead, partition your Delta tables based on common filter columns. Use native features to reduce compute time. For example, native Spark features like Catalyst optimizer and adaptive query execution (AQE) dynamically optimize joins and partitioning at runtime. Databricks Photon engine runs queries faster.
 - *Apply code optimization design patterns.* Use patterns like Competing Consumers, Queue-Based Load Leveling, and Compute Resource Consolidation within Azure Databricks environments.
- ✓ **Monitor consumption.** Databricks Unit (DBU) is an abstracted billing model based on compute usage. Azure Databricks gives you detailed information that provides visibility into usage metrics about clusters, runtime hours, and other components. Use that data for budget planning and cost control.
- ✓ **Implement automated spending guardrails.** To avoid overspending and ensure efficient use of resources, enforce policies that regulate resource usage. For example, have checks on the types of clusters that can be created, and limit the cluster size or its lifetime. Set alerts to notify you when resource usage approaches the allowed budget boundaries. For example, if a job suddenly starts to consume a specific number of DBUs, a script can alert the admin or shut down the job.

Take advantage of Databricks system tables to track cluster usage and DBU consumption. You can query the table to detect cost anomalies.

Recommendations

Recommendation	Benefit
Deploy job clusters for scheduled workloads instead of all-purpose clusters to eliminate idle compute costs. Set jobs to automatically end when they finish.	Job clusters reduce costs by automatically ending jobs when they finish and optimizing DBU consumption by precisely matching compute time to actual processing requirements.
Turn on cluster autoscaling and set minimum and maximum node limits based on workload analysis to handle baseline loads and peak demand requirements. Set up scaling policies to respond quickly to workload changes and avoid unnecessary scaling oscillations that can increase costs unnecessarily.	Automatic scaling further reduces overprovisioning costs compared to fixed-size clusters. It maintains performance levels during peak periods and automatically reduces resources during low-demand periods.
Set up automatic termination for all interactive clusters with appropriate timeout periods based on usage patterns. These periods are typically 30 to 60 minutes for development environments.	Automatic termination reduces interactive cluster costs without affecting user productivity. This approach eliminates costs from clusters that run overnight or on weekends.
Adopt serverless SQL warehouses for interactive SQL workloads to eliminate infrastructure management overhead and optimize costs through consumption-based billing. Set up appropriate sizing based on concurrency requirements and turn on autostop functionality to minimize costs during inactive periods. Migrate from classic SQL endpoints to serverless SQL warehouses for better performance and cost efficiency. Use built-in Photon acceleration capabilities.	Serverless SQL warehouses further reduce SQL workload expenses compared to always-on clusters by applying usage-based billing that eliminates idle time costs. Built-in Photon acceleration improves performance while providing predictable costs for each query for interactive analytics scenarios.
Implement cluster pools for frequently used configurations to reduce startup times and optimize resource allocation based on usage patterns and demand forecasting.	Cluster pools reduce startup time from minutes to seconds while eliminating DBU charges for idle pool instances.
Use Delta Lake optimization features, including <code>OPTIMIZE</code> commands, <code>Z-ORDER</code> clustering, and <code>VACUUM</code> operations, to reduce storage costs and improve query performance. Schedule regular optimization jobs to compact small files, implement data retention policies,	Delta Lake optimization reduces storage costs through data compaction and efficient compression. It improves query performance by reducing file scan requirements.

Recommendation	Benefit
and set compression settings based on data access patterns.	
Implement compute policies to enforce cost-effective configurations across all workspaces and teams by restricting instance types and enforcing automatic termination settings. Create different policy templates for development, staging, and production environments that have different levels of restrictions and appropriate tags for cost attribution.	Compute policies reduce average cluster costs by preventing overprovisioning and ensuring adherence to cost optimization standards while maintaining governance.
Monitor costs by using Databricks system tables and Microsoft Cost Management integration to see DBU consumption patterns and spending trends. Implement automated cost reporting dashboards that track usage by workspace, user, job, and cluster type. Set up cost alerts for proactive management. Use Unity Catalog system tables to analyze detailed usage patterns and create chargeback models for different teams and projects based on actual resource consumption.	Use comprehensive cost monitoring to see DBU consumption patterns and accurately attribute costs through detailed usage analytics and tagging strategies. Integrate with Cost Management to use organization-wide cost governance and establish responsible resource usage patterns across teams.
Purchase Databricks reserved capacity through Databricks Commit Units (DBCUs) for predictable workloads that have stable usage patterns and optimal commitment terms.	Reserved capacity achieves more cost savings through DBCU compared to pay-as-you-go pricing while providing cost predictability over one to three-year terms for stable production workloads.
Optimize workload-specific compute configurations by selecting appropriate compute types for different use cases, like job clusters for extract, transform, load (ETL) pipelines and graphics processing unit (GPU) instances for machine learning training. Match instance types and cluster configurations to specific workload requirements rather than using generic configurations across all scenarios.	Workload-specific optimization further reduces costs compared to one-size-fits-all approaches by eliminating overprovisioning and using specialized compute types optimized for specific use cases.

Recommendation	Benefit
Implement automated data life cycle policies with scheduled cleanup operations, including VACUUM commands, log file retention, and checkpoint management, based on business requirements.	Automated life cycle management reduces storage costs by systematically removing unnecessary data versions, logs, and temporary files and preventing storage bloat over time.
Use the Standard tier for development and testing environments. Use the Premium tier only for production workloads that require advanced security features and compliance certifications.	Strategic tier selection optimizes licensing costs by using the Standard tier for nonproduction workloads where advanced security features aren't required. Premium tier features like RBAC and audit logging are applied only where business requirements and security policies justify the extra cost investment.
Implement serverless jobs for variable and intermittent workloads that have unpredictable scheduling patterns or resource requirements for unplanned analytics and experimental workloads. Set up serverless compute for batch processing jobs where usage patterns are difficult to predict and use automatic optimization capabilities. Migrate appropriate workloads from traditional clusters to serverless compute based on usage analysis and cost-benefit evaluation to optimize resource utilization.	Serverless jobs eliminate idle time costs and provide automatic optimization for variable resource requirements, which reduces costs for unpredictable workloads. The consumption-based billing model ensures that you pay only for compute time that you use, which makes it ideal for development environments and sporadic production workloads that need automatic resource optimization.
Set up cost alerts and budgets by using Cost Management and Databricks usage monitoring to proactively manage costs with multiple alert thresholds. Set up escalation procedures for different stakeholder groups and implement automated responses for critical cost overruns. Review budgets regularly.	Proactive cost monitoring helps you detect cost anomalies and budget overruns early so that you can prevent surprise expenses and act before costs significantly affect budgets.
Optimize data formats and turn on Photon acceleration to reduce compute time through efficient data processing with columnar storage formats and compression algorithms. Implement partitioning strategies that minimize data scanning requirements and use Photon acceleration for supported workloads to take advantage of vectorized query execution.	Data format optimization and Photon acceleration reduce compute time and costs through columnar storage optimizations and vectorized query execution capabilities. These optimizations compound over time as data volumes grow, providing increasing cost benefits for analytical workloads and complex data processing pipelines without requiring architectural changes.

Operational Excellence

Operational Excellence primarily focuses on procedures for **development practices, observability, and release management**.

The [Operational Excellence design principles](#) provide a high-level design strategy for achieving those goals for the operational requirements of the workload.

Workload design checklist

Start your design strategy based on the [design review checklist for Operational Excellence](#) for defining processes for observability, testing, and deployment related to Azure Databricks.

- ✓ **Collect monitoring data.** For your Azure Databricks workload, focus on tracking key areas like cluster health, resource usage, jobs and pipelines, data quality, and access activity. Use these metrics to confirm that the system behaves as expected. You can also use them to audit how data and resources are accessed and used and to enforce governance.
 - *Monitor the cluster.* When you monitor Azure Databricks clusters, focus on indicators that reflect performance and efficiency. Track overall cluster health and observe how nodes use resources like central processing unit (CPU), memory, and disks.
 - *Monitor jobs and pipelines.* Capture metrics that show how jobs flow when they run. These metrics include job success and failure rates and run durations. Gather information about how jobs are triggered to learn why they run.

Use Databricks System tables to capture job status, dependency chains, and throughput natively.

- *Monitor data source connectivity.* Monitor integrations and dependencies with external systems. This data includes source connectivity status, API dependencies, and service principal authentication behavior. You can use Unity Catalog to manage and monitor external locations. This approach helps you identify potential access or configuration problems.
- *Monitor data quality.* Collect signals that validate both the integrity and freshness of your data. Monitor for schema evolution problems by using tools like Auto Loader. Implement rules that do completeness checks, null value detection, and anomaly identification. You can use Lakeflow Spark Declarative Pipelines to enforce built-in quality constraints during data processing.

Capturing data lineage through Unity Catalog helps you trace how data flows and transforms across systems, which gives your pipelines greater transparency and

accountability.

Built-in monitoring tools in Azure Databricks integrate with Azure Monitor.

- ✓ **Set up automated and repeatable deployment assets.** Use IaC to define and manage Azure Databricks resources.

Automate provisioning of workspaces, including region selection, networking, and access control, to ensure consistency across environments. Use cluster templates to standardize compute configurations. This approach reduces the risk of misconfiguration and improves cost predictability. Define jobs and pipelines as code by using formats like JSON Azure Resource Manager templates (ARM templates) so that they're version-controlled and reproducible.

Use branching strategies and rollback procedures in [Databricks Asset Bundles](#) to version control notebook source code, job configurations, pipeline definitions, and infrastructure settings in Git repositories.

- ✓ **Automate deployments.** Use CI/CD pipelines in Azure Databricks to automate the deployment of pipelines, job configurations, cluster settings, and Unity Catalog assets. Instead of manually pushing changes, consider tools like Databricks Repos for version control, Azure DevOps or GitHub Actions for pipeline automation, and Databricks Asset Bundles for packaging code and configurations.
- ✓ **Automate routine tasks.** Commonly automated tasks include managing cluster life cycles, like scheduled starts and stops, cleaning up logs, and validating pipeline health. By integrating with Azure tools like Azure Logic Apps or Azure Functions, teams can build self-healing workflows that automatically respond to problems, like restarting failed jobs or scaling clusters. This type of automation helps maintain reliable, efficient Azure Databricks operations when workloads grow.
- ✓ **Have strong testing practices.** Azure Databricks-specific strategies include unit testing for notebook code, integration testing for data pipelines, validation of Lakeflow Spark Declarative Pipelines logic, permission testing with Unity Catalog, and verifying infrastructure deployments. These practices help catch problems early and reduce incidents in production.
- ✓ **Develop operational runbooks to handle incidents.** [Operational runbooks](#) provide structured, step-by-step guidance for handling common Azure Databricks scenarios. These runbooks include diagnostic commands, log locations, escalation contacts, and recovery procedures with estimated resolution times. Use runbooks to quickly and consistently respond to incidents across teams.

- ✓ **Develop backup and recovery procedures.** [Backup and recovery procedures](#) ensure BCuity through protection of workspace configurations, analytics source code, job definitions, and data assets. Backup and recovery procedures include automated backup schedules and cross-region replication that meet RTOs and recovery point objectives (RPOs).
- ✓ **Implement team collaboration and knowledge management.** Team collaboration practices optimize Azure Databricks productivity through shared workspace organization, notebook collaboration features, and documentation standards that facilitate knowledge transfer and reduce project duplication across development teams.

Recommendations

 Expand table

Recommendation	Benefit
Set up diagnostic settings for Azure Databricks workspaces to send platform logs, audit logs, and cluster events to an Azure Monitor Log Analytics workspace. Turn on all available log categories, including workspace, clusters, accounts, jobs, notebook, and Unity Catalog audit logs, for observability coverage.	Centralizes all Azure Databricks telemetry in Log Analytics and turns on advanced KQL queries for troubleshooting, automated alerting on critical events, and compliance reporting. Diagnostics provide unified visibility across workspace activities, cluster performance, and data access patterns for proactive operational management.
Deploy Azure Databricks workspaces by using ARM templates or Bicep files that have parameterized configurations for consistent environment provisioning. Include workspace settings, network configurations, Unity Catalog enablement, and security policies in the template definitions to ensure standardized deployments across development, testing, and production environments.	Eliminates configuration drift between environments and reduces deployment errors through consistent, version-controlled infrastructure definitions. Further accelerates environment provisioning compared to manual deployment processes and allows rapid recovery through automated workspace re-creation during disaster scenarios.
Integrate Azure Databricks notebooks and other source code with Git repositories by using Databricks Repos for source control and collaborative development. Set up automated CI/CD pipelines through Azure DevOps or GitHub Actions to deploy source code changes, job and pipeline	Allows collaborative development with version history, branch-based workflows, and merge conflict resolution for code. Reduces deployment risks through automated testing and staged releases while maintaining complete audit trails of all production changes.

Recommendation	Benefit
configurations, and cluster templates across environments with appropriate testing and approval workflows.	
Deploy automated cluster rightsizing solutions by using Azure Databricks cluster metrics and Azure Monitor data to analyze utilization patterns and recommend optimal instance types and sizes.	Optimizes infrastructure costs by automatically matching cluster resources to actual workload requirements. Maintains performance service-level objectives (SLAs) and reduces compute costs through intelligent resource allocation and automated scaling decisions.
Set up automatic scaling policies based on CPU, memory, and job queue metrics to automatically adjust cluster capacity according to workload demands.	Eliminates manual monitoring overhead and allows proactive capacity management through data-driven insights about resource usage patterns and optimization opportunities.
Activate Unity Catalog audit logging to track all data access tasks, permission changes, and governance activities within Azure Databricks workspaces. Set up log retention policies and integrate with Microsoft Sentinel or partner SIEM solutions for automated security monitoring and compliance reporting.	Provides complete audit trails for data access patterns, permission modifications, and governance tasks required for regulatory compliance frameworks. Allows automated threat detection and investigation of suspicious data access behaviors through centralized security monitoring.
Implement Lakeflow Spark Declarative Pipelines that have data quality expectations and monitoring rules to automate data validation and pipeline quality assurance. Set up expectation thresholds, quarantine policies, and automated alerting for data quality violations to maintain pipeline reliability and data integrity.	Automates data quality validation by using declarative rules that prevent bad data from propagating downstream, which reduces manual validation efforts. Provides transparent data quality metrics and automated remediation workflows that maintain pipeline reliability and business confidence in data accuracy.
Establish automated backup procedures for Azure Databricks workspace artifacts by using the Databricks REST API and Azure Automation runbooks. Schedule regular backups of analytics source content, job definitions, cluster configurations, and workspace settings with versioned storage in Storage accounts and cross-region replication.	Ensures rapid recovery from accidental deletions, configuration changes, or workspace corruption by using automated restoration capabilities. Maintains BCuity through versioned backups and reduces RTOs from days to hours through standardized backup and restore procedures.
Create standardized workspace folder hierarchies by using naming conventions that include project codes, environment indicators,	Improves project discoverability and reduces onboarding time for new team members through consistent workspace organization. Accelerates

Recommendation	Benefit
and team ownership. Implement shared folders for common libraries, templates, and documentation with appropriate access controls to facilitate knowledge sharing and collaboration.	development through shared code libraries and standardized project structures that eliminate duplication of effort across teams.
Set up Cost Management with resource tagging strategies for Azure Databricks workspaces, clusters, and compute resources. Implement cost alerts, budget thresholds, and automated reporting to track spending across projects, teams, and environments with chargeback capabilities and optimization recommendations.	Provides granular cost visibility and accountability across organizational units through detailed spend analysis and automated budget monitoring. Proactively optimize costs through spending alerts and usage pattern insights that prevent budget overruns and identify optimization opportunities. Supports accurate cost allocation and chargeback processes with detailed resource utilization reporting and automated cost center assignment based on resource tags.
Set up service principal authentication for Azure Databricks integrations with external systems, data sources, and Azure services. Implement managed identity where possible and establish credential rotation policies with Key Vault integration for secure, automated authentication management.	Eliminates shared credential security risks and allows automated authentication without manual intervention. Provides centralized credential management with audit trails and supports fine-grained access control policies that align with least-privilege security principles.
Establish cluster life cycle policies that have automated termination schedules, idle timeout configurations, and resource usage limits to enforce organizational governance standards. Set up policy-based cluster creation restrictions, instance type limitations, and maximum runtime controls to prevent resource waste and ensure compliance.	Reduces compute costs through automated cluster life cycle management and prevents resource waste from idle or forgotten clusters. Enforces organizational policies consistently across all users and teams while maintaining operational flexibility for legitimate use cases.
Deploy Azure Monitor alert rules for critical Azure Databricks tasks, including cluster failures, job execution errors, workspace capacity limits, and Unity Catalog access violations. Set up automated notification workflows that have escalation procedures and integrate with	Helps you proactively respond to incidents by notifying you of critical problems before they affect business operations. Reduces mean time to detection (MTTD) from hours to minutes and supports automated escalation procedures that notify the right team members based on severity levels.

Recommendation	Benefit
incident management systems like ServiceNow or Jira.	
Implement environment-specific workspace configurations with RBAC policies that enforce separation between development, testing, and production environments.	Prevents unauthorized access to production data and reduces risk of accidental changes in critical environments through enforced security boundaries.
Set up Unity Catalog governance rules, network security groups, and data access permissions that meet each environment's security and compliance requirements.	Maintains regulatory compliance by ensuring that development activities can't affect production systems and data integrity is preserved across environment boundaries.

Performance Efficiency

Performance Efficiency is about **maintaining user experience even when there's an increase in load** by managing capacity. The strategy includes scaling resources, identifying and optimizing potential bottlenecks, and optimizing for peak performance.

The [Performance Efficiency design principles](#) provide a high-level design strategy for achieving those capacity goals against the expected usage.

Workload design checklist

Start your design strategy based on the [design review checklist for Performance Efficiency](#). Define a baseline that's based on key performance indicators for Azure Databricks.

- ✓ **Plan capacity.** Analyze workloads and monitor resource usage to determine how much compute and storage your workloads actually need. Use that information to rightsize clusters, optimize job schedules, and forecast storage growth. This approach helps you avoid underprovisioning, which leads to resource constraints.
- ✓ **Choose optimal compute configurations for workload characteristics.** Evaluate serverless options, which can provide better automatic scaling and faster startup times. Compare them with traditional clusters to choose the best fit.

For clusters, optimize configurations, including instance types, sizes, and scaling settings, based on data volume and processing patterns. Be sure to analyze trade-offs between instance families for specific use cases. For example, evaluate memory-optimized versus compute-optimized instances and local solid-state drives (SSDs) versus standard storage options to match performance requirements.

Spark clusters can run different types of workloads, which require their unique performance tuning. In general, you need to run jobs faster and avoid compute bottlenecks. Fine-tune settings like executor memory, parallelism, and garbage collection to achieve these goals.

For more information about how to choose the right services for your workload, see [Architecture strategies for selecting the right services](#).

- ✓ **Prioritize resource allocation for critical workloads.** Separate and prioritize workloads that run at the same time. Use features like resource pools, cluster pools, isolation modes, and job queues to avoid interference between jobs. Set resource quotas and scheduling rules to help ensure that background or lower-priority processes don't slow down high-priority tasks.
- ✓ **Set up automatic scaling for variable workloads.** Set up automatic scaling policies in Azure Databricks by defining scaling triggers that cause the cluster to scale, determine how quickly it adds or removes nodes, and set resource limits. These settings help Azure Databricks respond efficiently to changing workloads, optimize resource usage, and avoid performance problems during scaling events.
- ✓ **Design efficient data storage and retrieval mechanisms.** Performance improvements for data-intensive tasks require careful planning and tuning.
 - *Organize data strategically.* Design data partitioning schemes that optimize query performance when you organize Delta Lake tables. Good partitioning allows Spark to prune partitions so that it reads only the relevant subsets of data during a query instead of scanning the entire table.

File sizing plays a key role. Files that are too small create excessive metadata overhead and slow down Spark jobs, while files that are too large can cause memory and performance problems.

Align your data layout with how users or jobs typically query the data. Otherwise, full-table scans might degrade performance.

- *Implement effective caching.* Use caching for hot datasets and monitor cache hit ratios to ensure that you aren't unnecessarily using memory. Spark provides built-in caching mechanisms and Azure Databricks provides Delta Cache, which further optimizes by caching data at the disk level across nodes.
- *Write efficient queries.* Avoid unnecessary data scans, excessive shuffling, and long run times, which all contribute to inefficient performance.

Use indexing, predicate pushdown, projection pushdown, and join optimization techniques that use query plan analysis to run jobs more efficiently to optimize SQL queries and Spark operations.

Azure Databricks provides built-in optimizations. The Catalyst optimizer rewrites queries for efficiency. AQE adjusts plans at runtime to handle data skew and improve joins. Delta Lake features like table statistics, Z-order clustering, and Bloom filters further reduce scanned data for faster, more cost-effective queries.

- *Choose the right data formats and compression.* Formats like Parquet and smart compression algorithms like Zstandard (zstd) reduce storage and speed up reads without compromising performance.

- ✓ **Optimize network and input/output (I/O) performance.** Choose high-performance storage options like Premium or SSD-backed storage and design your architecture to minimize data movement by processing data close to where it's stored.

Also use efficient data transfer strategies, like batching writes and avoiding unnecessary shuffles, to maximize throughput and reduce latency.

- ✓ **Optimize job execution based on the type of workload.** Tailor optimization strategies to specific needs.

- *Stream processing:* Real-time data pipelines require low-latency and high-throughput performance. In Azure Databricks, you must tune parameters like trigger intervals, micro-batch sizes, watermarking, and checkpointing to meet these requirements. Use Structured Streaming and Delta Lake capabilities like schema evolution and exactly-once delivery to ensure consistent processing under different loads.
- *Machine learning:* Machine learning training and inference jobs are typically compute-intensive. You can boost performance by using distributed training, GPU acceleration, and efficient feature engineering pipelines. Azure Databricks supports machine learning performance tuning through MLflow, Databricks Runtime for machine learning, and integrations with tools like Horovod. Tune resource configurations and apply data preprocessing optimizations to significantly reduce training time and inference latency.

Use Lakeflow Spark Declarative Pipelines to simplify and automate the implementation of these optimization recommendations.

- ✓ **Use your monitoring system to identify performance bottlenecks.** Implement [comprehensive performance monitoring](#) to learn how jobs, clusters, and queries behave so that you can identify bottlenecks or inefficiencies that increase costs and slow down workloads.

- Analyze anomalies in key metrics like CPU and memory usage, job run times, query latencies, and cluster health. This information helps you pinpoint slowdowns that poor Spark configurations, unoptimized queries, or underprovisioned and overprovisioned clusters can cause.
- Use built-in tools like the Spark UI to analyze query plans and job stages, Azure Monitor to track infrastructure-level metrics, and custom metrics or logs for deeper insights. These tools support proactive tuning so that you can fix problems before they affect users or critical pipelines.
- ✔ **Conduct systematic performance testing.** Use load testing, stress testing, and benchmarking to validate run times, resource usage, and system responsiveness. Establish performance baselines and incorporate automated tests into your CI/CD pipelines to detect slowdowns early and measure the effects of optimizations.

Recommendations

 Expand table

Recommendation	Benefit
Set up Azure Databricks clusters to use memory-optimized instance types like E-series or M-series VMs when you process large datasets that require extensive in-memory caching, machine learning model training, or complex analytical tasks. Evaluate memory requirements based on dataset size and processing patterns, and then select VM sizes that provide sufficient memory capacity and high memory-to-CPU ratios for optimal performance.	Eliminates memory bottlenecks that can cause job failures or severe performance degradation. Helps memory-intensive operations like large-scale machine learning training and complex analytics workloads run smoothly.
Set up cluster autoscaling policies to use appropriate minimum and maximum node limits based on workload patterns and performance requirements. Set minimum nodes to handle baseline workloads efficiently while establishing maximum limits to prevent runaway costs. Define scaling triggers based on CPU utilization, memory usage, or job queue depth, and configure scaling velocity to balance responsiveness with cost optimization.	Maintains consistent performance during demand fluctuations while optimizing costs through automatic resource adjustment that scales up during peak periods and scales down during low utilization periods.
Run OPTIMIZE commands with Z-ordering on Delta Lake tables to improve data clustering and query performance. Choose Z-order columns based on frequently used filter and join conditions in your queries. These conditions typically include columns used in <code>WHERE</code> clauses, <code>GROUP BY</code> operations, and <code>JOIN</code> predicates.	Reduces query run time through improved data skipping and minimized I/O operations, while also decreasing storage costs through better compression ratios achieved by clustering related data together.

Recommendation	Benefit
Schedule regular optimization tasks by using Azure Databricks jobs or Lakeflow Spark Declarative Pipelines to maintain optimal performance as data grows.	Provides cumulative performance improvements as optimization benefits compound over time with regular maintenance and intelligent data organization that aligns with actual query patterns.
Turn on Delta Cache for cluster configurations where you frequently access the same datasets across multiple queries or jobs. Edit cache settings to use local nonvolatile memory express (NVMe) SSD storage effectively to allocate adequate cache size based on your dataset characteristics and access patterns. Monitor cache hit ratios and adjust cache configurations to maximize performance benefits for your specific workloads.	Accelerates query performance for frequently accessed data through intelligent SSD-based caching that bypasses slower network storage. This approach significantly reduces latency for iterative analytics and machine learning workloads.
Turn on Photon engine for cluster configurations and SQL warehouses to accelerate SQL queries and DataFrame operations through vectorized execution. Photon provides the most significant benefits for analytical workloads that have aggregations, joins, and complex SQL tasks. Set up Photon-enabled compute resources for data engineering pipelines, business intelligence (BI) workloads, and analytical applications that process large datasets.	Improves performance for SQL and DataFrame operations through native vectorized execution. Reduces compute costs by improving processing efficiency and reducing run time. Allows you to process larger datasets within the same time constraints and supports more concurrent users without degrading performance by improving overall system throughput.
Set Spark executor memory settings between 2 and 8 gigabytes (GB) for each executor and driver memory based on your largest dataset size and processing complexity. Set <code>spark.executor.cores</code> to two to five cores for each executor to balance parallelism and resource efficiency. Adjust these settings based on your specific workload characteristics, data volume, and cluster size to prevent out-of-memory errors while maximizing resource utilization.	Prevents job failures that memory problems cause and optimizes resource allocation efficiency to reduce run time and resource waste.
Set up Storage accounts with Premium SSD performance tiers for Azure Databricks workloads that require high input/output operations per second (IOPS) and low latency. Use Premium block blob storage for data lake scenarios that have intensive read and write tasks.	Provides up to 20,000 IOPS and submillisecond latency for storage operations. Improves performance for data-intensive workloads and reduces job run times by eliminating storage I/O bottlenecks.

Recommendation	Benefit
Ensure that storage accounts are in the same region as your Azure Databricks workspace to minimize network latency.	
Design data partitioning strategies based on commonly used filter columns in your queries. These filters include date columns for time-series data or categorical columns for dimensional data. Avoid overpartitioning by limiting partitions to fewer than 10,000 and ensuring that each partition has at least 1 GB of data. Use partition pruning-friendly query patterns and consider liquid clustering for tables that have multiple partition candidates.	Reduces data scanning through effective partition pruning. Improves query performance and reduces compute costs by processing only relevant data partitions. Queries behave predictably and scale linearly with filtered data size rather than total table size. Maintains consistent response times as datasets grow to petabyte (PB) scale.
Use Parquet file format with zstd or Snappy compression for analytical workloads to optimize storage efficiency and query performance. Zstd provides better compression ratios for cold data. Snappy provides faster decompression for frequently accessed datasets. Set appropriate compression levels and evaluate compression trade-offs based on your access patterns and storage costs.	Reduces storage costs while improving query performance through columnar storage efficiency and optimized compression. Scans data faster and reduces network I/O.
Deploy serverless SQL warehouses for BI and analytical workloads that require unplanned querying and interactive analytics. Set up appropriate warehouse sizes, like 2X-Small to 4X-Large, based on concurrency requirements and query complexity. Turn on autostop and autoresume features to optimize costs while ensuring rapid query responsiveness for users.	Eliminates cluster management overhead while providing instant scaling and Photon-accelerated performance. Delivers better price performance compared to traditional clusters for SQL workloads. Provides consistent subsecond query startup times and automatic optimization that adapts to changing workload patterns without manual intervention or configuration tuning.
Turn on AQE in Spark configurations to use runtime optimization capabilities, including dynamic coalescing of shuffle partitions, dynamic join strategy switching, and optimization of skewed joins. Set AQE parameters like target shuffle partition size and coalescing thresholds based on your typical data volumes and cluster characteristics.	Improves query performance through intelligent runtime optimizations that adapt to actual data characteristics and run patterns. Automatically addresses common performance problems like small files and data skew.
Create cluster pools that have prewarmed instances that match your most common cluster configurations to reduce startup times for interactive clusters and job clusters.	Reduces cluster startup time from 5 to 10 minutes to less than 30 seconds. Improves developer productivity and

Recommendation	Benefit
Set pool sizes based on expected concurrent usage patterns and maintain idle instances during peak hours to ensure immediate availability for development teams and scheduled jobs.	runs jobs faster for time-sensitive data processing workflows.
Schedule regular optimization tasks by using Azure Databricks jobs to compact small files and improve query performance. Run <code>VACUUM</code> commands to clean up expired transaction logs and deleted files. Set optimization frequency based on data ingestion patterns, typically daily for high-volume tables and weekly for less frequently updated tables.	Maintains consistent query performance as data volumes grow by preventing file proliferation and data fragmentation. Reduces storage costs by cleaning up unnecessary files and improving compression ratios.
Monitor table statistics and file counts to determine optimal maintenance schedules.	Prevents performance degradation over time that commonly occurs in data lakes without appropriate maintenance. Ensures predictable query response times and optimal resource utilization.
Configure structured streaming trigger intervals based on latency requirements and data arrival patterns. Use continuous triggers for subsecond latency needs or micro-batch triggers with 1-second to 10-second intervals for balanced performance. Optimize checkpoint locations by using fast storage and configure appropriate checkpoint intervals to balance fault tolerance and performance overhead.	Achieves optimal balance between latency and throughput for real-time data processing. Allows consistent stream processing performance that can handle different data arrival rates while maintaining low end-to-end latency.
Deploy GPU-enabled clusters by using NC, ND, or NV-series VMs for deep learning model training and inference workloads. Configure appropriate GPU memory allocation and use MLflow for distributed training orchestration.	Accelerates model training by 10 to 100 times compared to CPU-only clusters by using parallel processing capabilities designed for machine learning operations. Reduces training time and allows faster model iteration cycles.
Select GPU instance types based on model complexity and training dataset size. Consider both memory capacity and compute performance requirements for your specific machine learning workloads.	

Azure policies

Azure provides an extensive set of built-in policies related to Azure Databricks and its dependencies. Some of the preceding recommendations can be audited through Azure Policy. For example, you can check whether:

- Azure Databricks workspaces use virtual network injection for enhanced network security and isolation.

- Azure Databricks workspaces block public network access when you use private endpoints.
- Azure Databricks clusters have disk encryption turned on to protect data at rest.
- Azure Databricks workspaces use customer-managed keys for enhanced encryption control.
- Azure Databricks workspaces have diagnostic logging turned on for monitoring and compliance.
- Azure Databricks workspaces are only deployed in approved geographic regions for compliance.
- Enterprise workloads use Azure Databricks Premium tier for enhanced security and compliance features.
- Azure Databricks workspaces use Unity Catalog for centralized data governance.

For comprehensive governance, review the [Azure Policy built-in definitions for Azure Databricks](#) and other policies that might affect the security of the analytics platform.

Azure Advisor recommendations

Azure Advisor is a personalized cloud consultant that helps you follow best practices to optimize your Azure deployments.

For more information, see [Azure Advisor](#).

Trade-offs

You might have to make design trade-offs if you use the approaches in the pillar checklists.



Analyze performance and cost trade-offs

Balance performance and cost to get the most value from your workloads. Overprovisioning wastes money, while underprovisioning can slow down workloads or lead to failures. Test different configurations, use performance benchmarks, and analyze costs to guide your choices.

Scenario architecture

[Stream processing by using Azure Databricks](#) shows a foundational architecture that demonstrates the key recommendations described in this article.

Related content

- [Azure Databricks documentation](#)
 - [Introduction to the well-architected data lakehouse](#)
-

Last updated on 02/11/2026